Microsoft

PowerApps Bootcamp: Basic

Hands-On Lab Guide

Published (April 2026)

Disclaimer

The content, code samples, and guidance provided in this lab document (the "Hands-On Lab Guide") are for informational and educational purposes only. The Materials are provided "as is" and reflect general practices and platform capabilities at the time of publication. While reasonable efforts have been made to ensure accuracy, no representation or warranty, express or implied, is made as to the completeness, accuracy, reliability, or suitability of the Materials for any purpose.

The Materials are not intended to be used as-is in production environments. It is the sole responsibility of the user to evaluate and test all solutions, configurations, and code examples in a controlled, non-production environment prior to any implementation in a live setting. Use of these Materials in a production environment is done entirely at the user's own risk.

No warranties or guarantees of any kind are provided, including but not limited to warranties of merchantability, fitness for a particular purpose, non-infringement, or the absence of errors or defects. Under no circumstances shall the authors, contributors, or any affiliated parties be held liable for any damages, including but not limited to direct, indirect, incidental, consequential, or special damages, arising out of or in connection with the use of the Materials.

By using or adapting any portion of the lab document in any capacity, including in production environments, you expressly acknowledge and agree that you are assuming full responsibility and liability for such use, and that you release the authors and affiliated parties from any and all claims or liabilities that may arise as a result.

Contents

Exercise 1: Preparing a Microsoft List

In this exercise, we will prepare the SharePoint List “Request Tracker” that will be used as the main data source for the Request Tracker Application.

Objectives

After completing this lab, you will be able to:

- Create a Microsoft List to store data

Requires

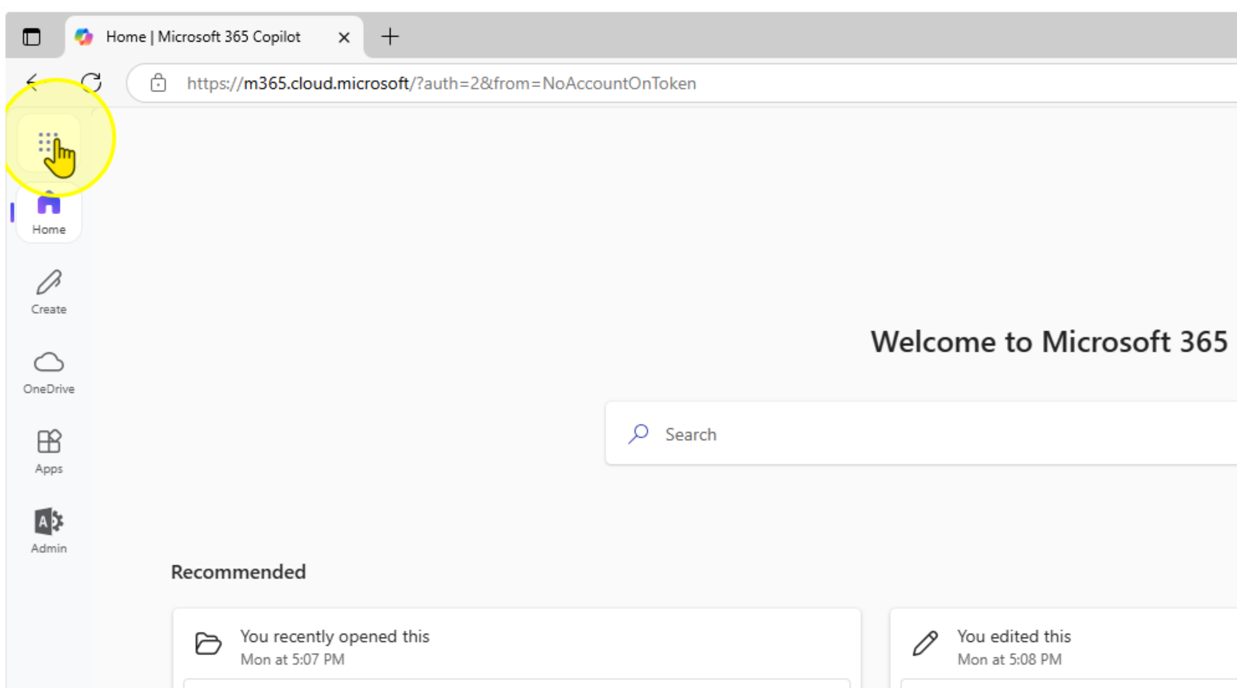
- “Request-Sample-Data.xlsx”

Estimated time to complete this lab

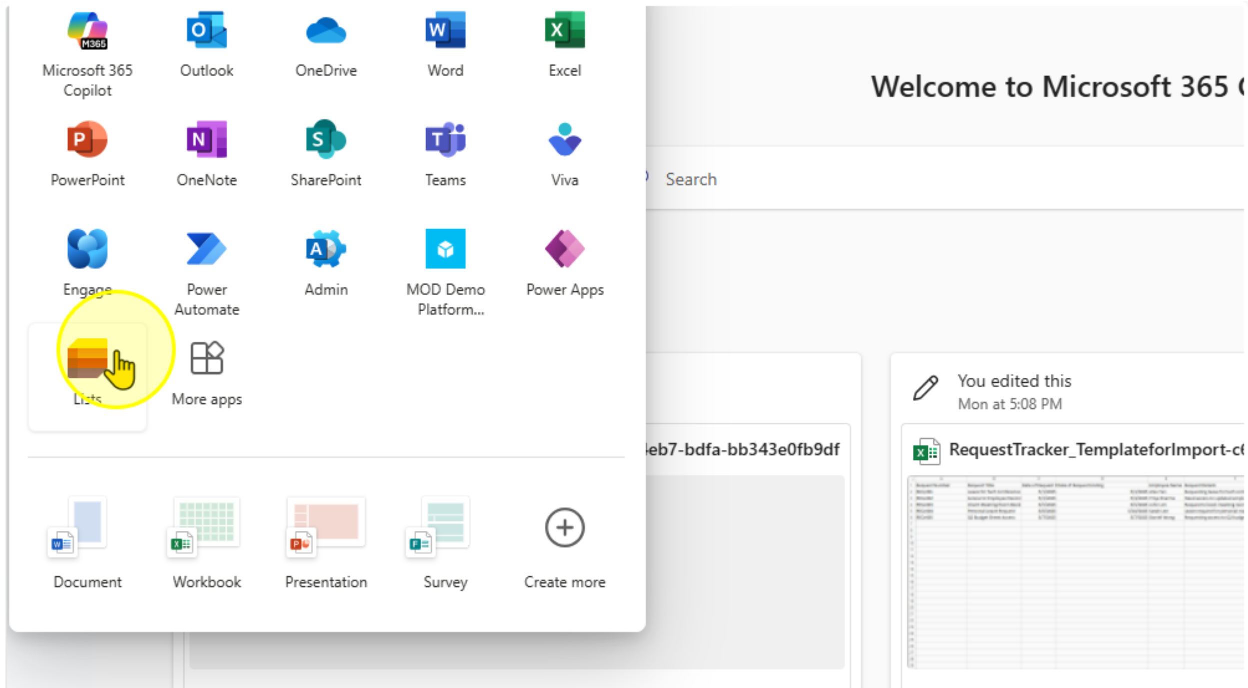
30 mins

Task 1: Creating a Microsoft List

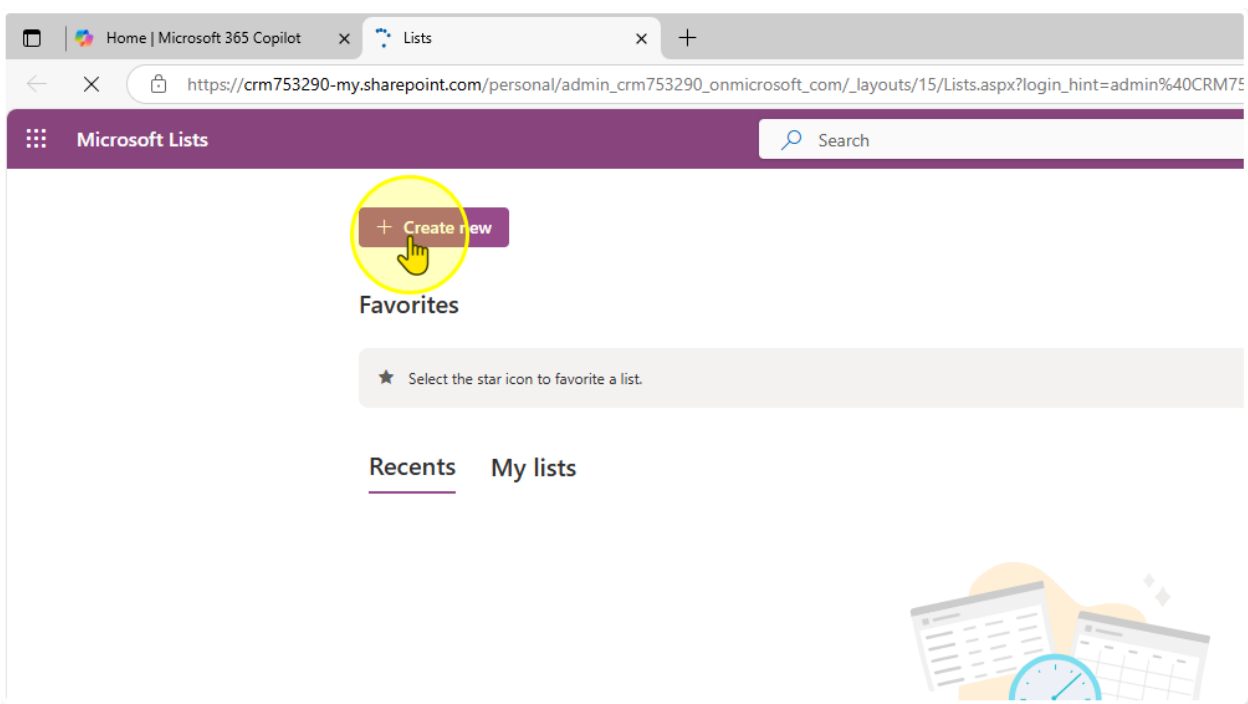
1. Click on the App Laucher (Link)



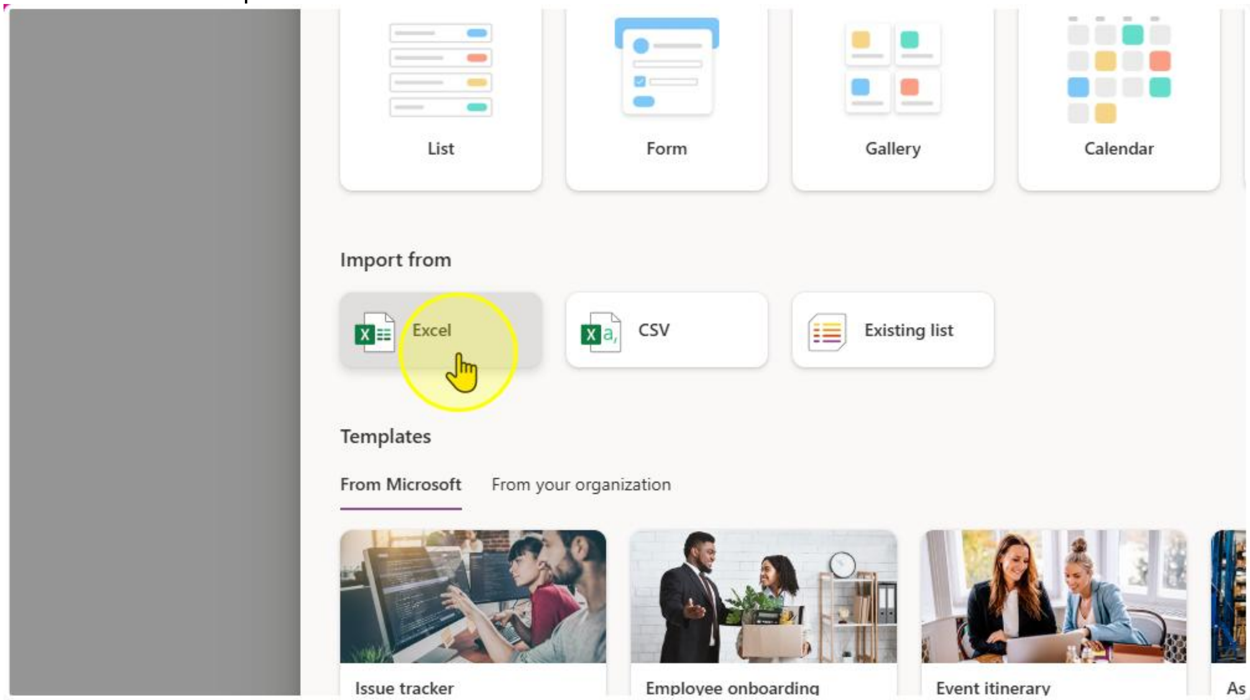
2. Select Lists



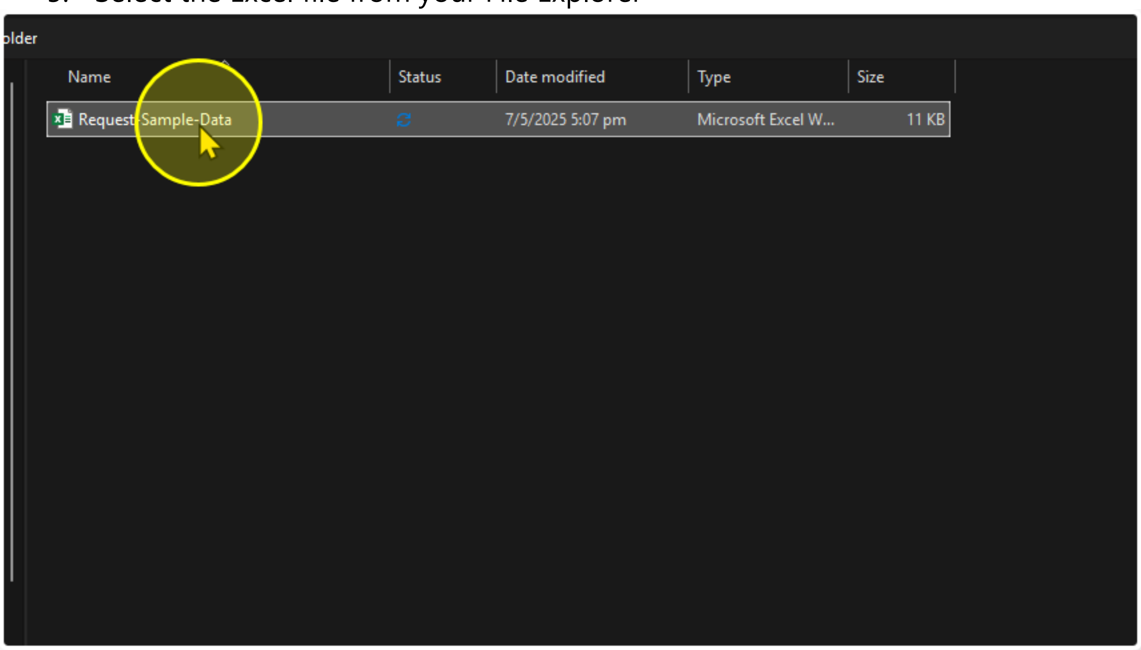
3. Create a New List



4. Click on import from excel

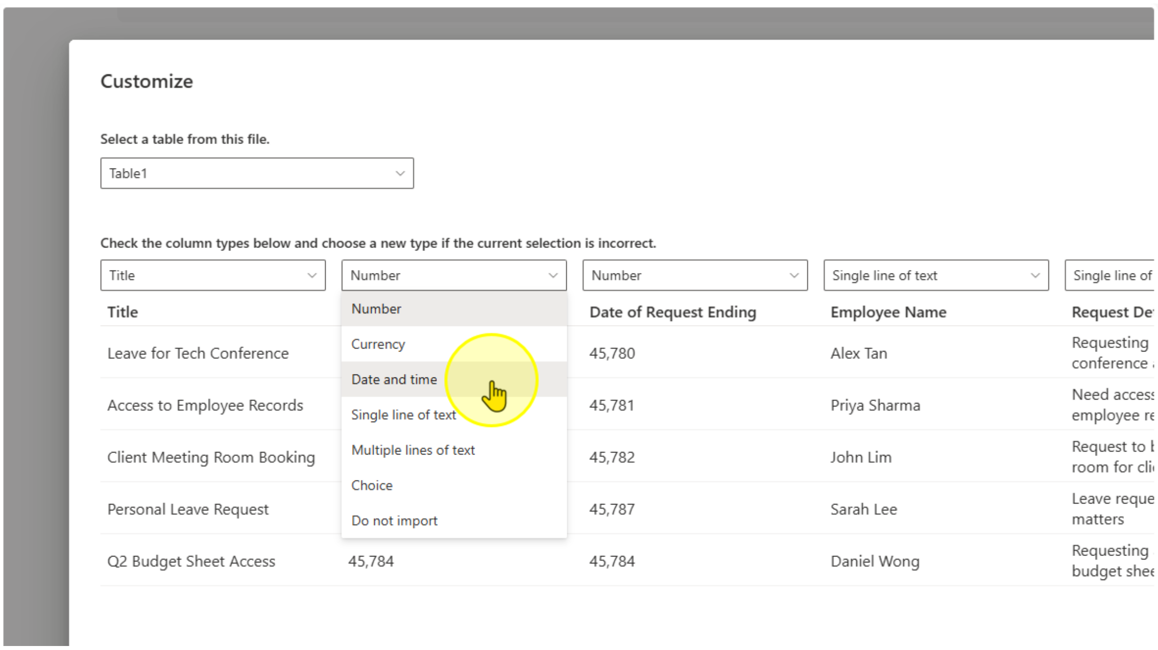


5. Select the Excel file from your File Explorer



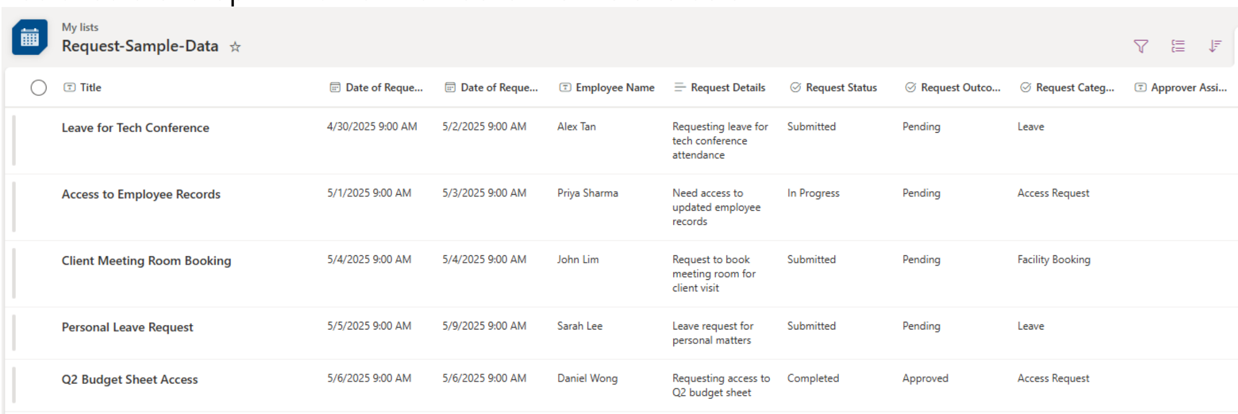
6. On the "Customize" Panel, you should see the columns that will be created that Lists is referencing from the excel file. Make sure you change the following Column and their corresponding Column Type.

Column Name	Column Type
Title	Title
Date of Request Starting	Date and Time
Date of Request Ending	Date and Time
Employee Name	Single Line of Text
Request Details	Multiple Lines of text
Request Status	Choice
Request Outcome	Choice
Request Category	Choice
Approver Assigned	Single Line of Text



7. Once completed, Click “Next” and then “Create”

You should end up with a List that looks like the one below.



My lists

Request-Sample-Data

Title	Date of Reque...	Date of Reque...	Employee Name	Request Details	Request Status	Request Outco...	Request Categ...	Approver Assi...
Leave for Tech Conference	4/30/2025 9:00 AM	5/2/2025 9:00 AM	Alex Tan	Requesting leave for tech conference attendance	Submitted	Pending	Leave	
Access to Employee Records	5/1/2025 9:00 AM	5/3/2025 9:00 AM	Priya Sharma	Need access to updated employee records	In Progress	Pending	Access Request	
Client Meeting Room Booking	5/4/2025 9:00 AM	5/4/2025 9:00 AM	John Lim	Request to book meeting room for client visit	Submitted	Pending	Facility Booking	
Personal Leave Request	5/5/2025 9:00 AM	5/9/2025 9:00 AM	Sarah Lee	Leave request for personal matters	Submitted	Pending	Leave	
Q2 Budget Sheet Access	5/6/2025 9:00 AM	5/6/2025 9:00 AM	Daniel Wong	Requesting access to Q2 budget sheet	Completed	Approved	Access Request	

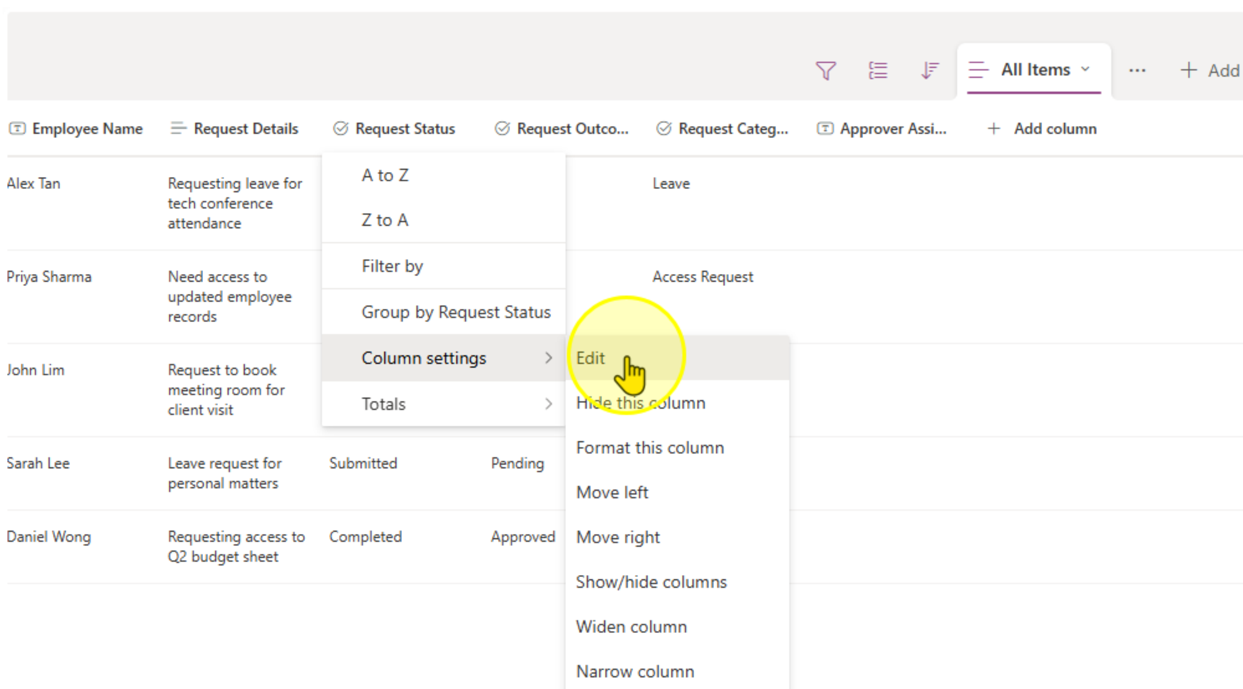
-- **End of Task** --

Task 2: Adding Choices on the List

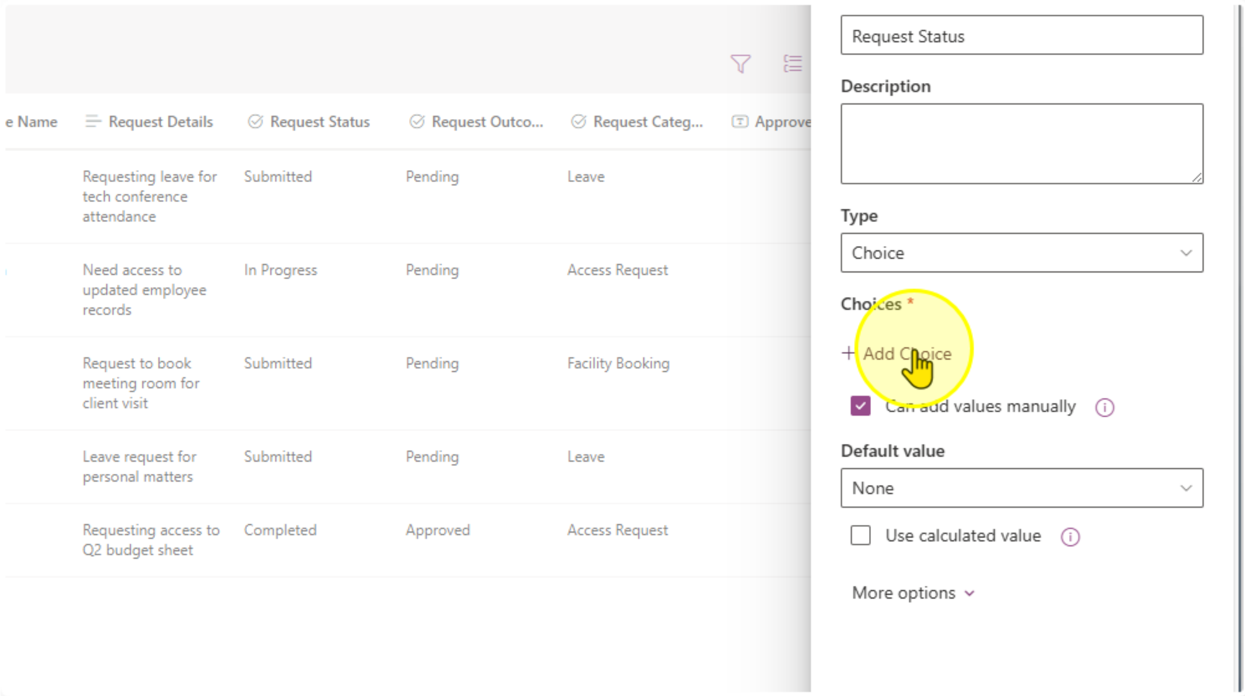
1. Go to the list and configure the 3 columns to the values in the table below

Column Name	Choice 1	Choices 2	Choice 3	Choice 4	Default Choice	Can add Values Manually?
Request Status	Pending	Completed	-	-	Pending	No
Request Outcome	Pending	Approved	Rejected	Request for more information	Pending	No
Request Category	Leave	Access Request	Facility Booking	Submission	-	Yes

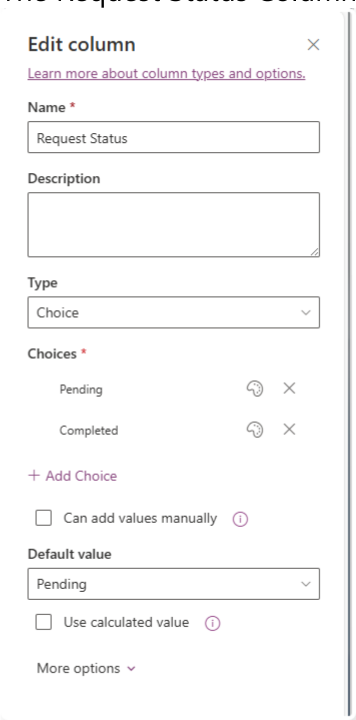
To configure Column Settings



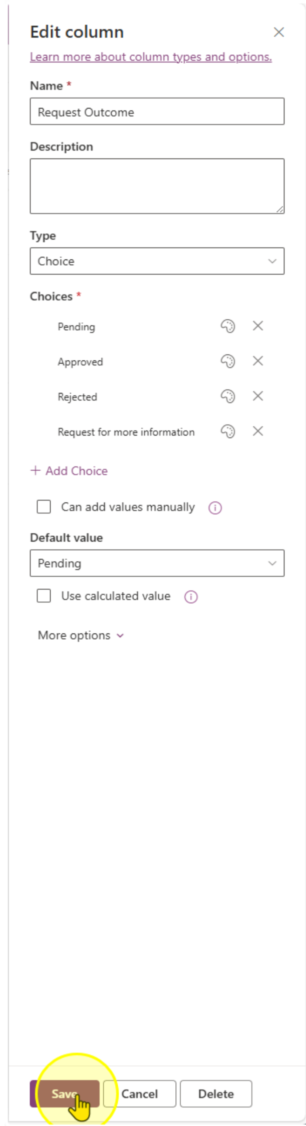
Add the choices for the Respective Choice Column here



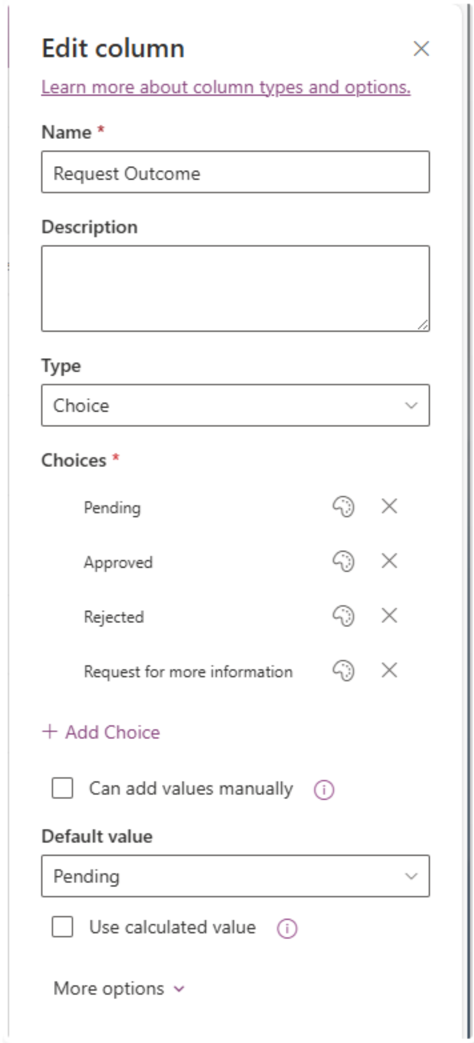
The Request Status Column should look like this



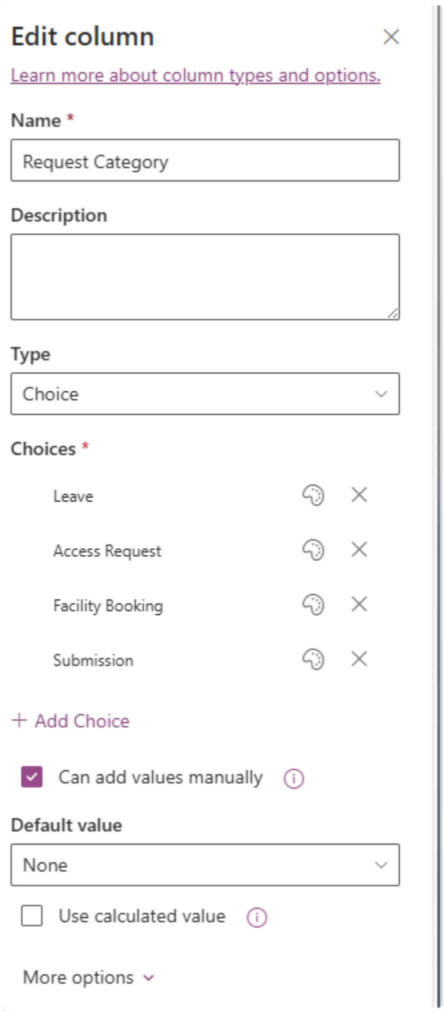
Ensure you hit the “Save” Button to save your changes



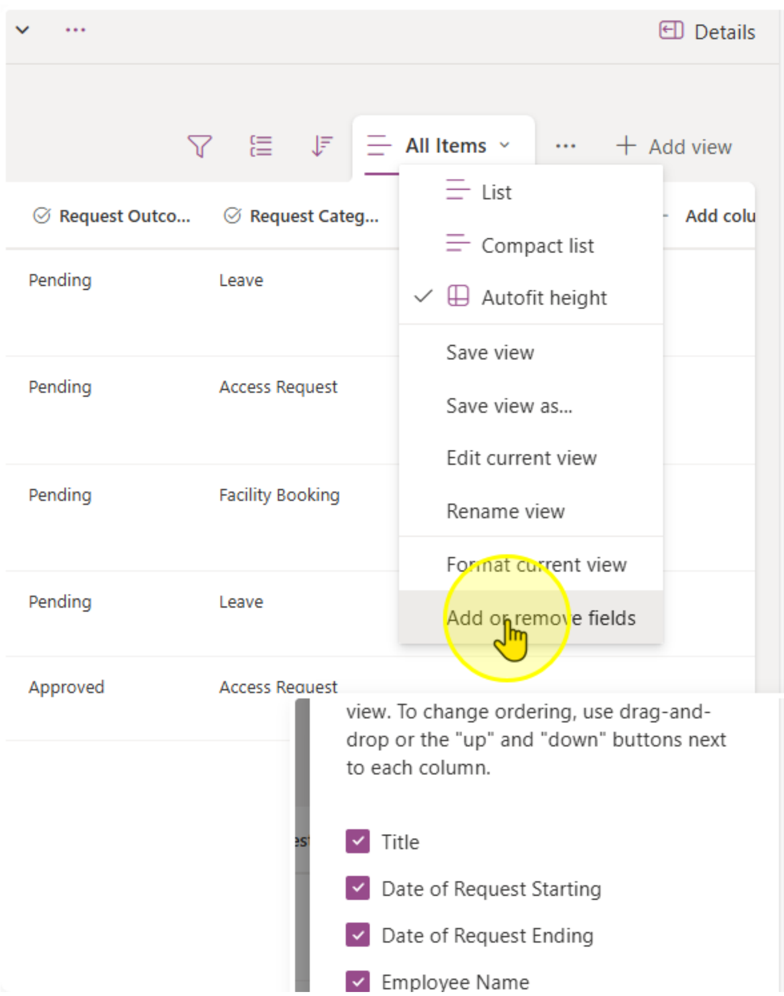
2. Repeat for the rest of the columns. The Request Outcome Column should look like this



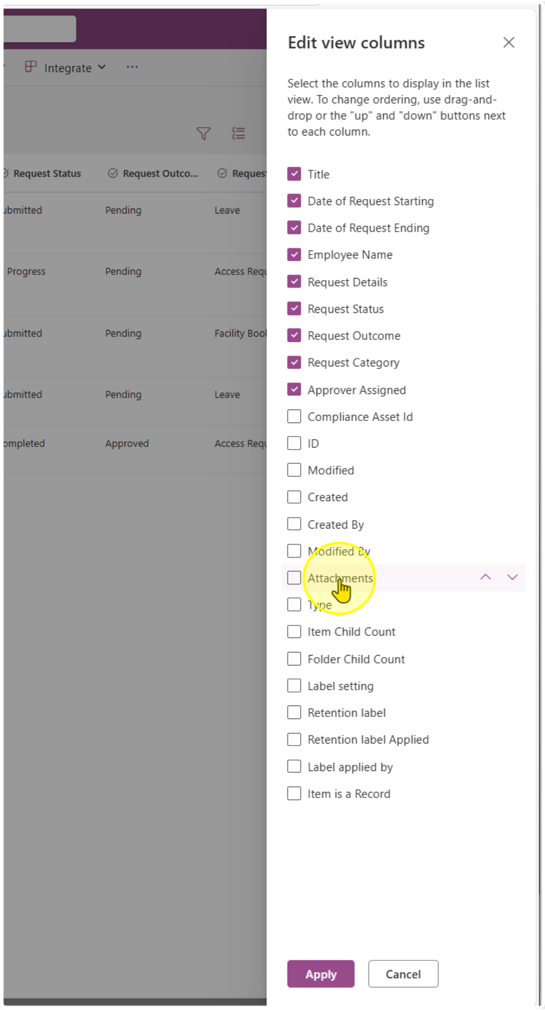
The Request Category Column should look like this



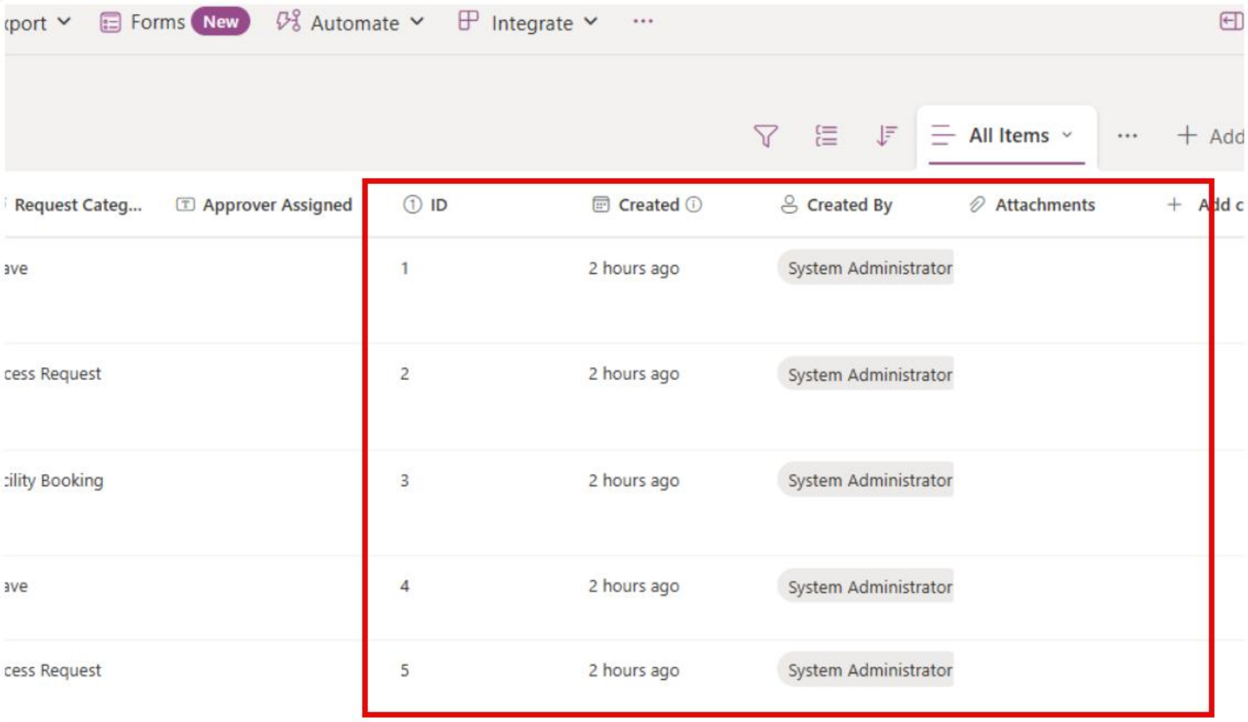
3. Once you have finished creating the list, take a look at all of the columns you have in the list. Microsoft Lists come with a set of pre-configured columns.



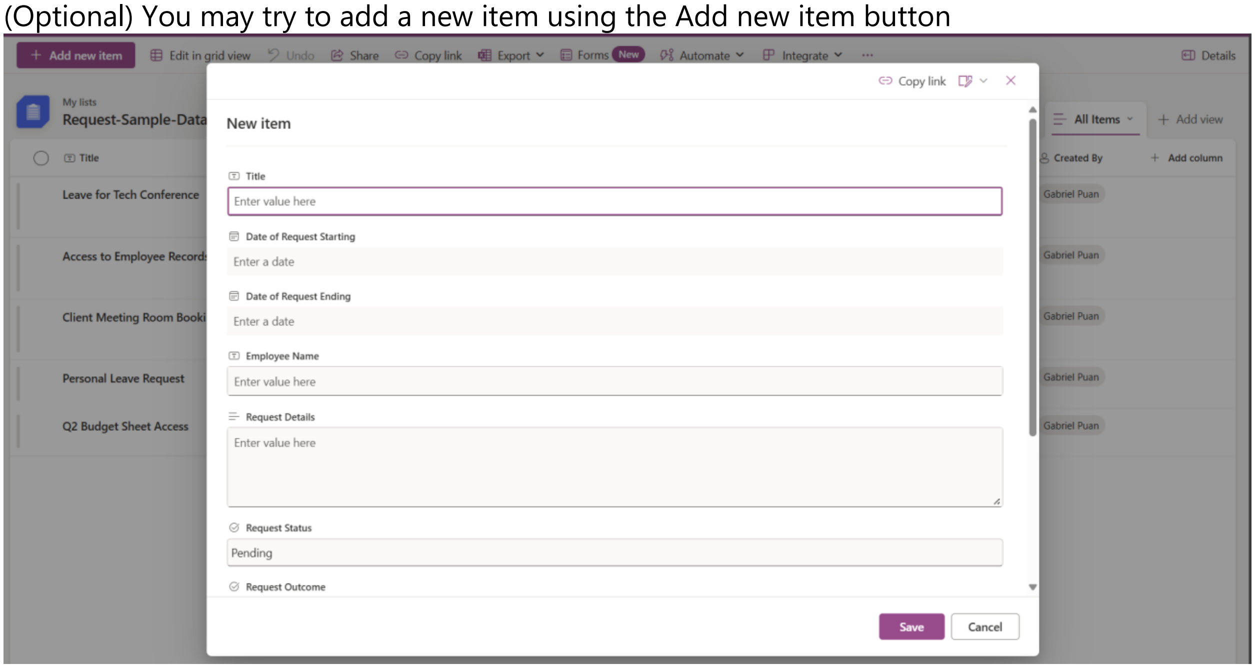
See the full list here. You can tick the checkboxes to see all columns in the list view



Notice how some columns are automatically populated



(Optional) You may try to add a new item using the Add new item button



--- **End of Task** ---

Exercise 2: Creating the Request App Header

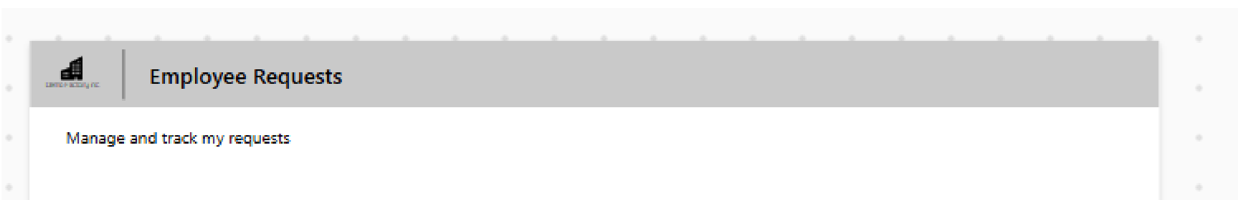
In this next exercise, we will create the “Request App” Header. It will be a App that we can pin on Microsoft Teams, SharePoint, or on the web for people to:

- Create a new Request
- Filter and Sort through Requests by Request

Objectives

After completing this lab, you will be able to:

- Rename screens in Power Apps
- Add and manipulate the label control in Power Apps



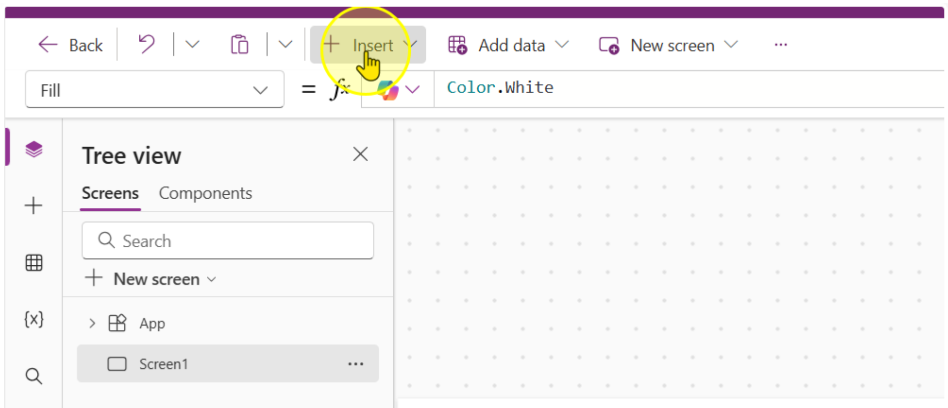
Estimated time to complete this lab

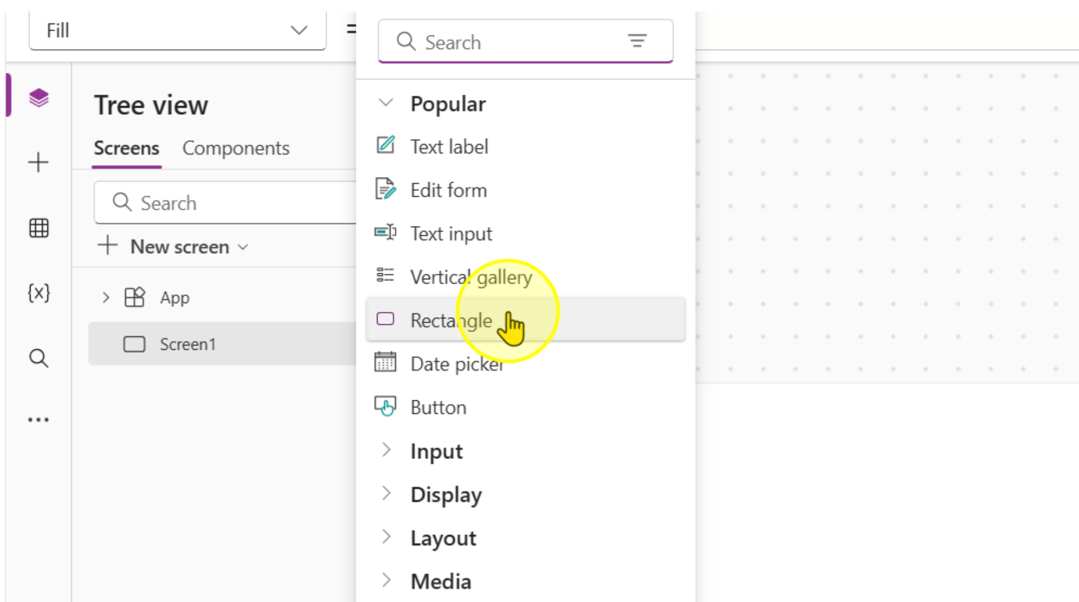
10 mins

Task 1: Building the app header

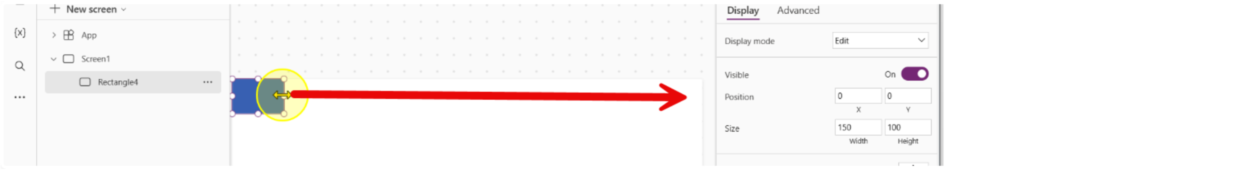
In this Task you will learn how to add a header to the app for branding and identification purposes.

1. Start off by learning how to create a new control. Click the insert button and add in a rectangle control. (Link to PowerApps)



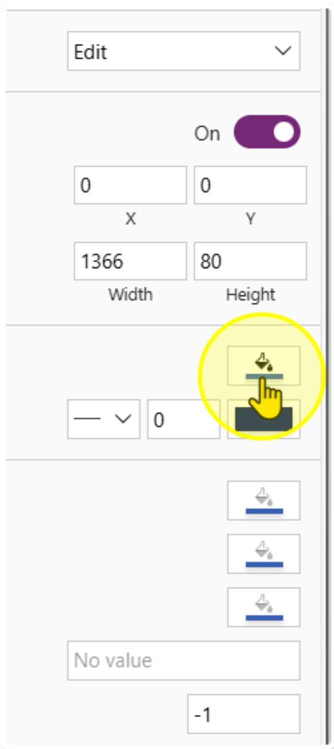


2. Drag the rectangle all the way to the end so that it occupies the full screen like this.

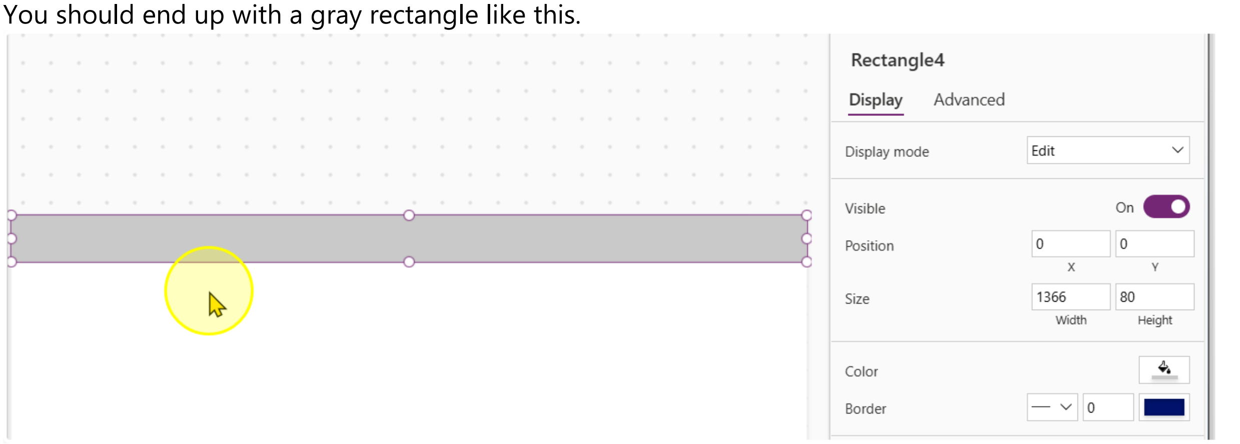


Adjust the following properties of the control to the corresponding values

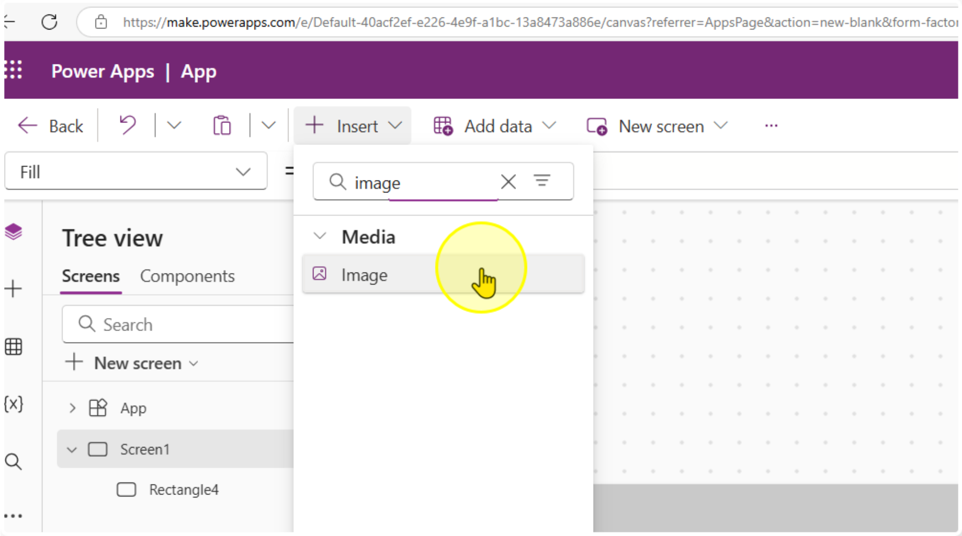
- Height: 80
- Width: 1366 (640 for Phone Size UI)
- Color: #C9C9C9



You should end up with a gray rectangle like this.



3. Next, insert an image control



Adjust the following properties of the control to the corresponding values

- Height: 80
- Width: 80

4. Fill the image property with the following link in between double inverted commas.

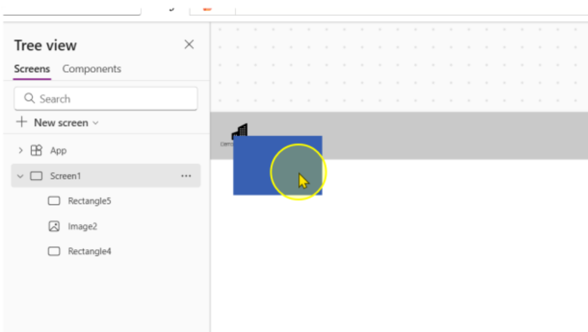
"https://raw.githubusercontent.com/KwangEng/PP-Bootcamp-Material/refs/heads/main/Day%201%20Power%20Apps/Demo%20Factory-Dark.svg"

OR

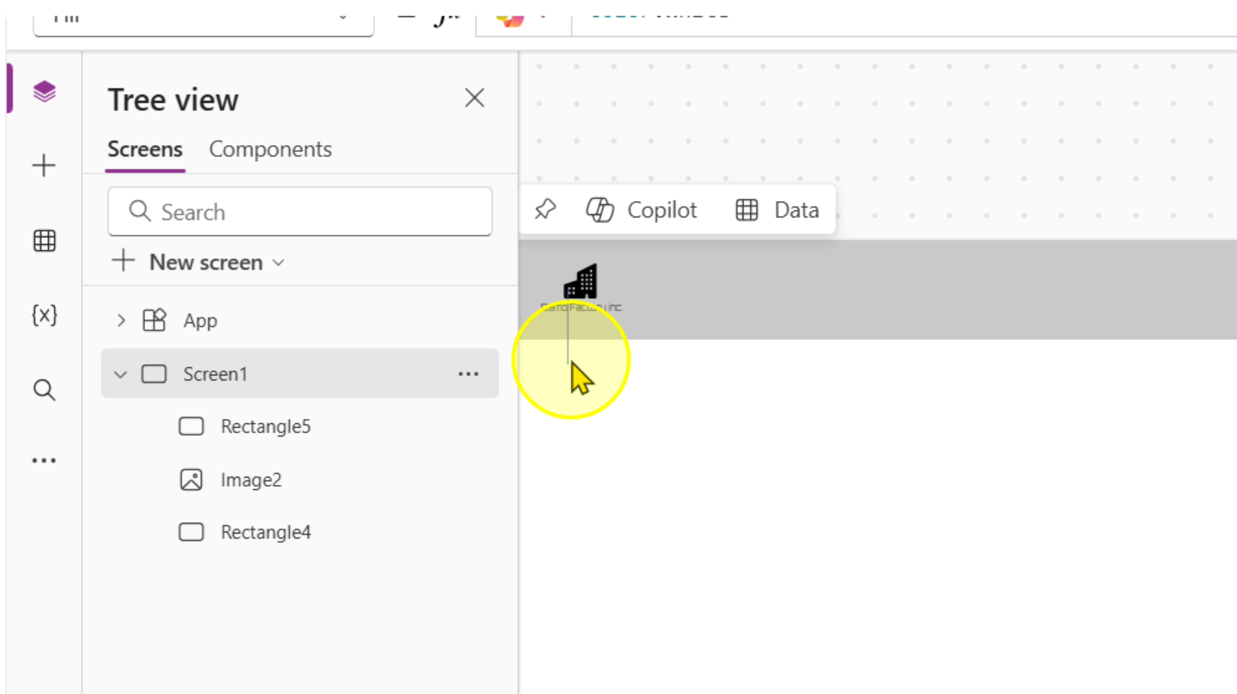
"https://ppbootcamp.blob.core.windows.net/shared/Demo%20Factory-Dark.svg"

Notice how this loads a logo. Position the logo so that it is in the top left hand position of the rectangle

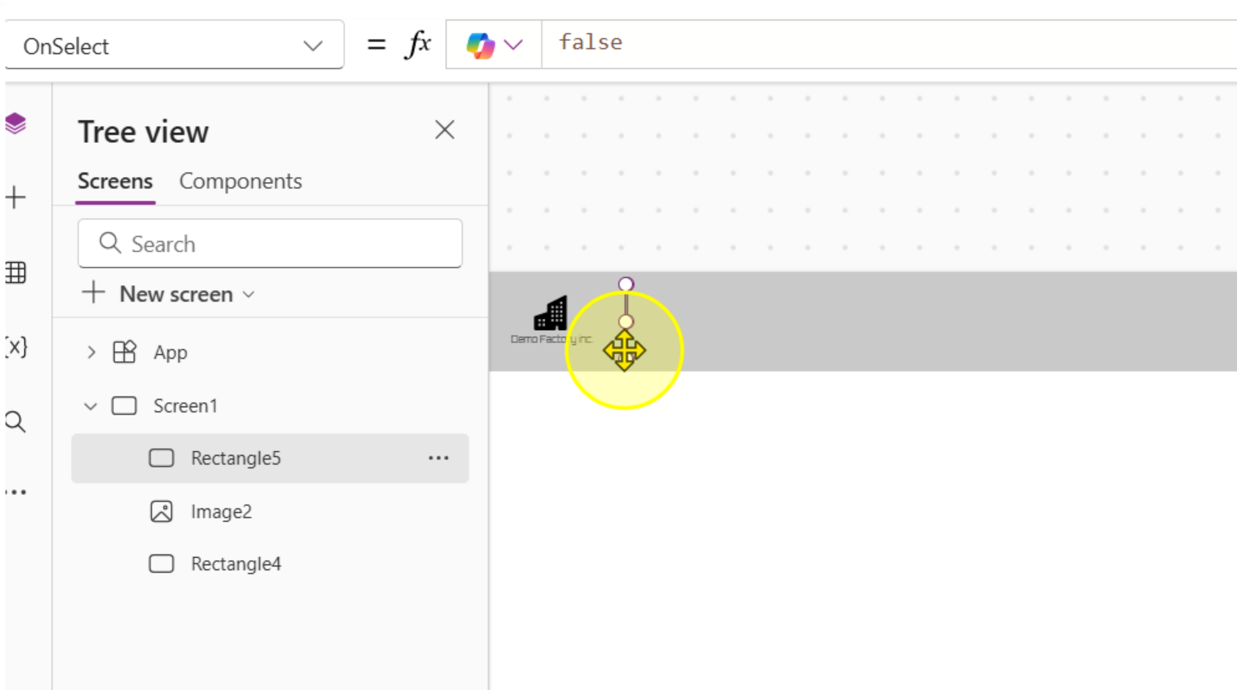
5. Add in another rectangle control



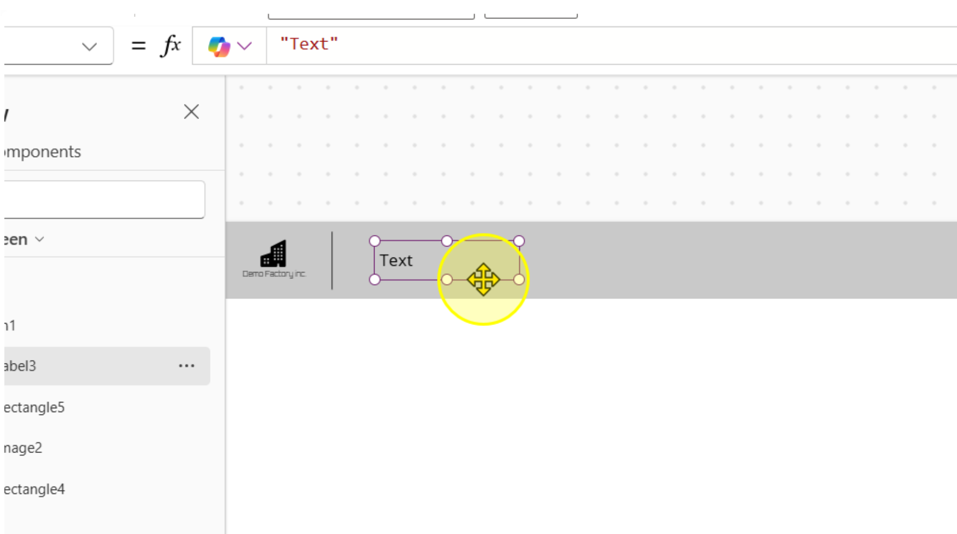
6. Change the following properties of the control to the corresponding values
   - Width: 1
   - Height: 60
   - Color: RGBA(0, 0, 0, 1)



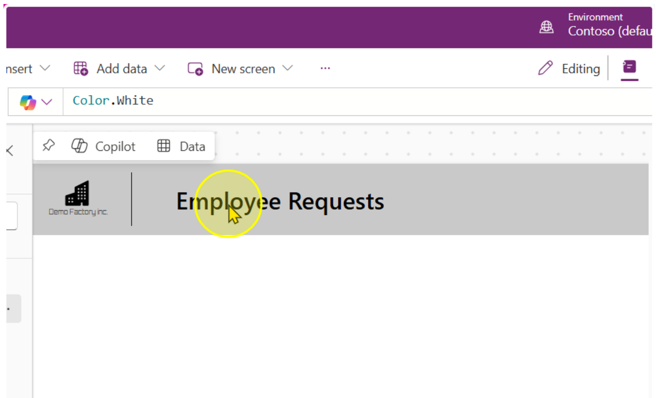
7. Position it so that it acts like a divider



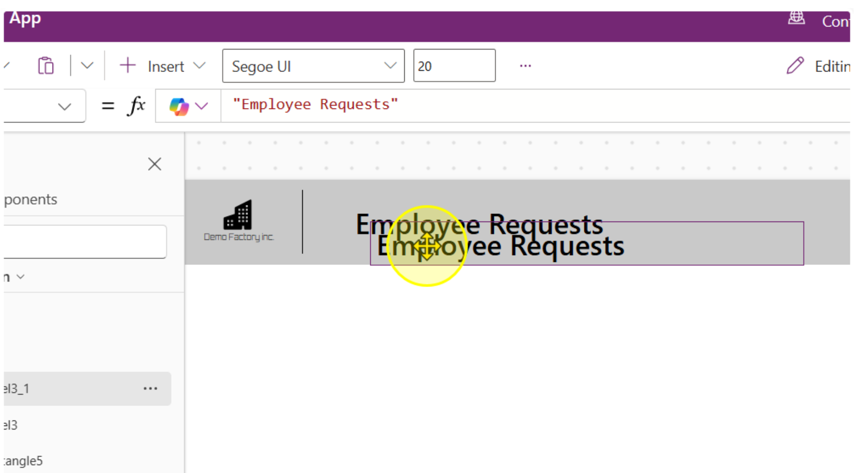
8. Add in a text label. Change the Text property to "Employee Requests", the font to "Segoe UI", the font size to 20 and the font weight to Semibold



You should end up with a text label that looks like this.

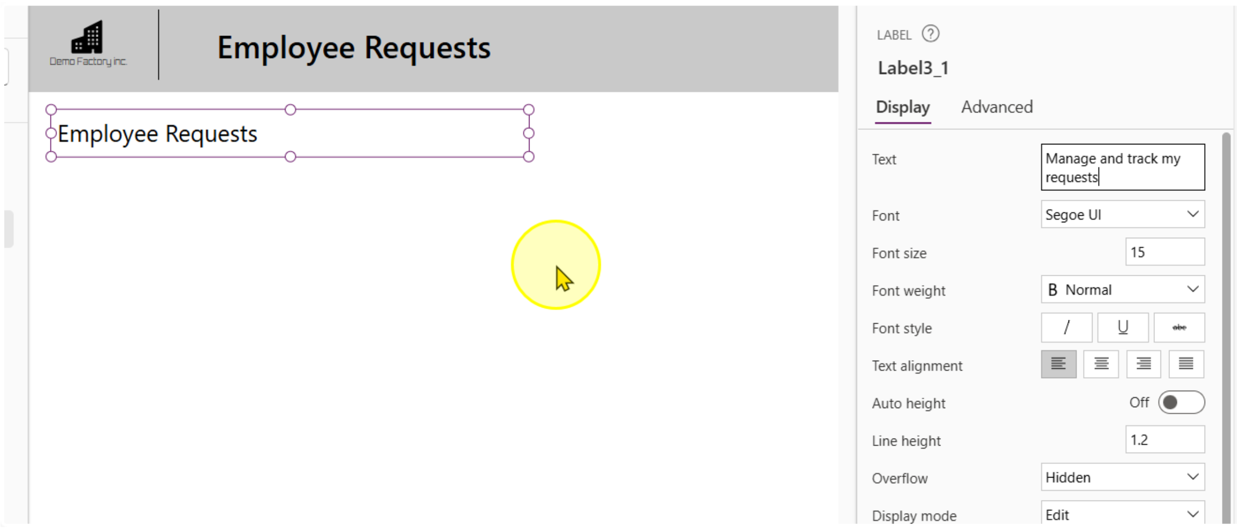


9. Copy and paste the Text Label to duplicate it

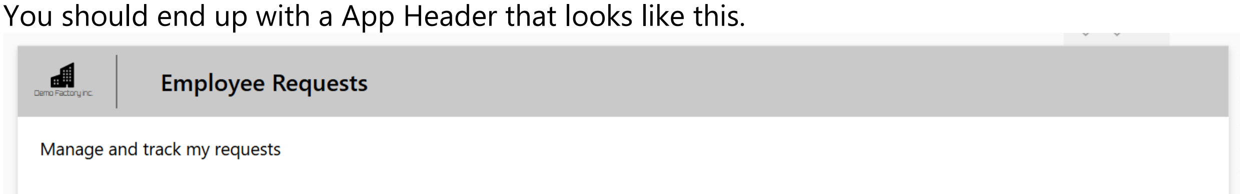


10. Change the Following properties of the text label

- Text: "Manage and track my requests"
- Font: Segoe UI
- Font Size: 15
- Font Weight: Normal



You should end up with a App Header that looks like this.

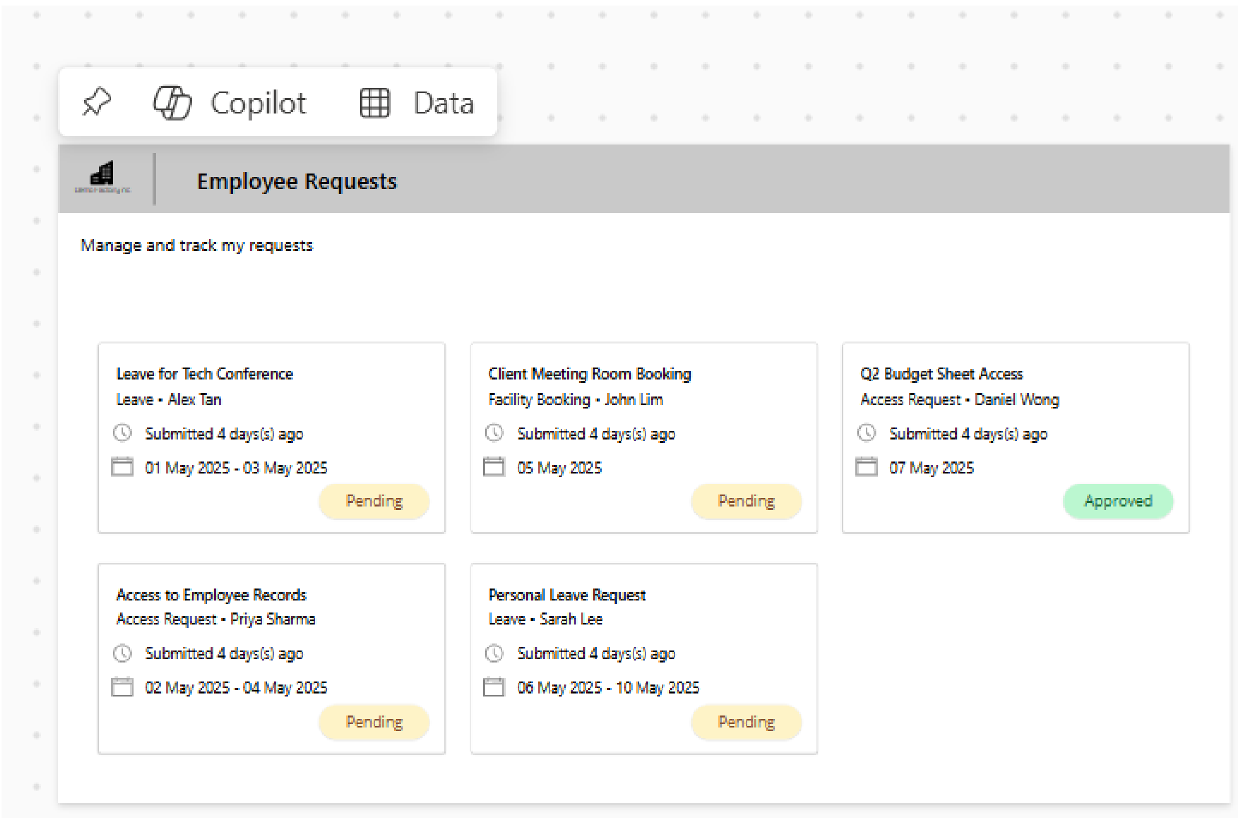


Important! Remember to Save the App.

--- **End of Task** ---

Exercise 3: Connecting to Data and Displaying it

In this exercise, we will be displaying data in the app from the List that we created in exercise 1. We will then customize the look and feel of the data being displayed to create a nice UI format.



Objectives

The exercise will achieve the following objectives.

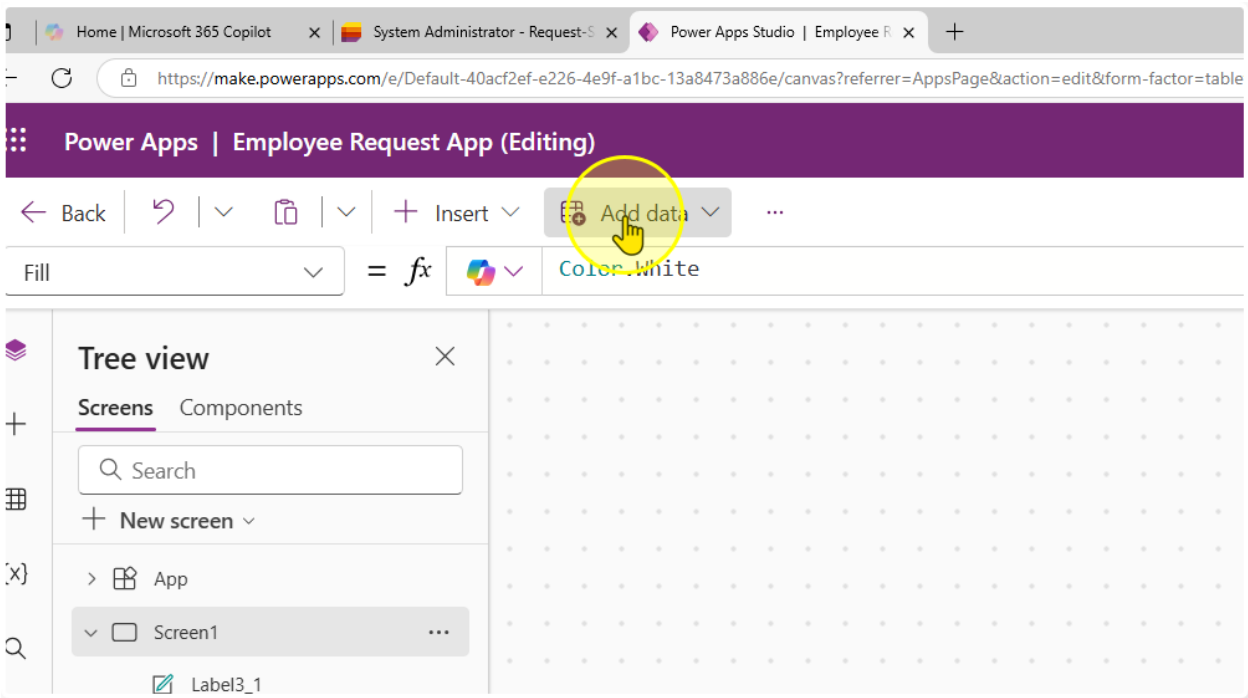
- Learn how to connect to a Microsoft List
- Display data in a Gallery control
- Change how data is display data in a Gallery control

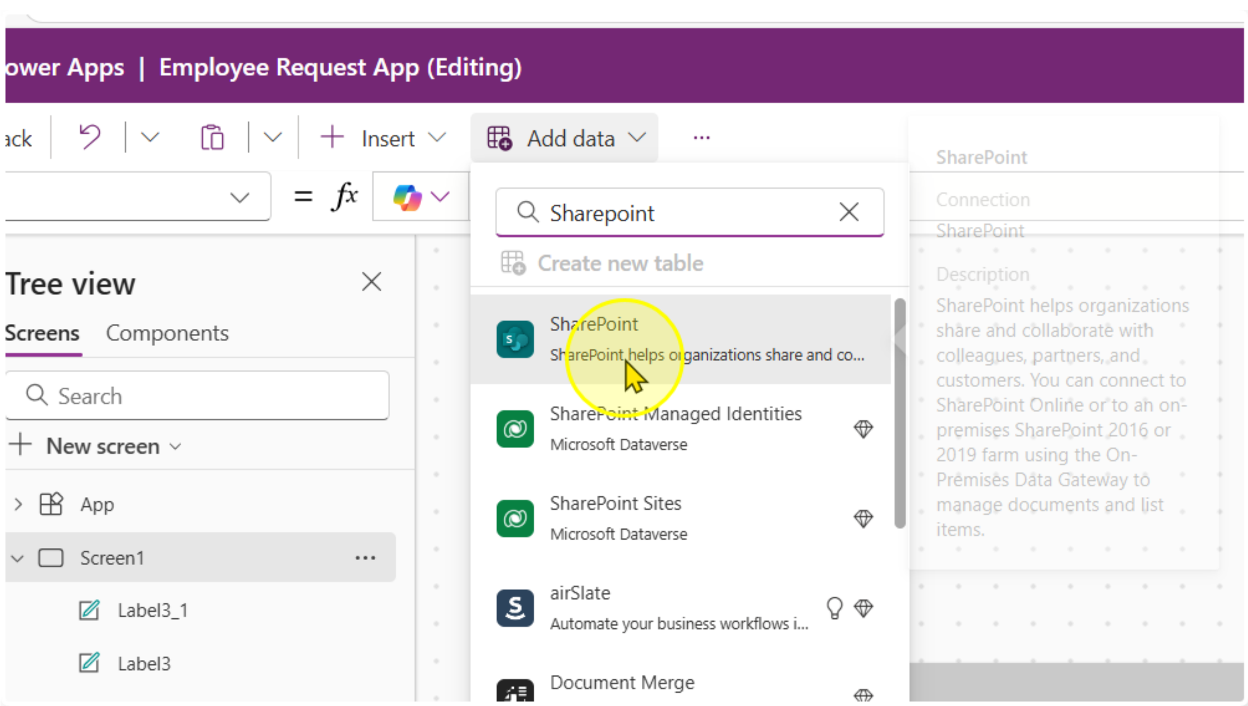
Estimated time to complete this lab

70 mins

Task 1: Connecting to the Microsoft List

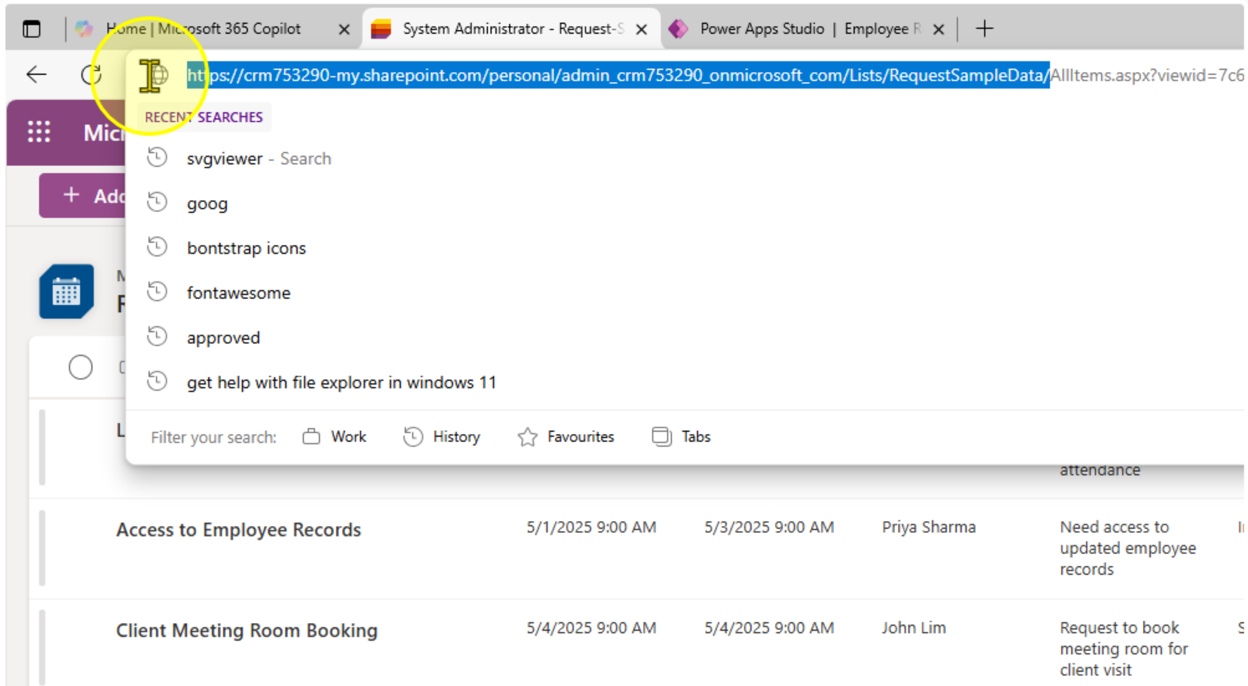
1. Click on add data and locate the SharePoint connector
   (Note: Do not use SharePoint Sites, Dataverse)





2. Click on Connect

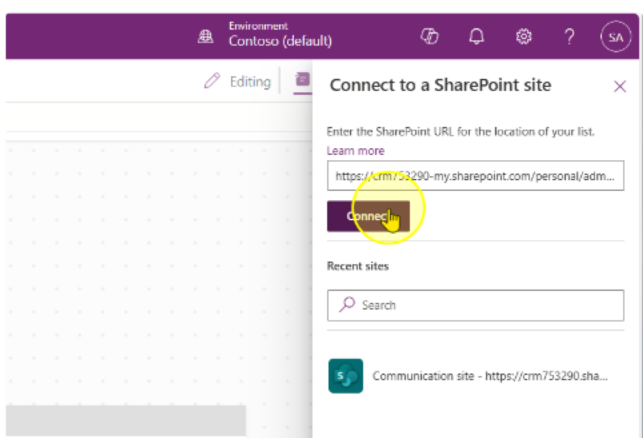
3. Find the URL of your list. Notice how the URL is structured as https://<OrgName>.sharepoint.com/personal/<Your Name + Company Domain + com>/Lists/<Your List Name>/...



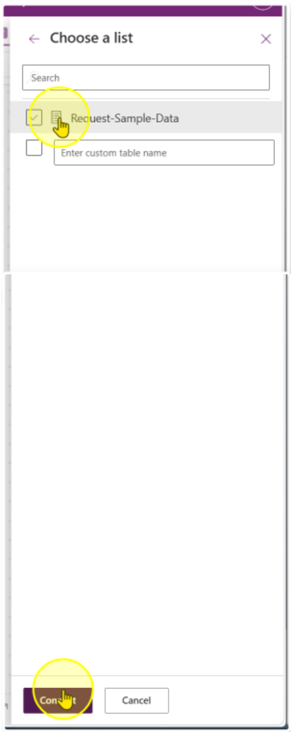
For instance if your organization is Contoso, and your email id is adam@contoso.com and your List is called 'My First List', your URL should look like:

https://contoso.sharepoint.com/personal/adam_contoso_com/Lists/My_First_List/...

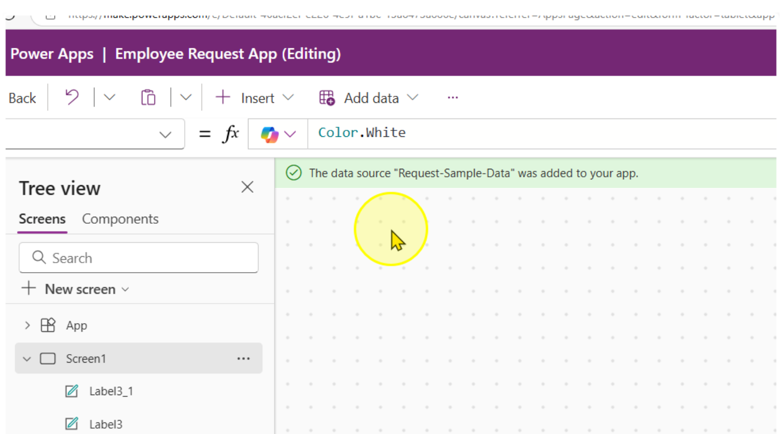
4. Copy that URL and paste it into the Text Input box. Click on Connect.



5. You should see your list you created here. Click on the list, then click Connect.



6. Observe that a Banner showing your data source name appears.

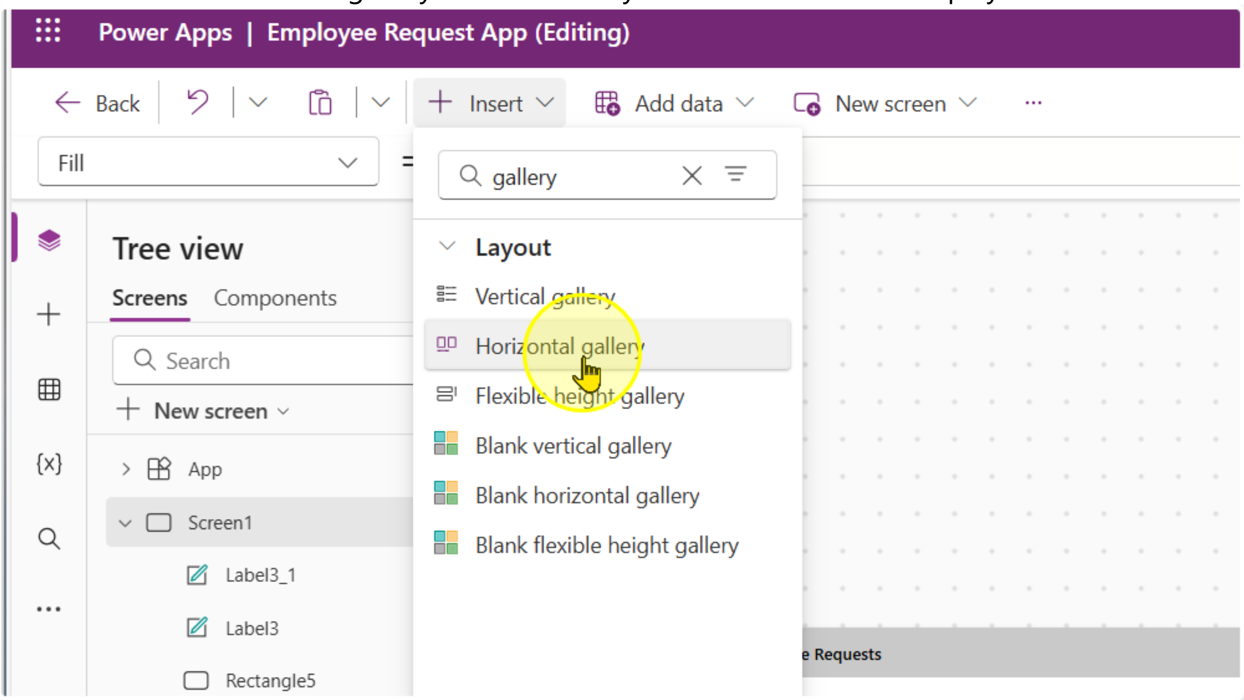


--- **End of Task** ---

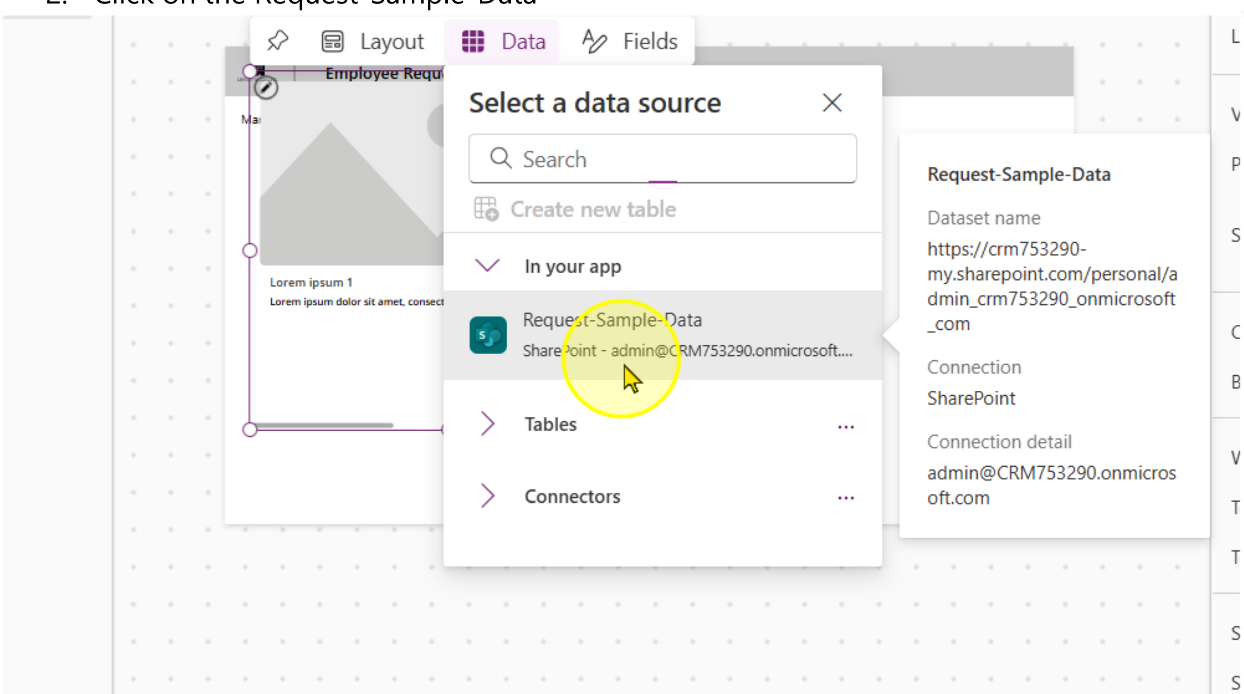
Microsoft Power Platform

Task 2: Displaying the Data in a Gallery Control

1. Add in a horizontal gallery control. Gallery controls are used to display data

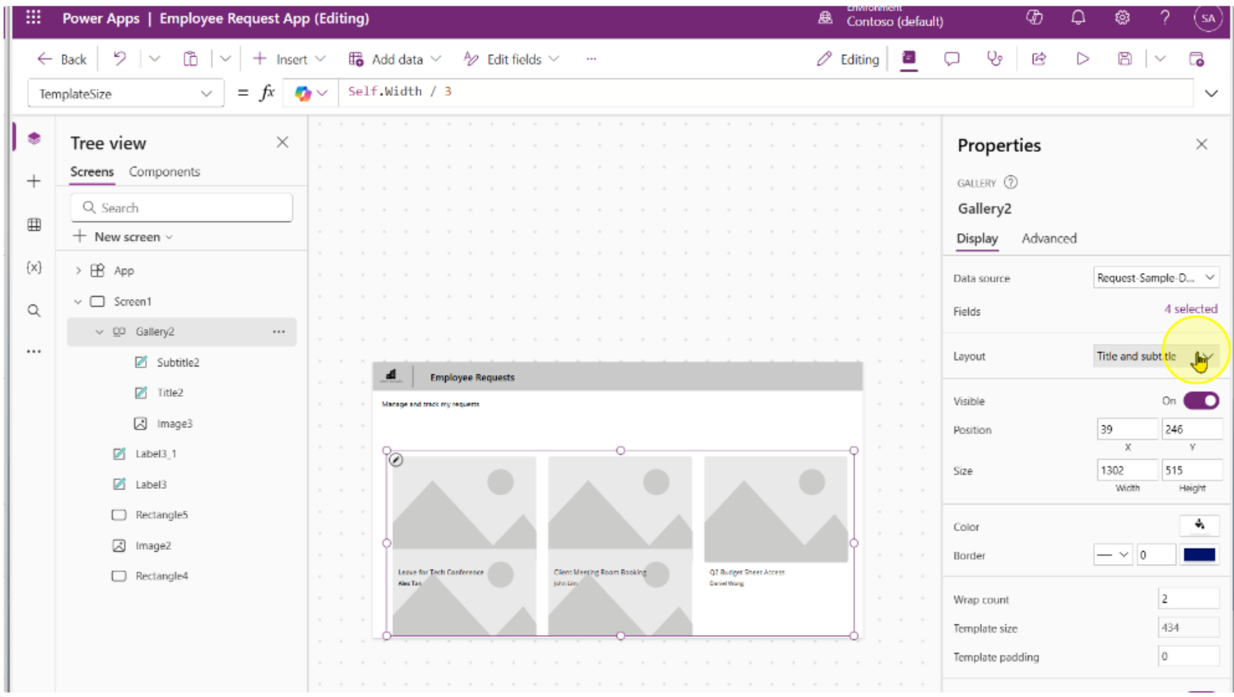


2. Click on the Request-Sample-Data

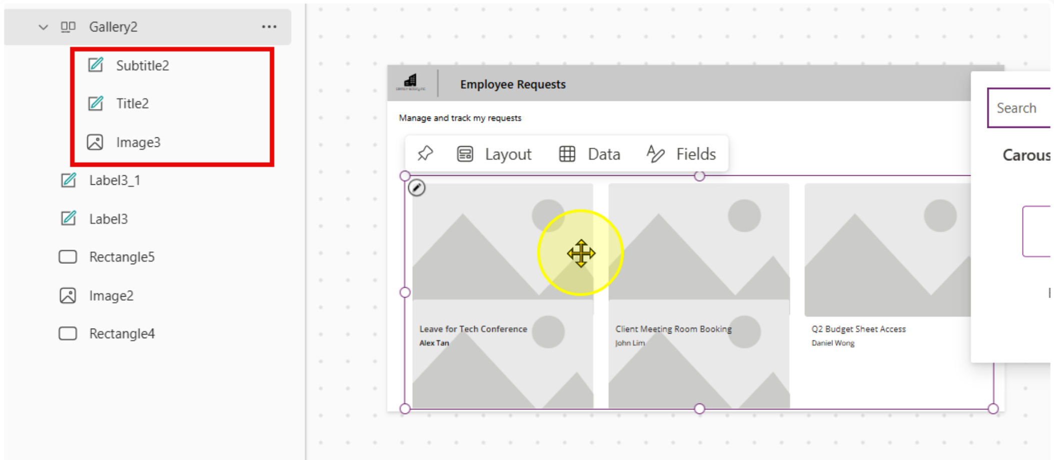


3. Expand the gallery to cover most of the screen like in the image below. Additionally, change the following properties of the gallery

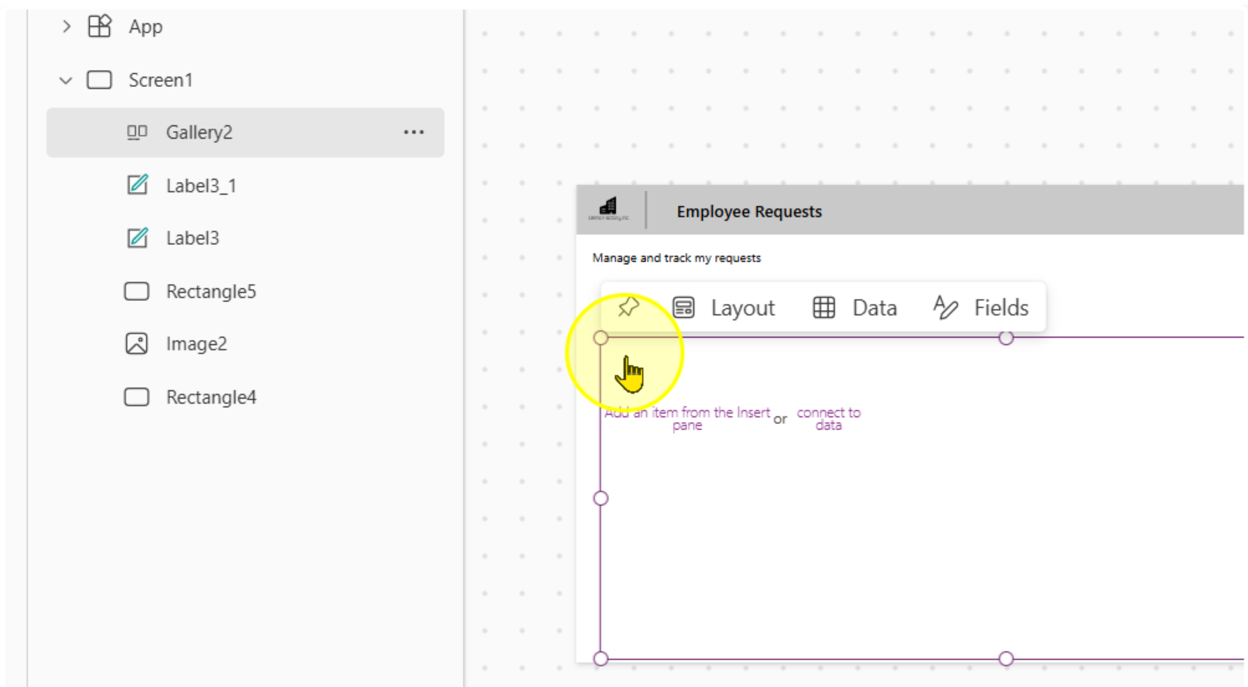
- Wrap count: 2
- Template size: Self.Width / 3
- Template Padding: 0



4. Take note of the controls in the gallery. Delete all of them.

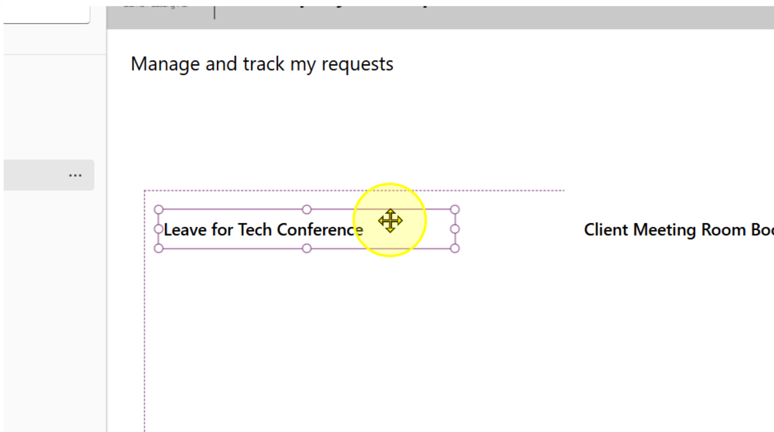


You should end up with a gallery that looks like this

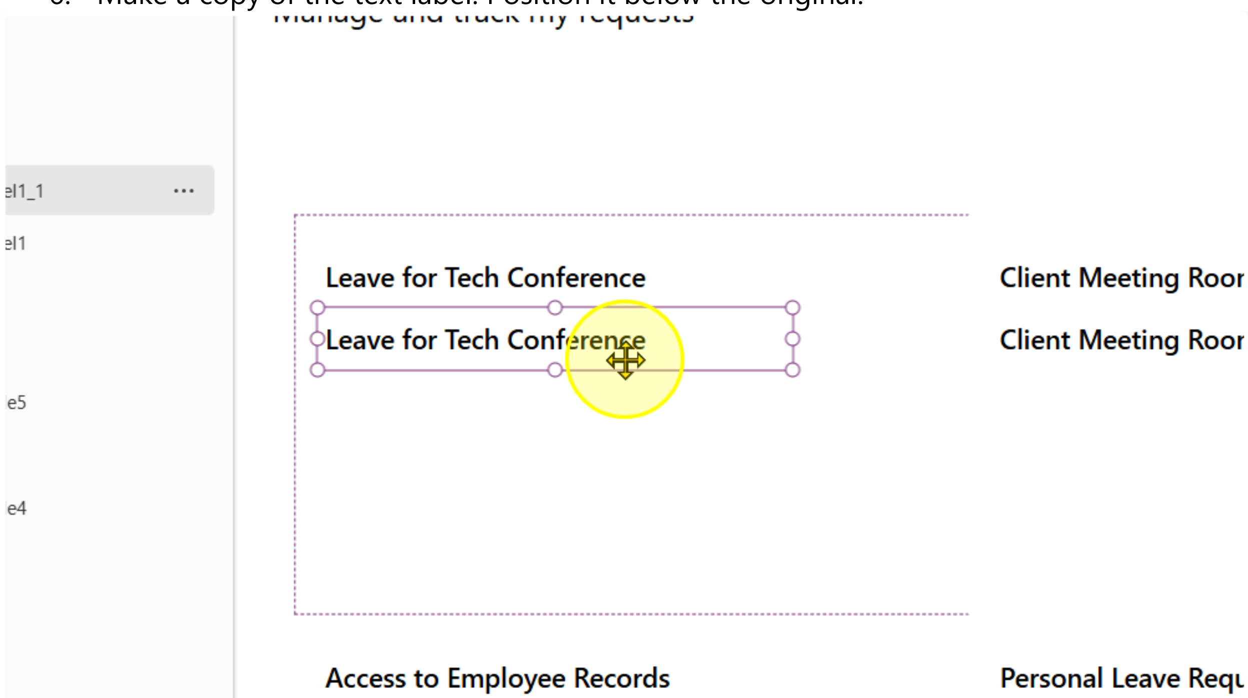


5. Add in a single text label. Position it at the top left hand of the Gallery Template. The recommended properties you should set it to are:

- Font: Segoe UI
- Font Weight: Semibold
- Font Size: 13
- Height: 50

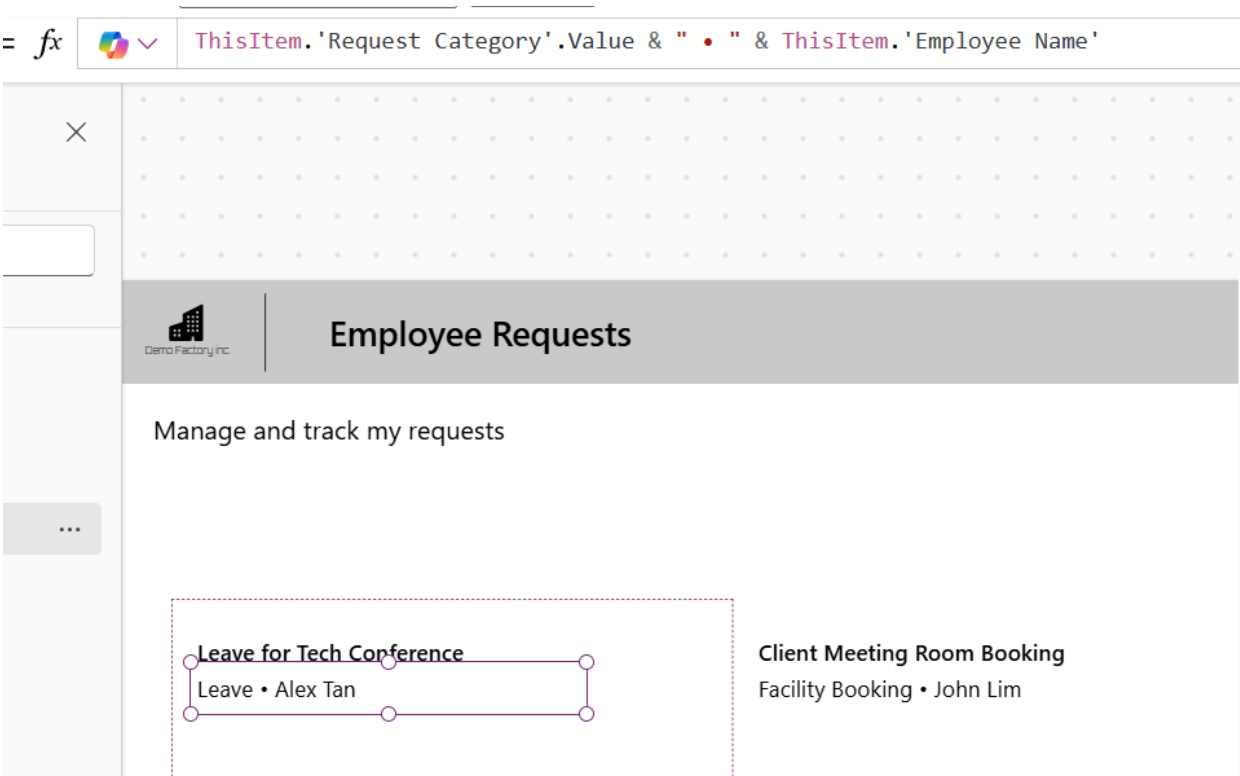


6. Make a copy of the text label. Position it below the original.



7. Change the font weight to Normal. For the text, write the following formula

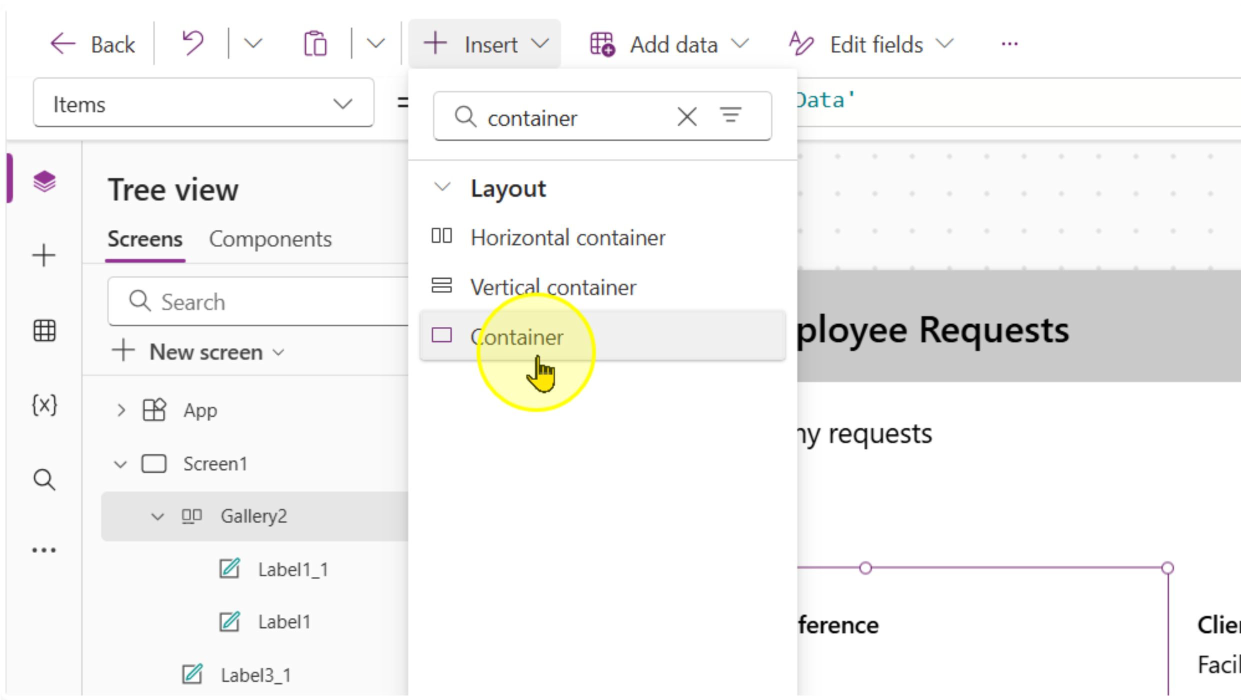
```
ThisItem.'Request Category'.Value & " • " & ThisItem.'Employee Name'
```



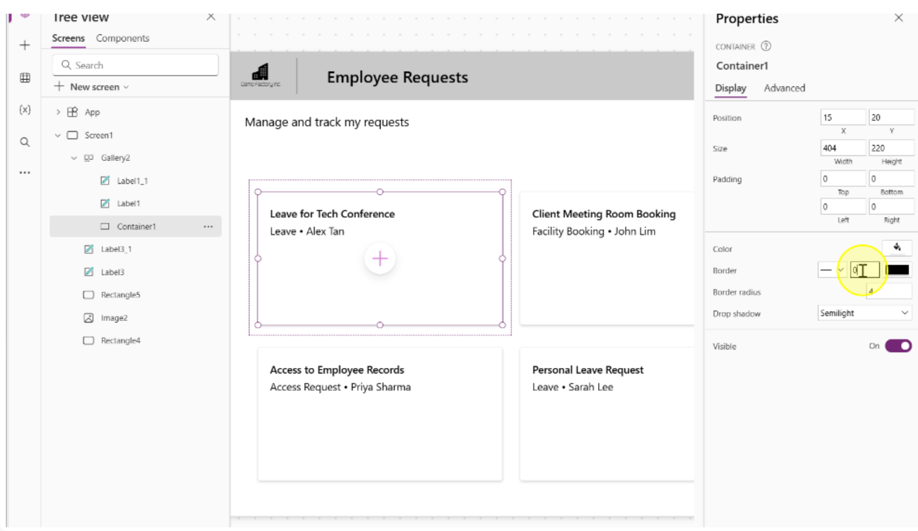
-- **End of Task** --

Task 3: Styling & UI/UX enhancements for Galleries (Optional)

1. Insert a container Control. We will use this control to create the curved rectangle border like effect.

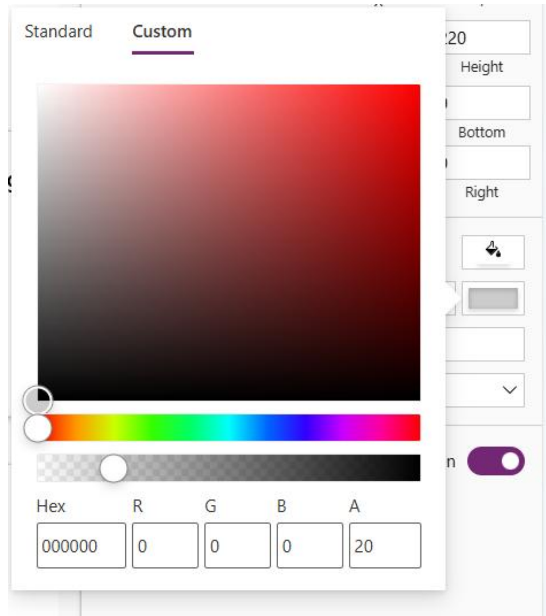


2. Center the container in the gallery template and position the labels



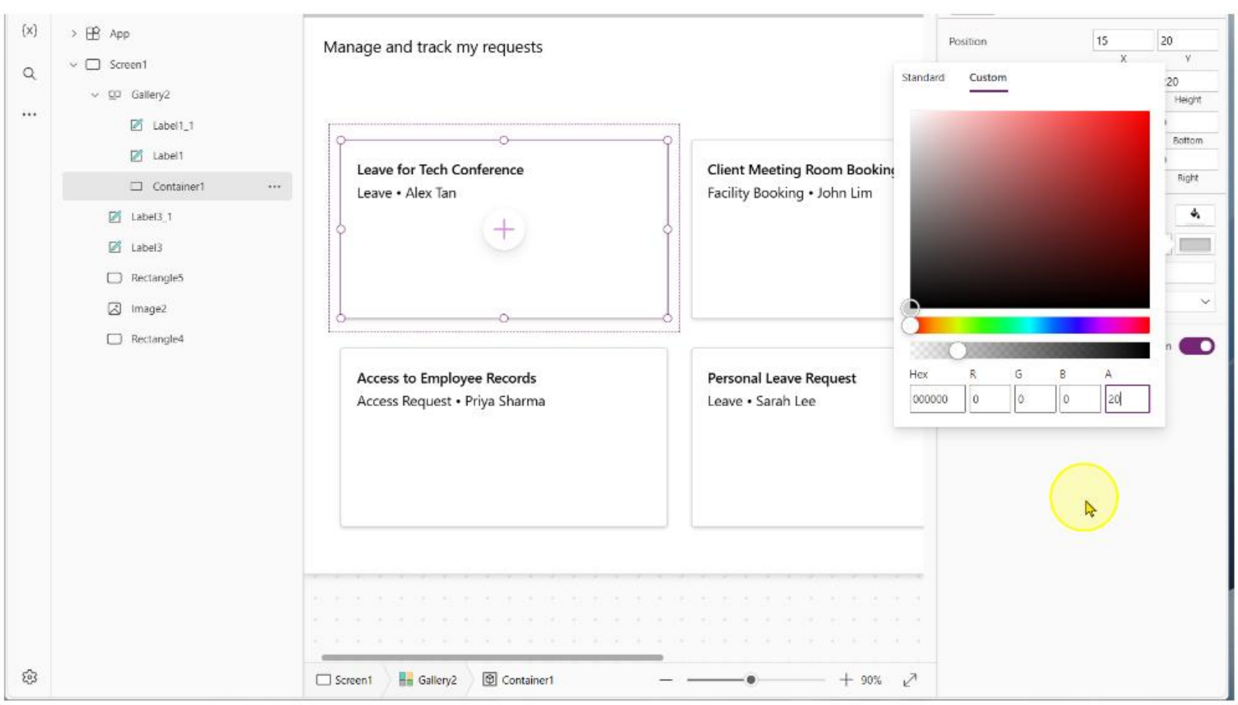
3. Change the following properties of the container

- BorderThickness: 1
- BorderColor

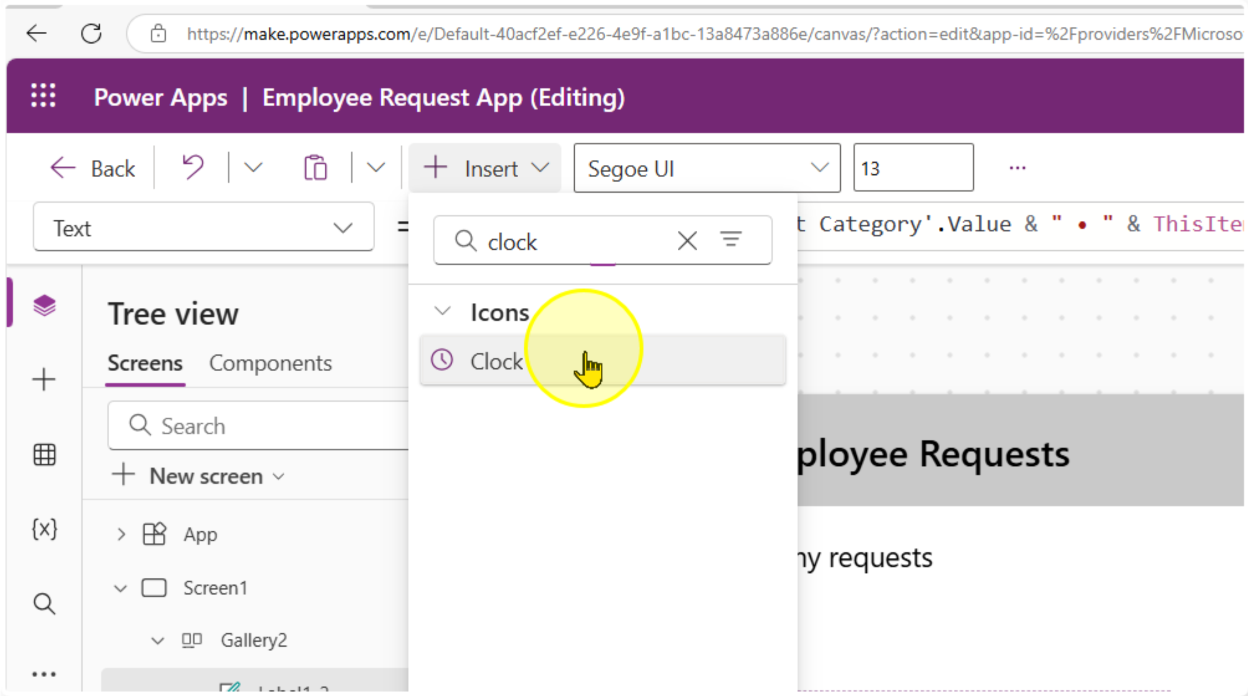


- Drop shadow: Semilight

4. This will give us the effect of the shadowed border, like a card display.



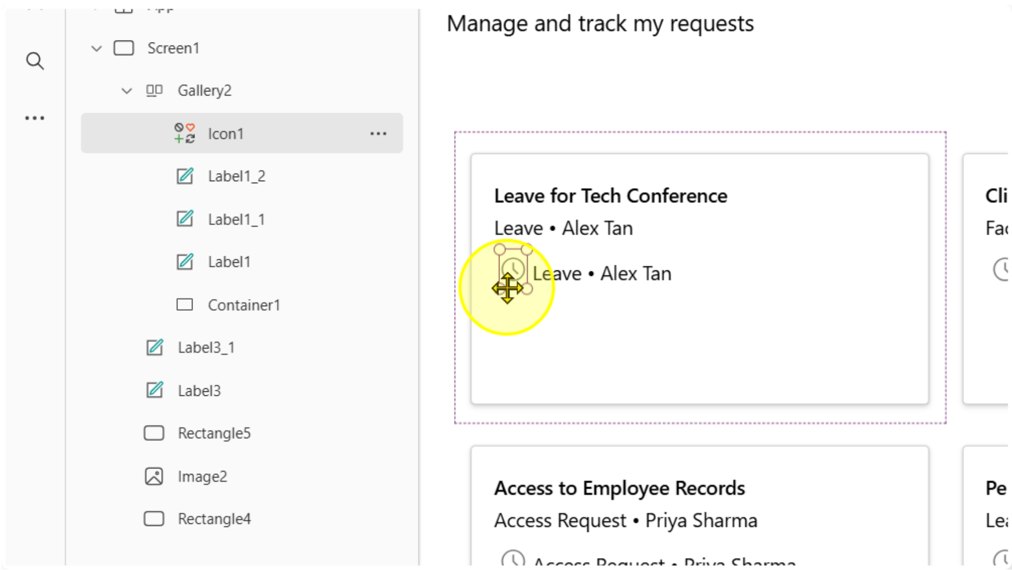
5. We will now add in a clock icon.



Adjust the following properties of the control to the corresponding values

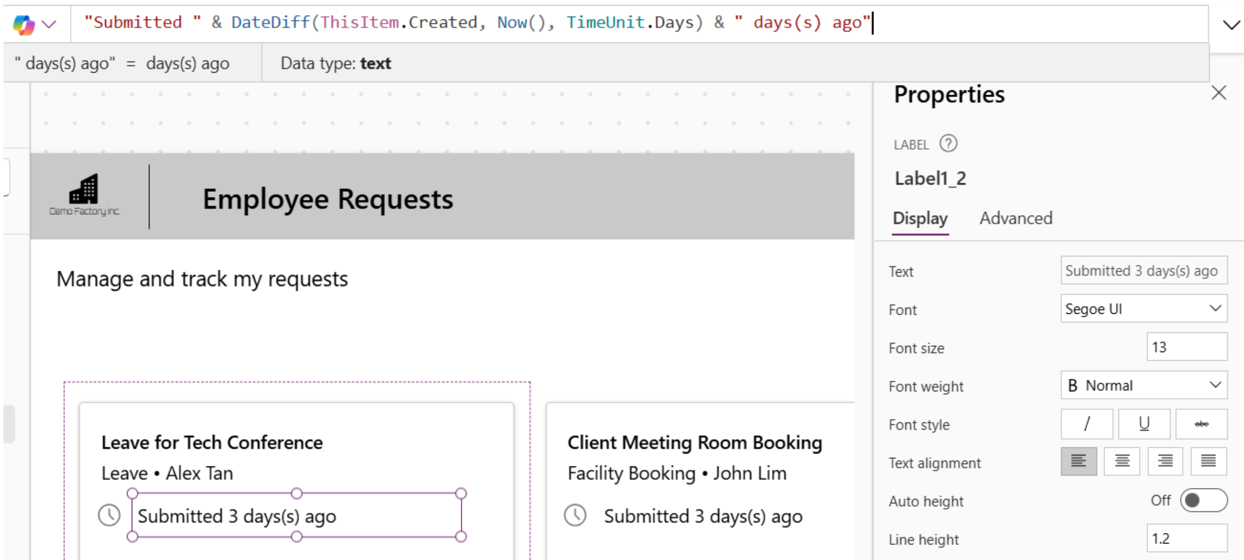
- Height: 50
- Width: 50
- Color: #808080

6. Position it below the 2 labels. We will use this label to show how many days ago a request was submitted. Additionally, make a copy of the second label.


Screen1
Gallery2
Icon1
Label1_2
Label1_1
Label1
Container1
Label3_1
Label3
Rectangle5
Image2
Rectangle4
Manage and track my requests
Leave for Tech Conference
Leave • Alex Tan
Leave • Alex Tan
Access to Employee Records
Access Request • Priya Sharma

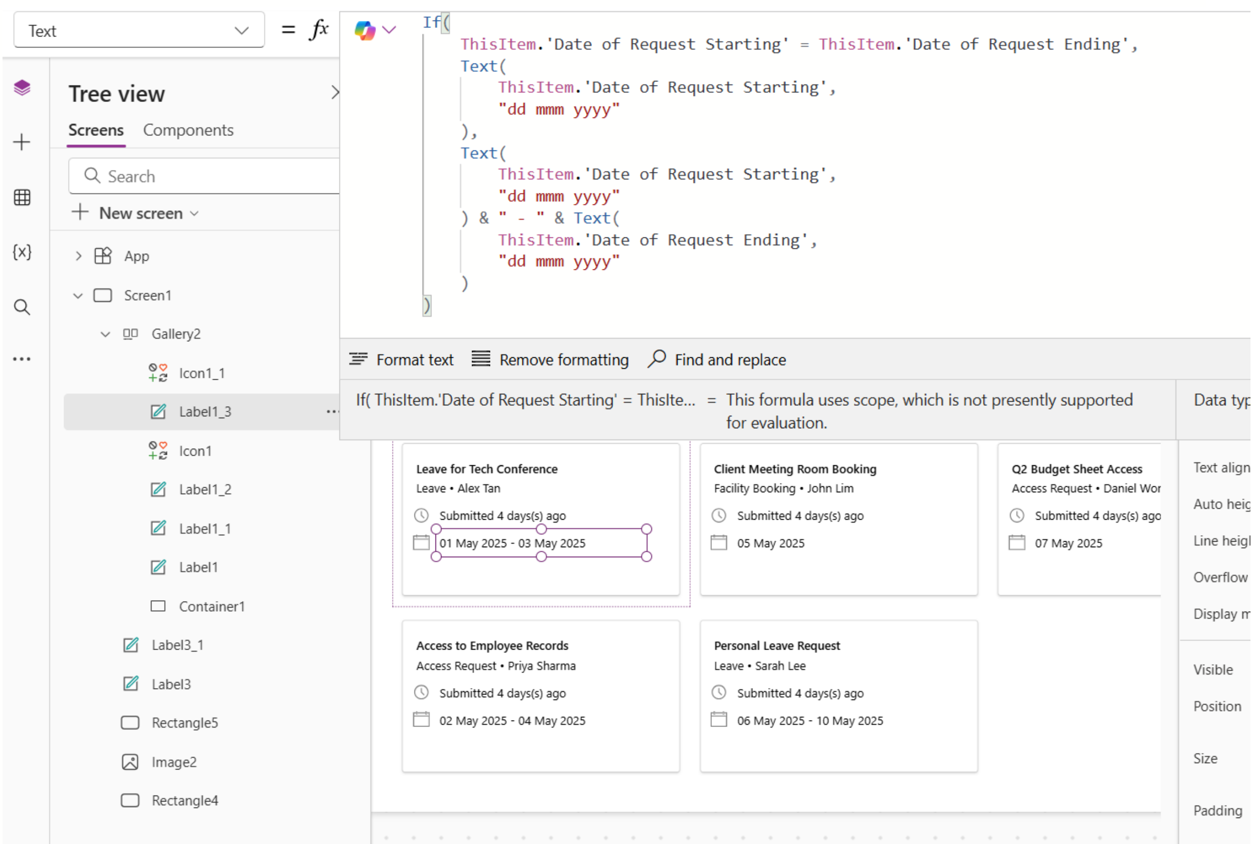
7. For the second label, change the text property to the following formula

```
"Submitted " & DateDiff(ThisItem.Created, Now(), TimeUnit.Days) & " days(s) ago"
```



8. Repeat the same steps. But with a calendar icon. For the label in this case, use the formula

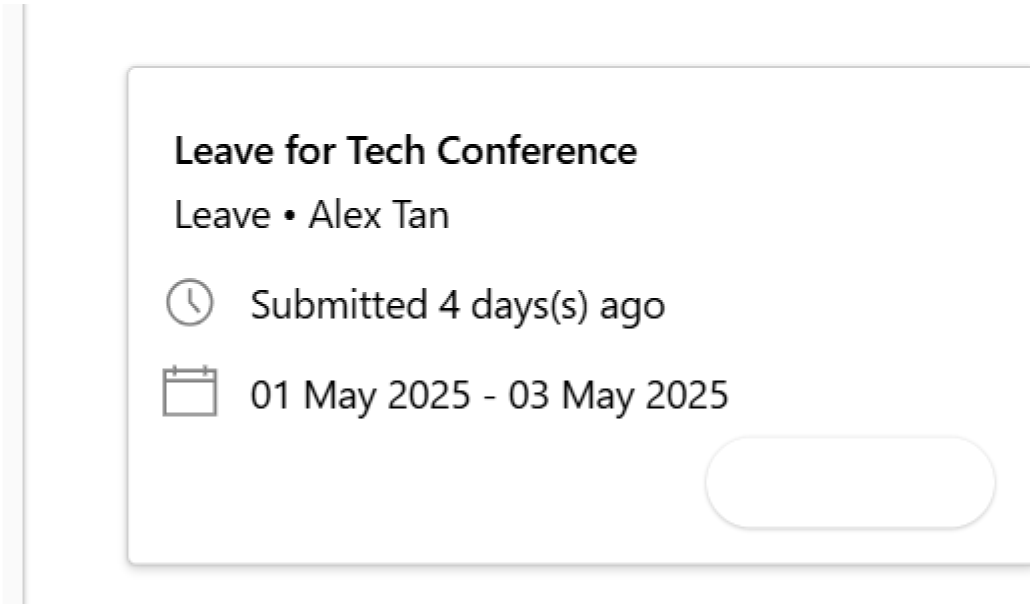
```
If(
    ThisItem.'Date of Request Starting' = ThisItem.'Date of Request
Ending',
    Text(
        ThisItem.'Date of Request Starting',
        "dd mmm yyyy"
    ),
    Text(
        ThisItem.'Date of Request Starting',
        "dd mmm yyyy"
    ) & " - " & Text(
        ThisItem.'Date of Request Ending',
        "dd mmm yyyy"
    )
)
```



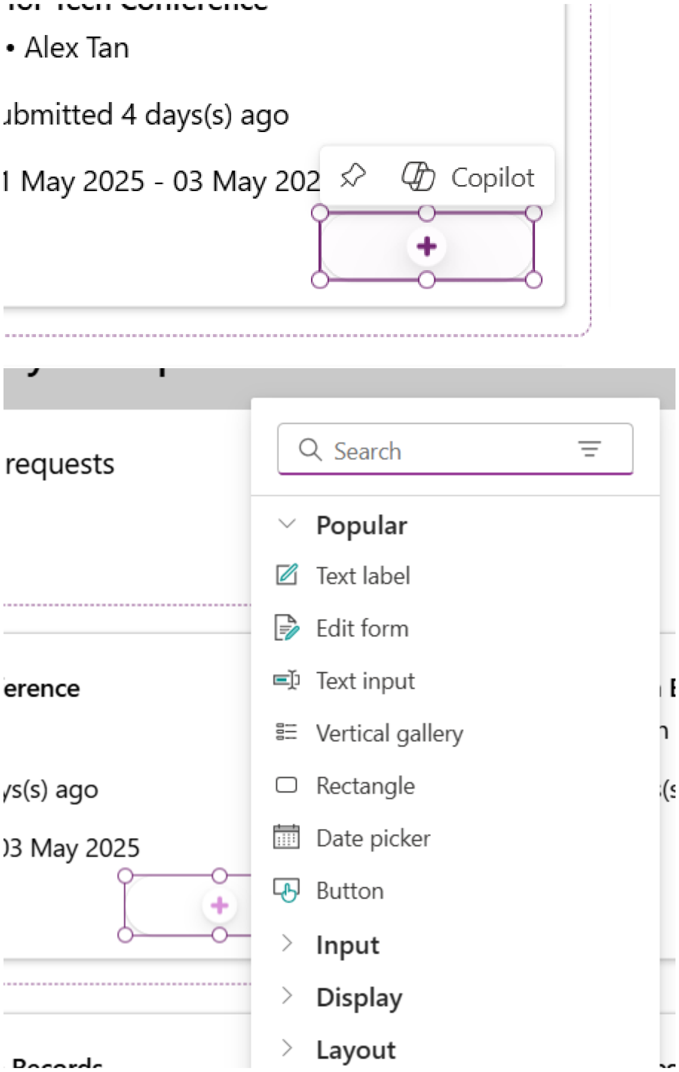
9. We will now add in another container.

Adjust the following properties of the control to the corresponding values

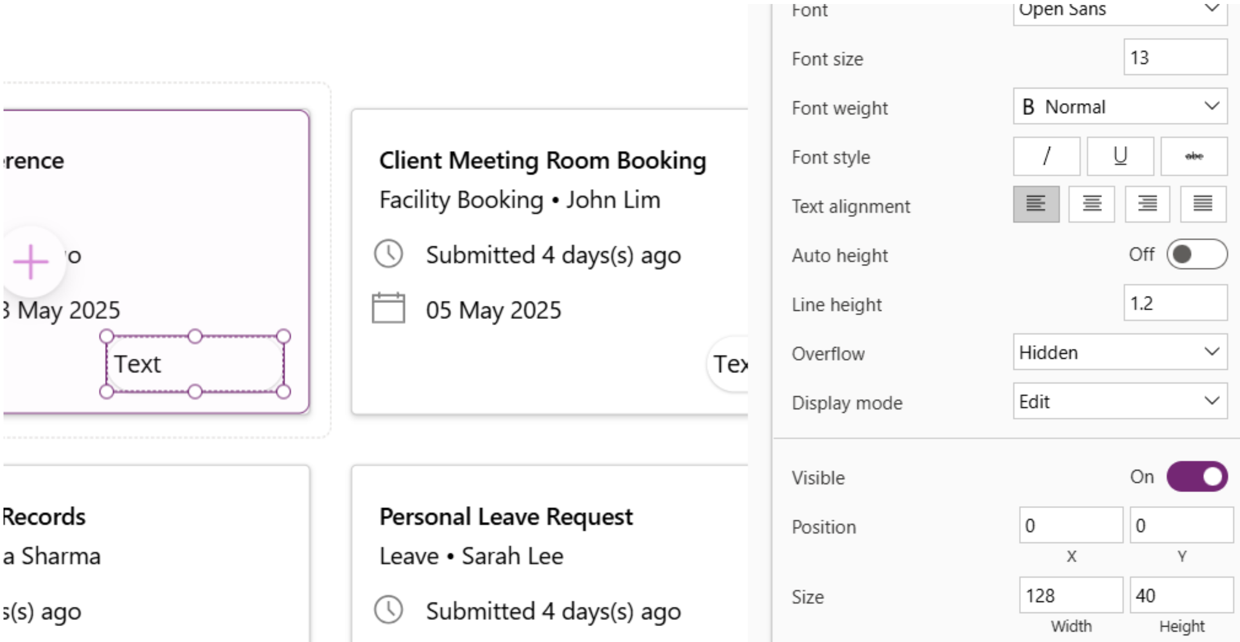
- Height: 50
- Width: 128
- Border radius: 25



10. In the container, add a text label. Click on the container, then the plus button in the container.

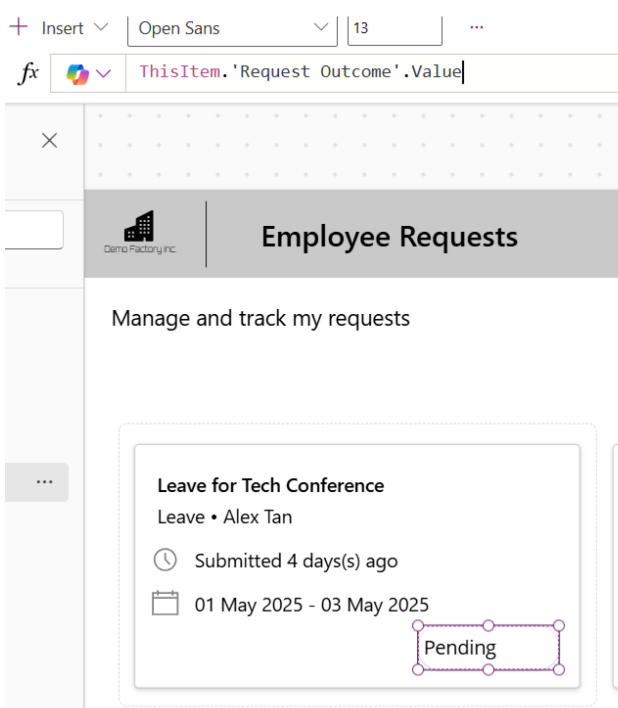


11. Add in a text label. Set the text label to have the same width and height of the container.



12. Change the Text property to the following expression

```
ThisItem.'Request Outcome'.Value
```



Change the text alignment to align center

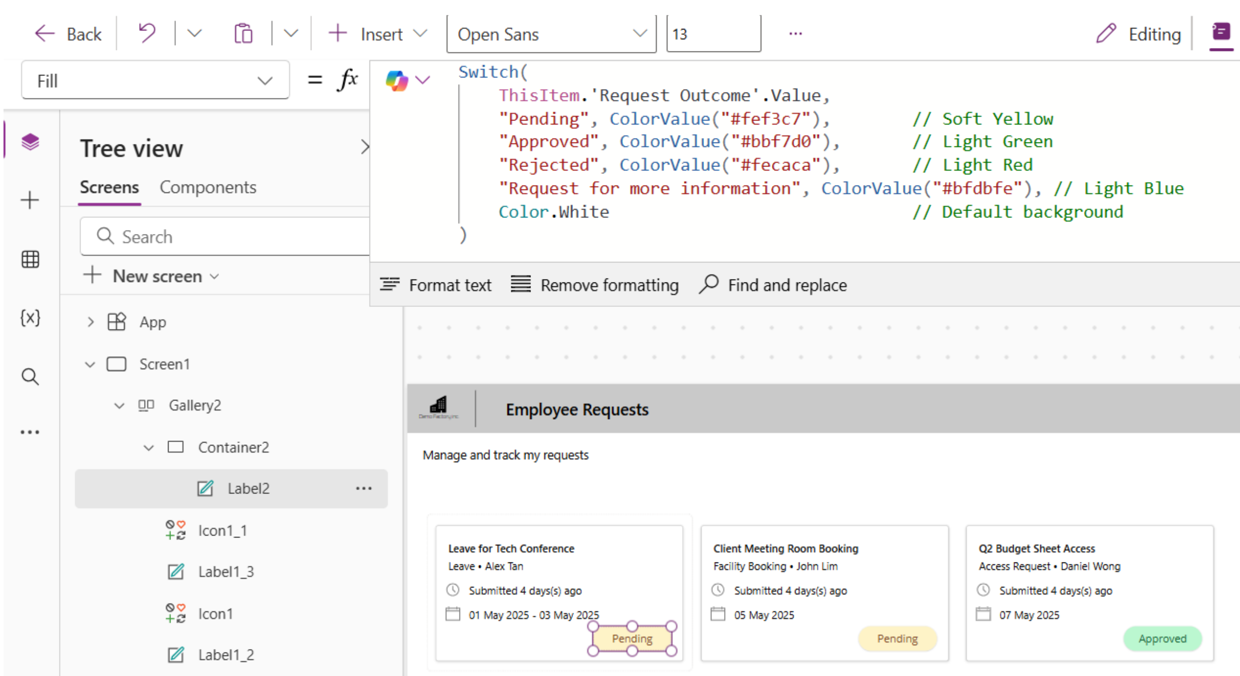
13. Change the label control's Color and fill properties to the following expressions to change the color depending on the value.

Color:

```
Switch(
    ThisItem.'Request Outcome'.Value,
    "Pending", ColorValue("#854d2b"),
    "Approved", ColorValue("#166534"),
    "Rejected", ColorValue("#991b1b"),
    "Request for more information", ColorValue("#1e3a8a"),
    Color.Black
)
```

Fill:

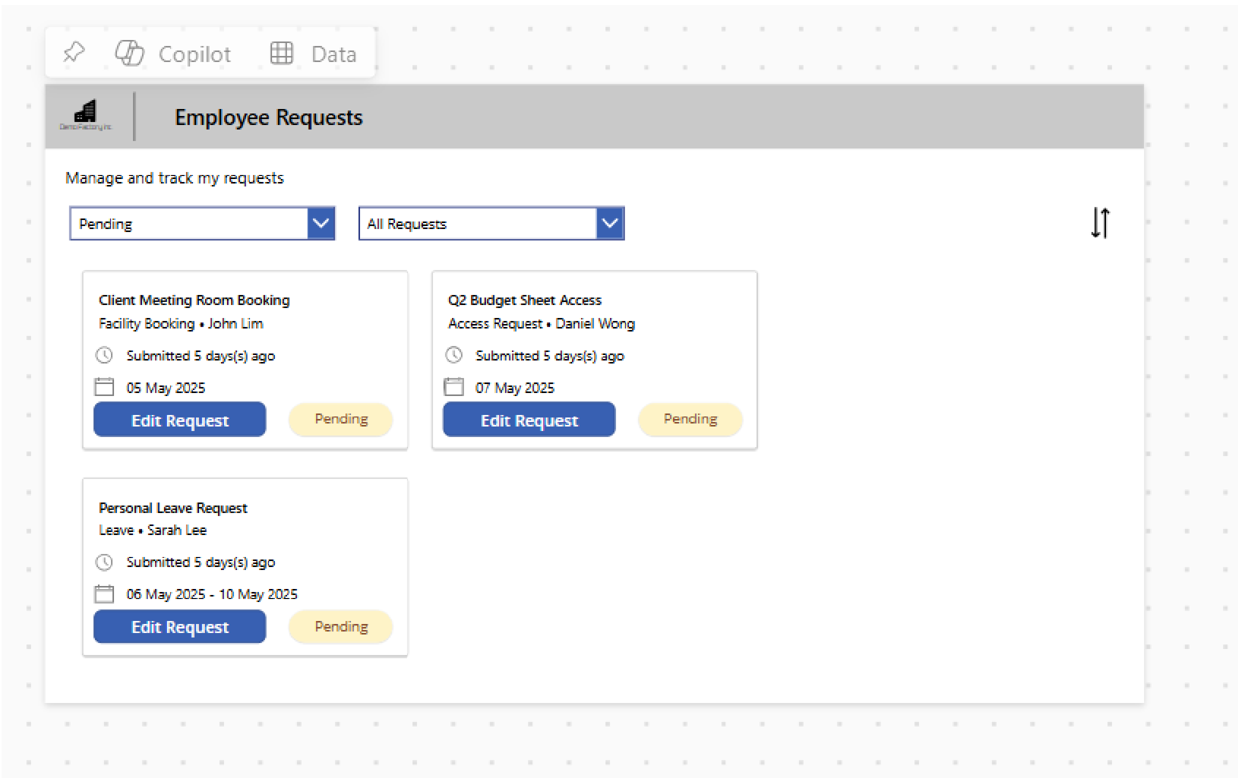
```
Switch(
    ThisItem.'Request Outcome'.Value,
    "Pending", ColorValue("#fef3c7"),        // Soft Yellow
    "Approved", ColorValue("#bbf7d0"),       // Light Green
    "Rejected", ColorValue("#fecaca"),       // Light Red
    "Request for more information", ColorValue("#bfdbfe"), // Light Blue
    Color.White                              // Default background
)
```



-- **End of Task** --

Exercise 4: Filtering & Sorting the Gallery

In this exercise, you will learn how to perform filtering and sorting on the gallery to quickly find information when needed. Sorting and filtering are basic data operations that represent simple business logic and help us narrow down the amount of information being displayed in the app.



Objectives

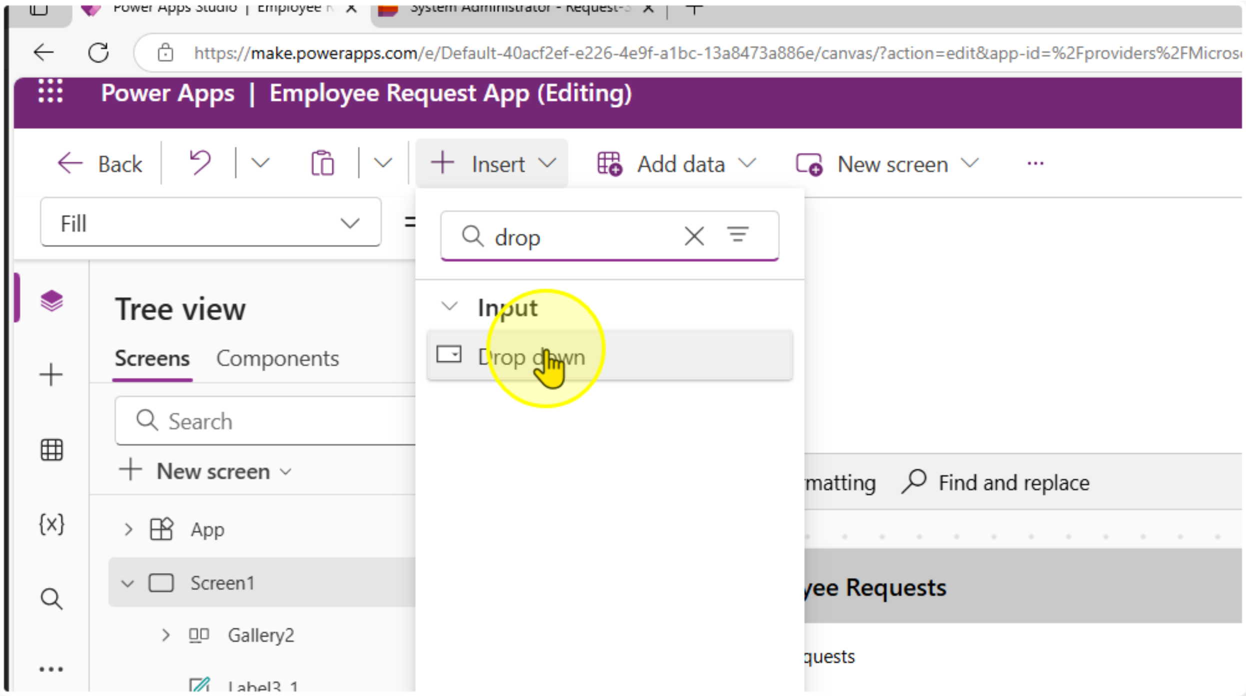
- Filter a gallery for a specific column value
- Filter a gallery for multiple column values from different controls
- Sort a gallery by the date field

Estimated time to complete this lab

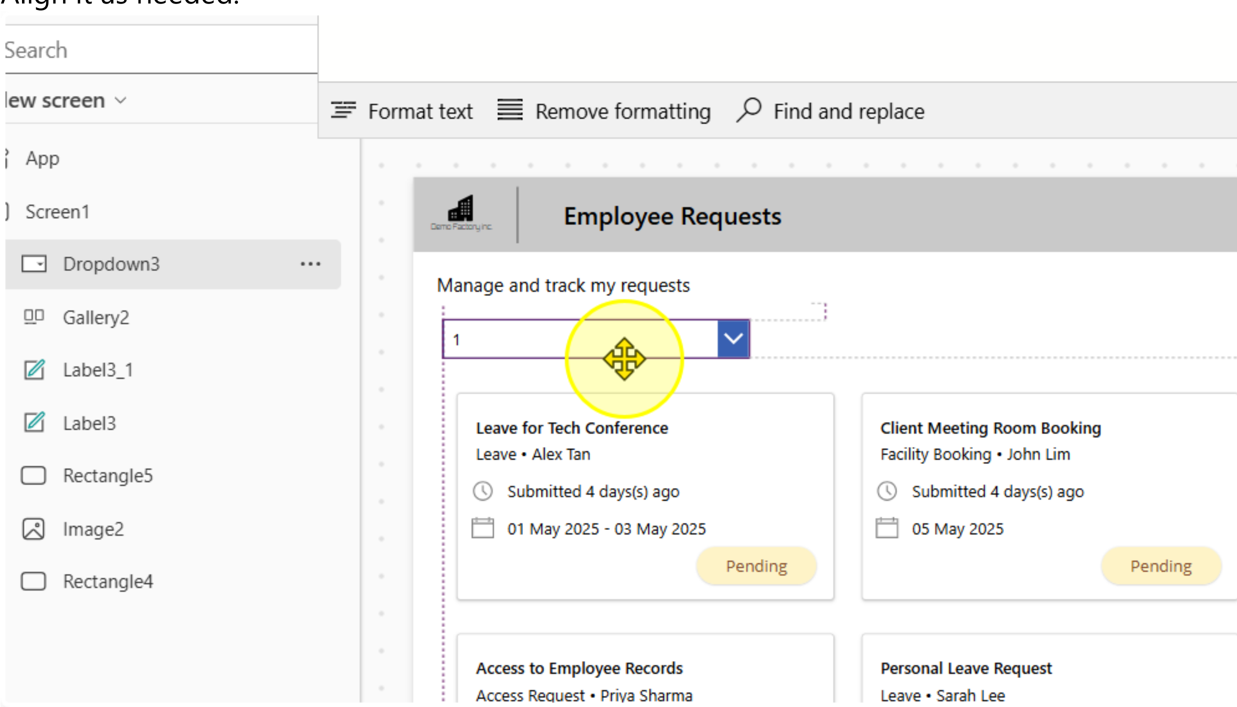
50 mins

Task 1: Filter based on Request Outcomes

1. Add in a dropdown control. Align it

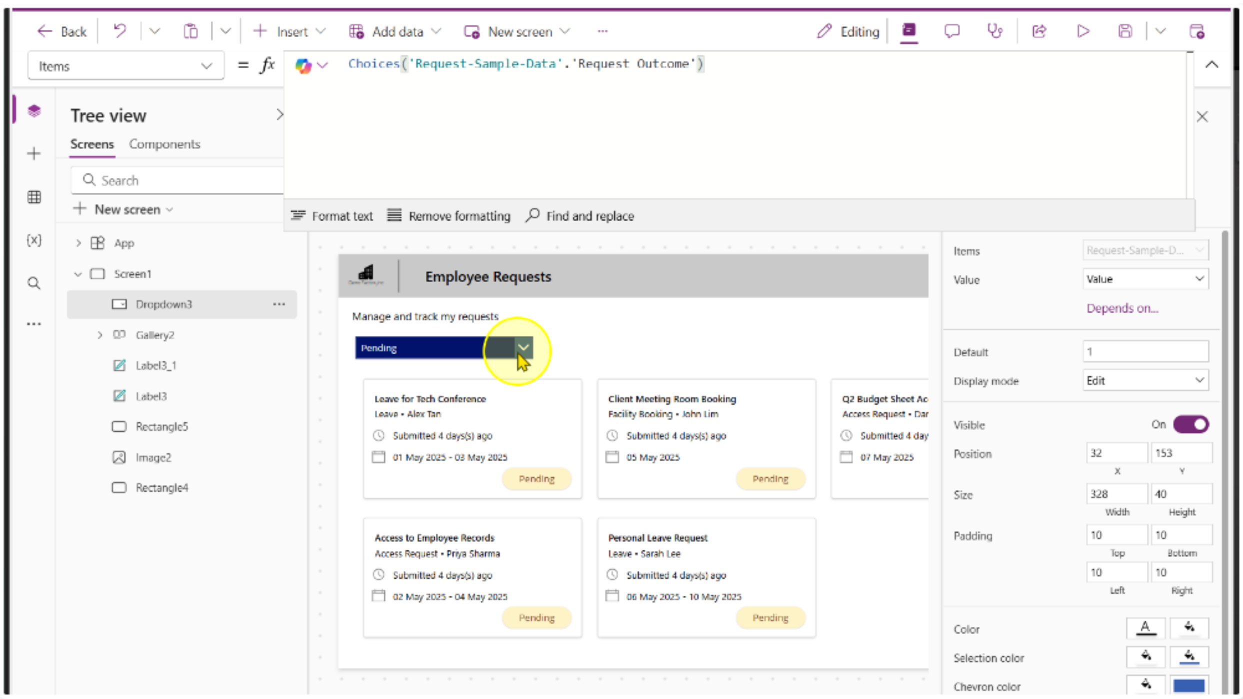


Align it as needed.



2. Enter the following expression into the dropdown's Items property

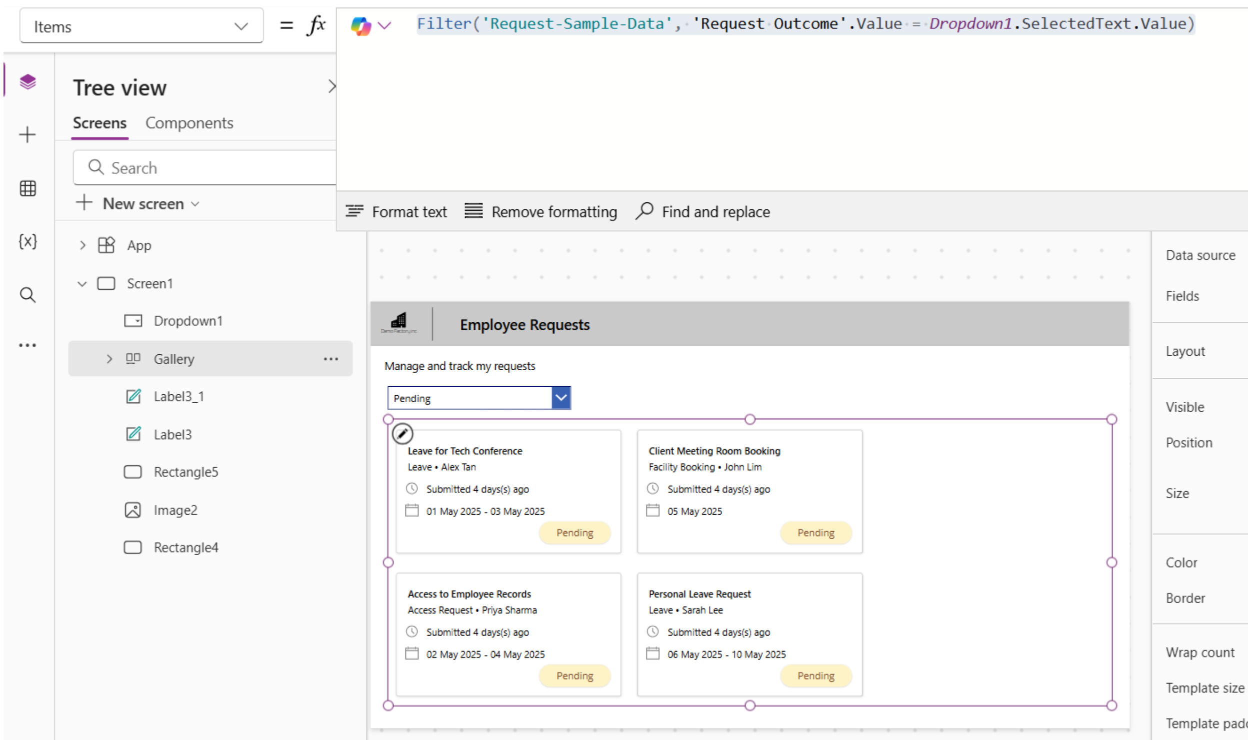
```
Choices('Request-Sample-Data'.'Request Outcome')
```



You should see the choices, "Pending", "Approved", "Rejected" & "Request for more information" get populated.

3. Select the Gallery Control. On the gallery control, change the items property to the following formula.

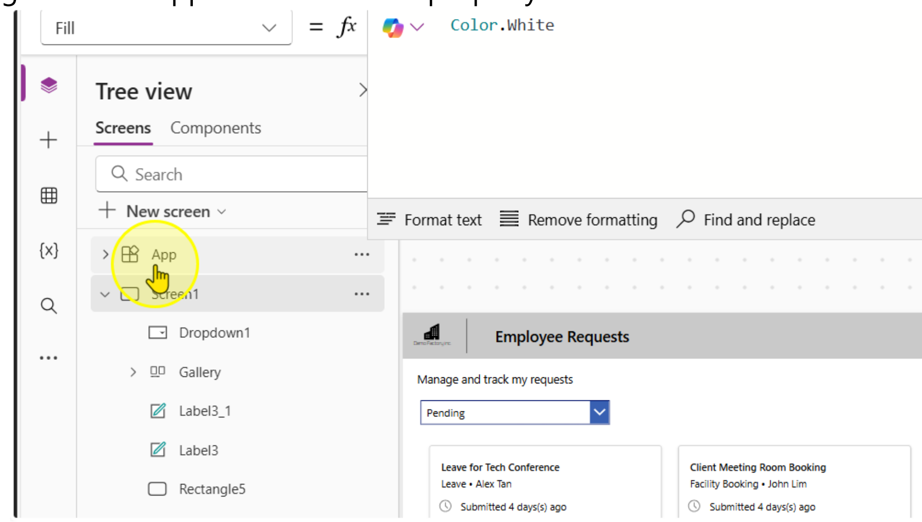
```
Filter('Request-Sample-Data', 'Request Outcome'.Value =
Dropdown1.SelectedText.Value)
```

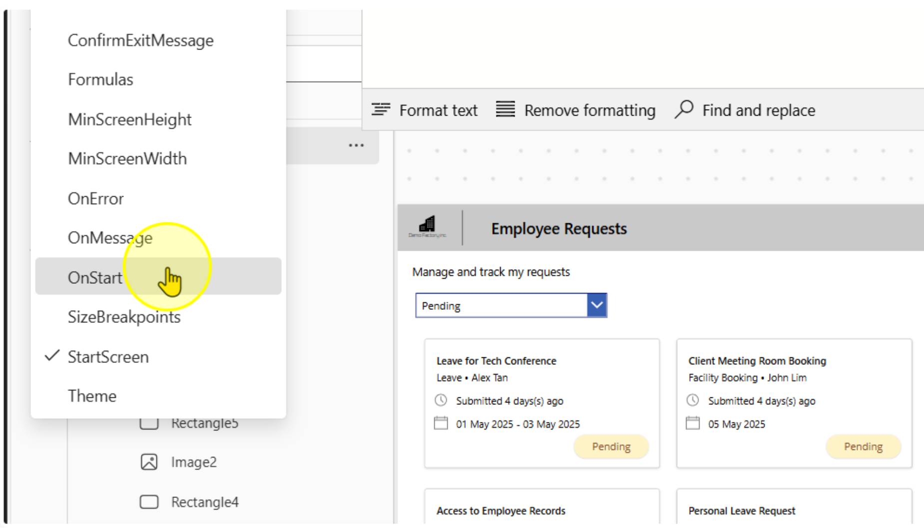


-- **End of Task** --

Task 2: Change the filter to have a "All" selection (Optional)

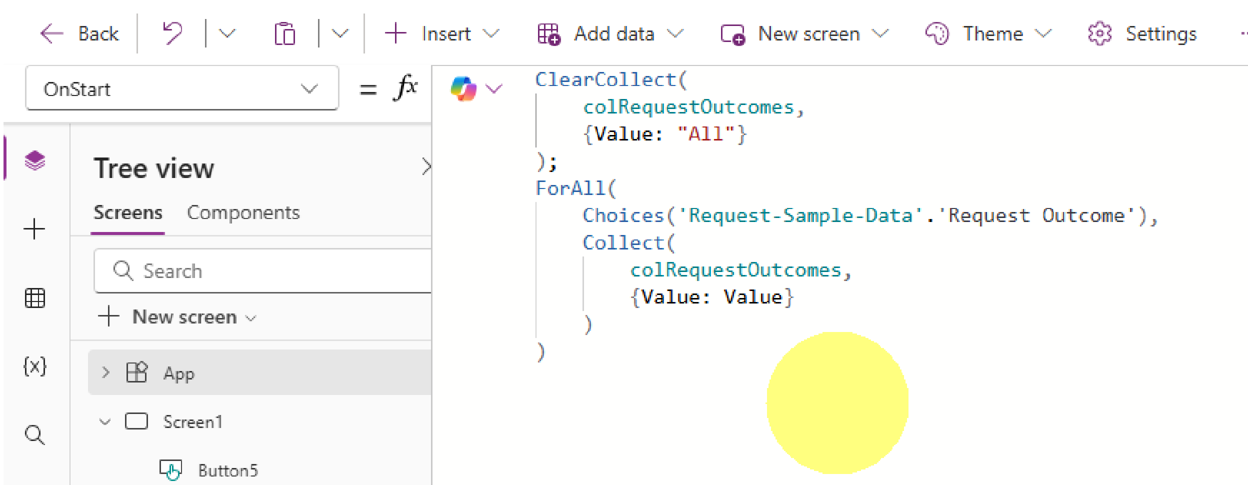
1. Navigate to the App Screen. Find the property called "OnStart"





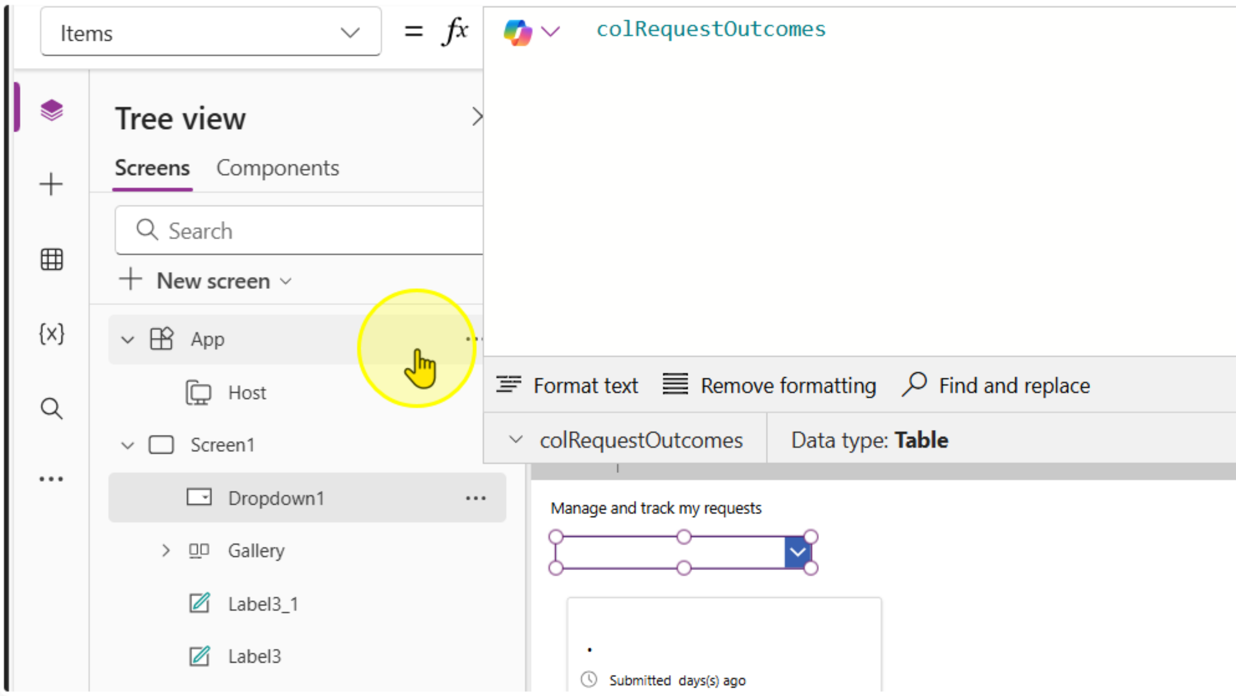
2. On this property, write the following expression

```
ClearCollect(
    colRequestOutcomes,
    {Value: "All"}
);
ForAll(
    Choices('Request-Sample-Data'.'Request Outcome'),
    Collect(
        colRequestOutcomes,
        {Value: Value}
    )
)
```



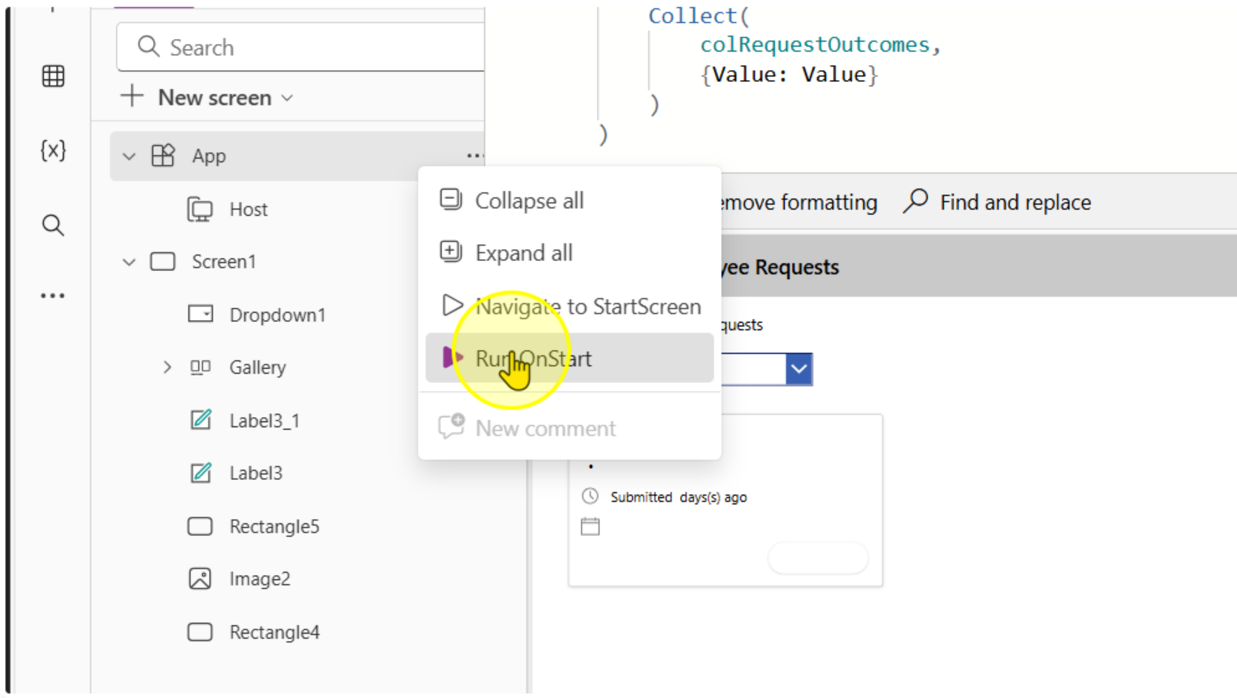
3. Change the Items property in the dropdown control to the following value

```
colRequestOutcomes
```

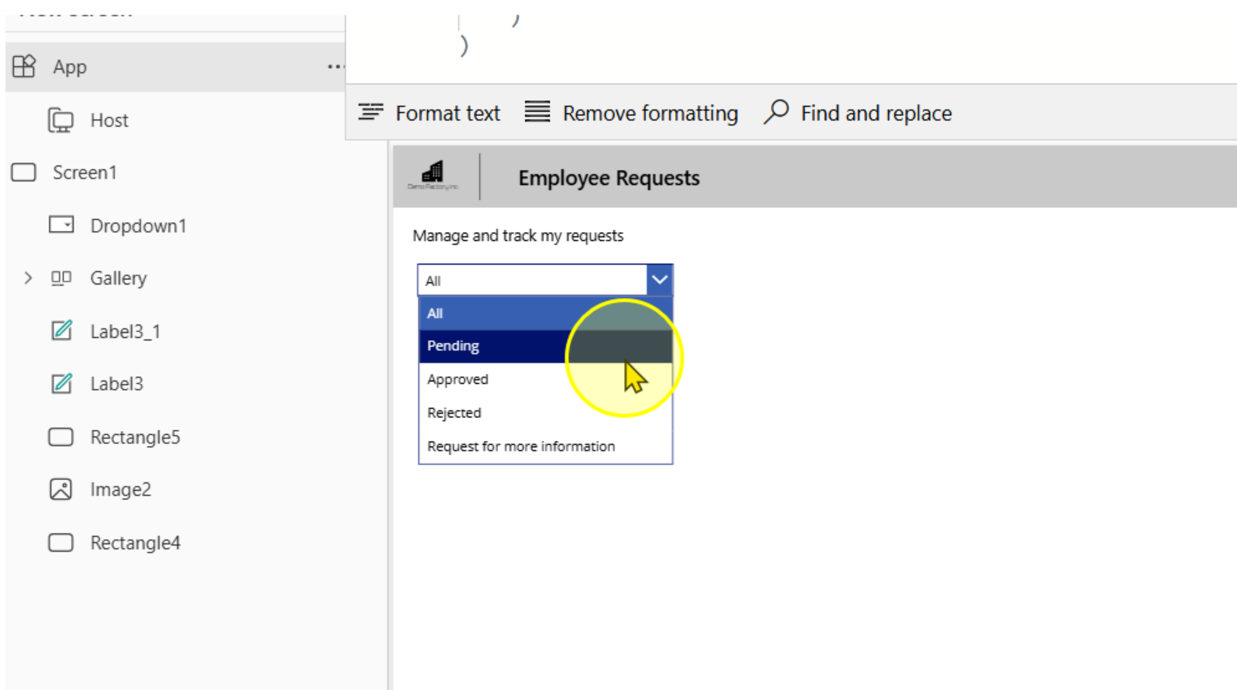


It should show a blank at first, because we have not ran the App "OnStart" property.

4. Right click on the App Screen. Left click to select the Run Onstart option



Once you perform this, you should see values appear in the dropdown selection



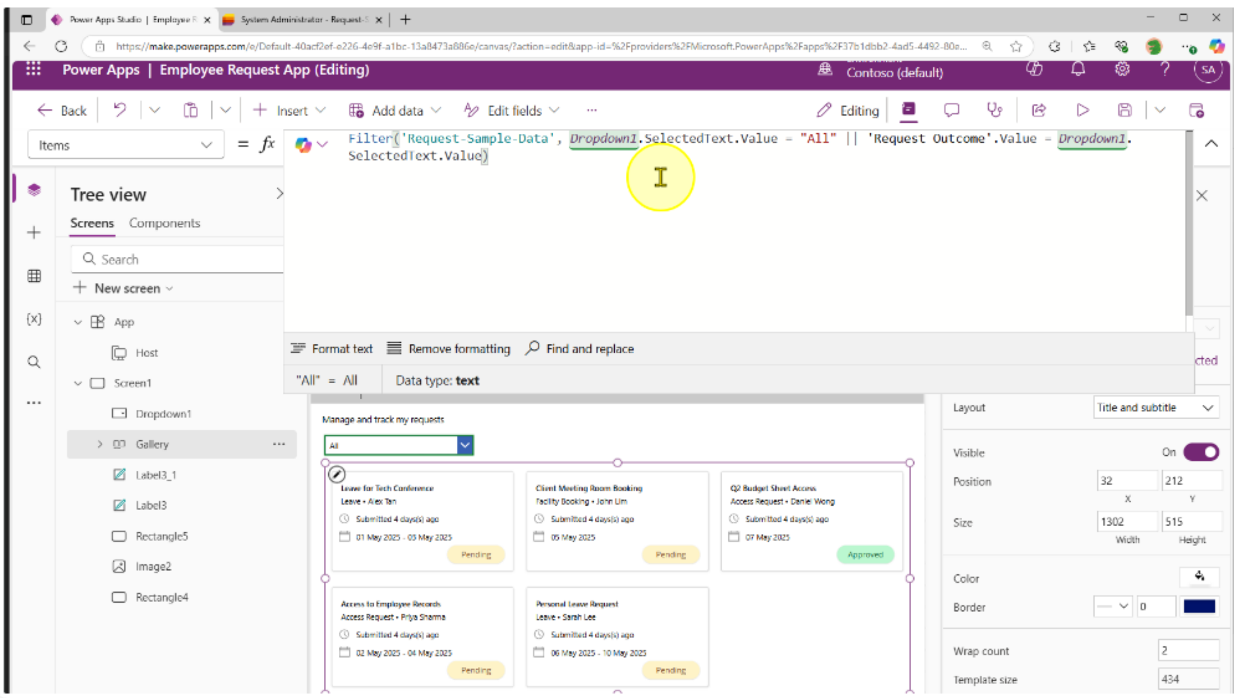
5. Go to the gallery control, change the expression to include this

Old:

```
Filter(
    'Request-Sample-Data',
    'Request Outcome'.Value = Dropdown1.SelectedText.Value
)
```

New:

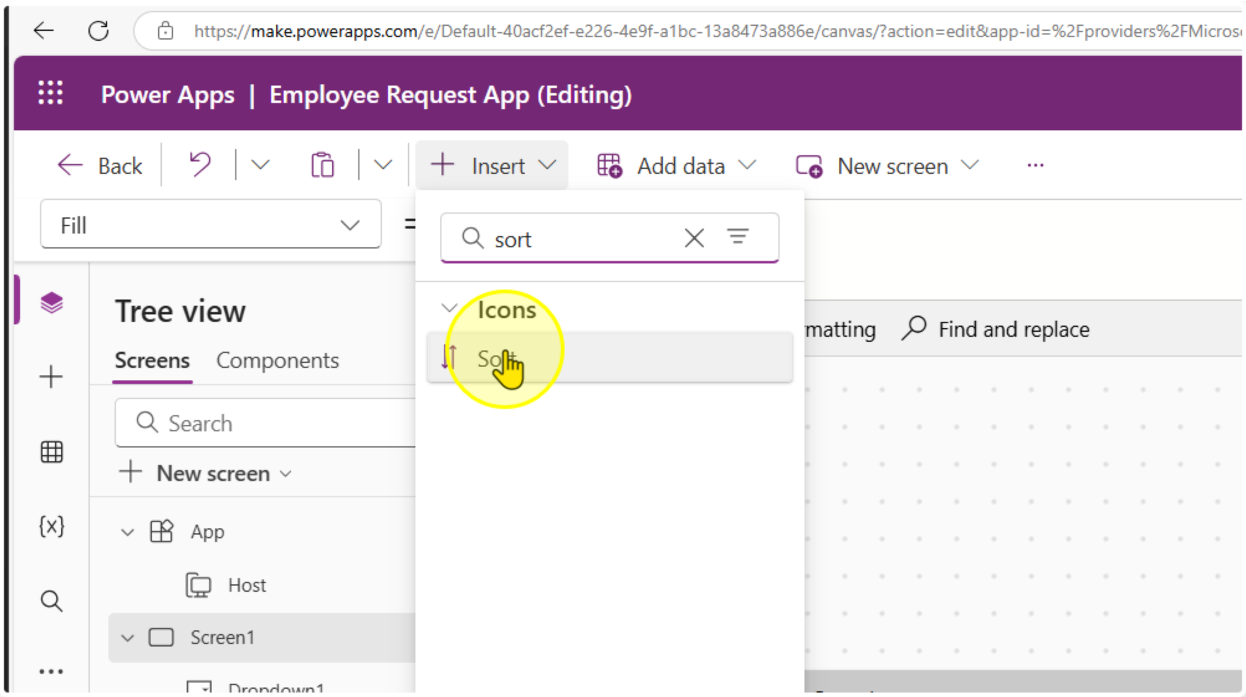
```
Filter(
    'Request-Sample-Data',
    Dropdown1.SelectedText.Value = "All" || 'Request Outcome'.Value =
Dropdown1.SelectedText.Value
)
```

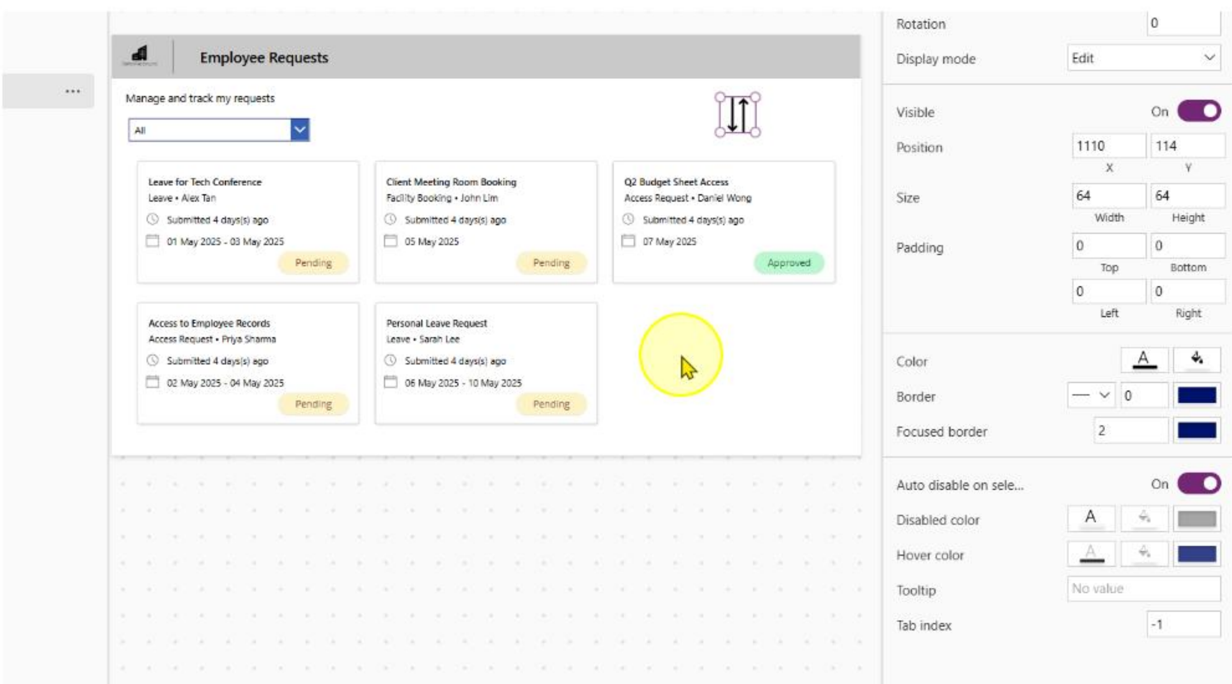


-- **End of Task** --

Task 3: Sort the Requests by Date

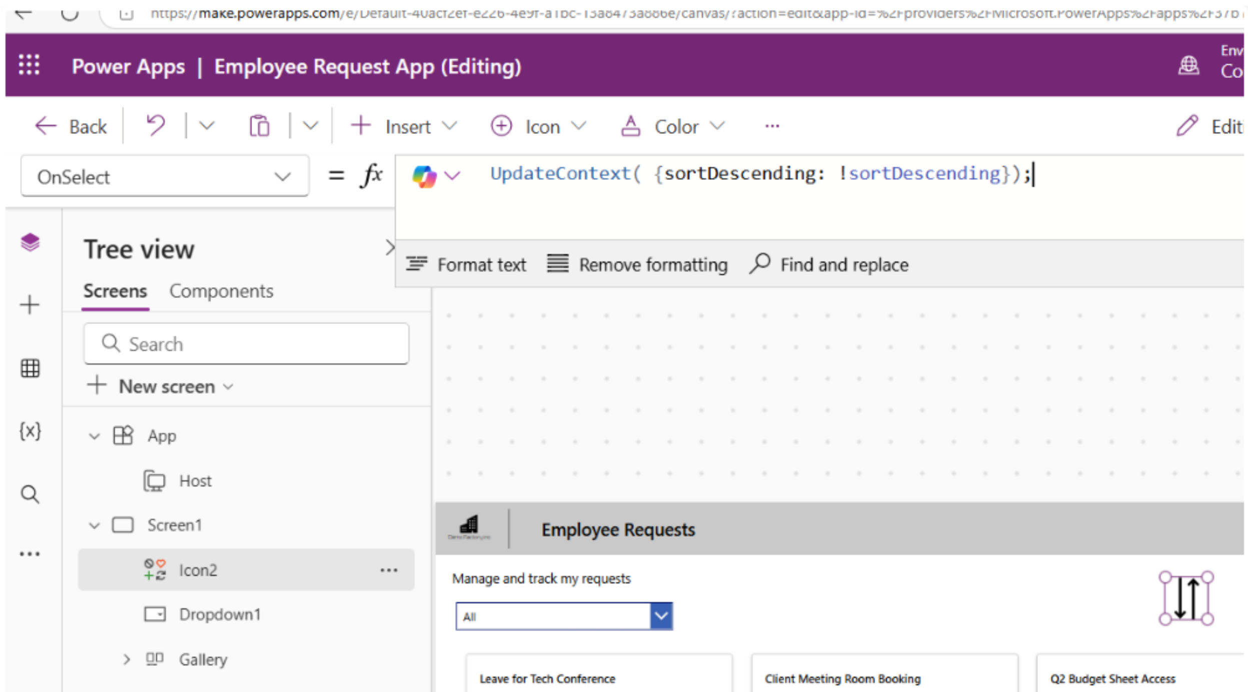
1. Insert a Sort icon position it, change its size and its color to your liking





2. Change the OnSelect Property of the Sort Icon to the following expression

```
UpdateContext( {sortDescending: !sortDescending});
```



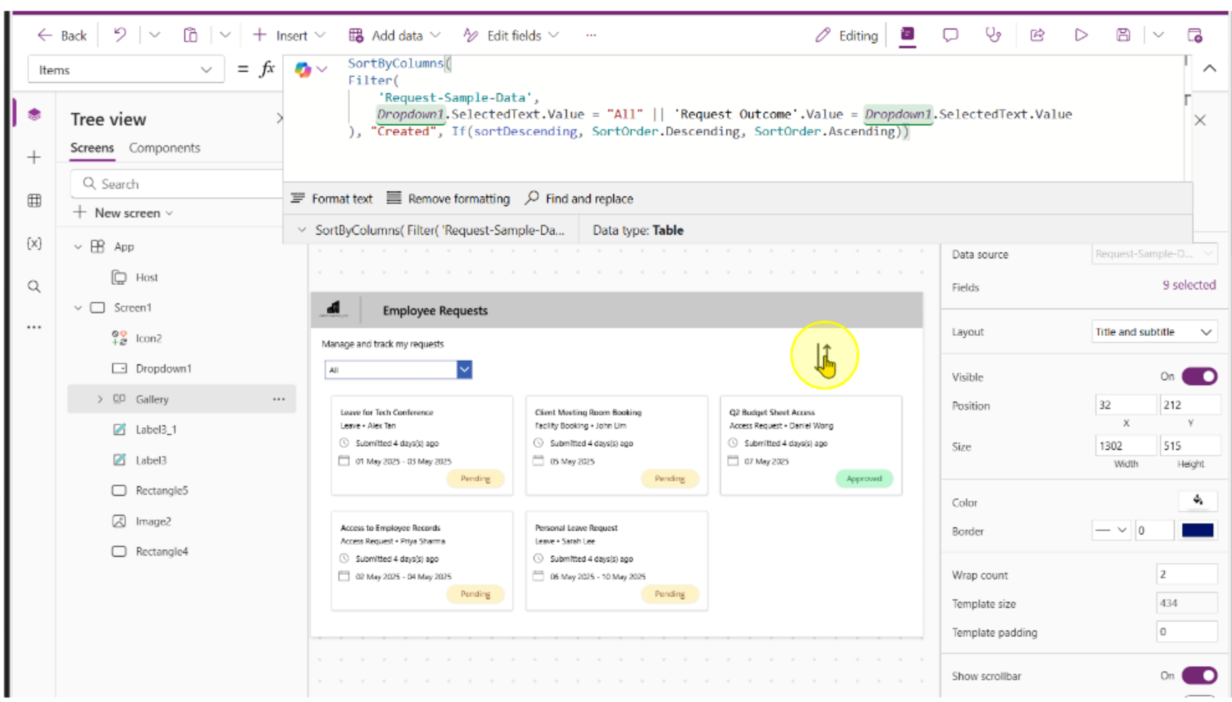
3. On the gallery control, change the expression to include the following

Old:

```
Filter(
        'Request-Sample-Data',
        Dropdown1.SelectedText.Value = "All" || 'Request Outcome'.Value = 
Dropdown1.SelectedText.Value
```

New:

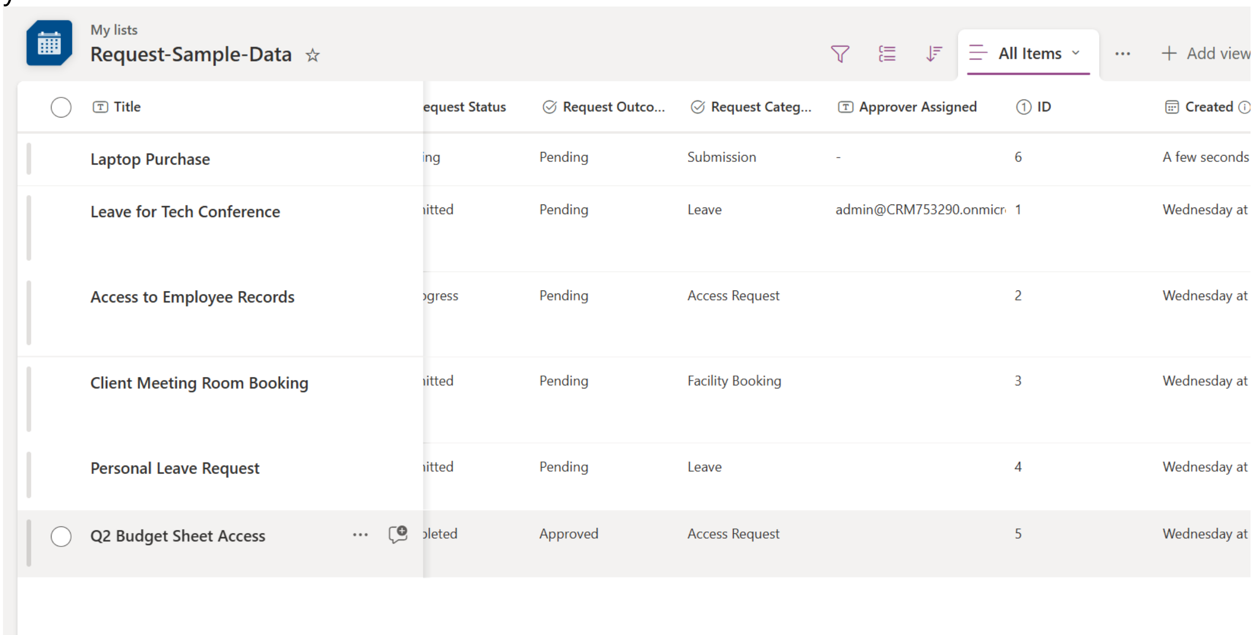
```
SortByColumns(
    Filter(
        'Request-Sample-Data',
        Dropdown1.SelectedText.Value = "All" || 'Request Outcome'.Value = 
Dropdown1.SelectedText.Value
    ),
    "Created",
    If(
        sortDescending,
        SortOrder.Descending,
        SortOrder.Ascending
    )
)
```



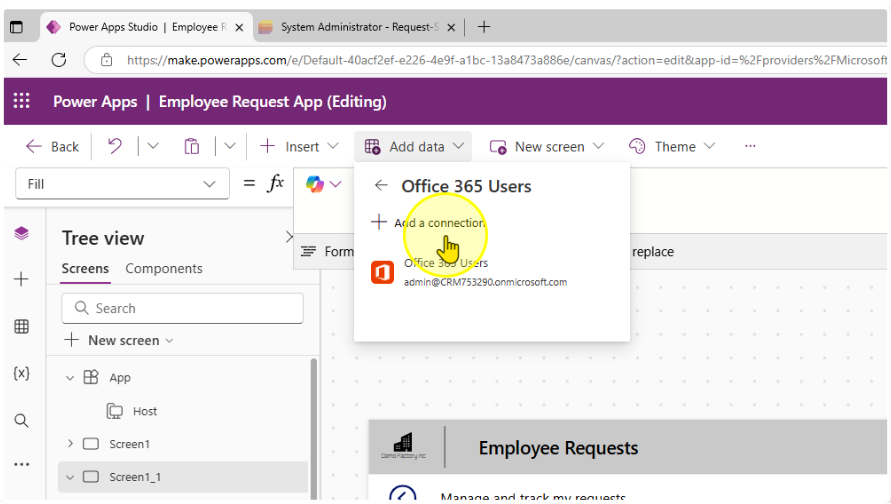
-- **End of Task** --

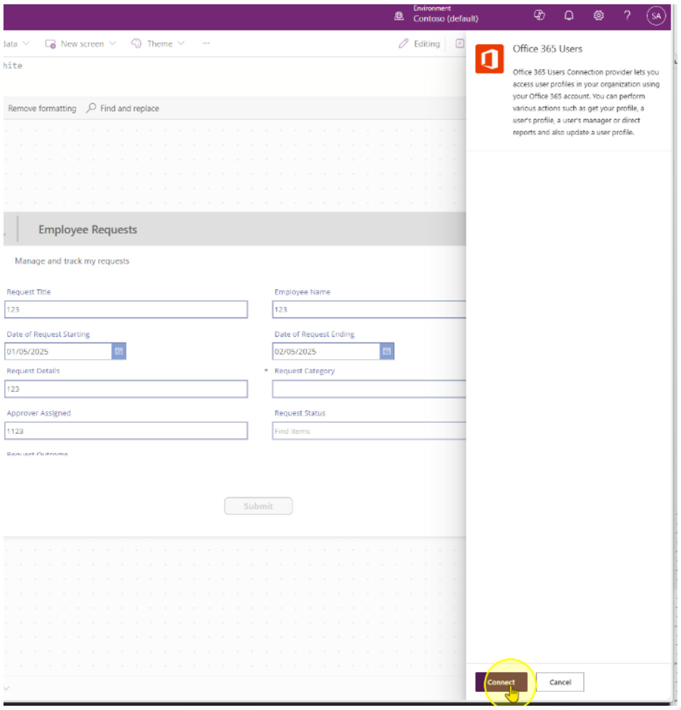
Task 4: View Requests assigned to you

1. Ensure you have at least 1 row on the 'Approver Assigned' Column that is populated with your email



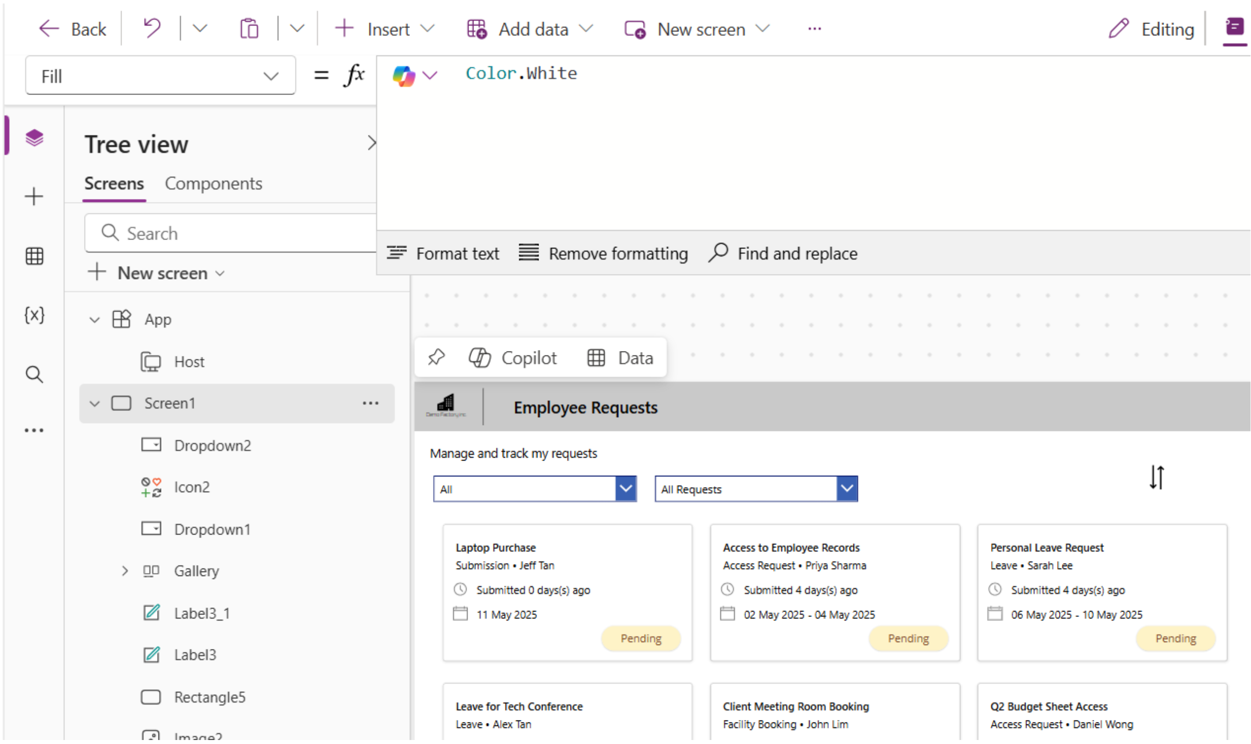
2. Create a connection to office365.




Environment
Contoso (default)
New screen
Theme
Editing
Remove formatting
Find and replace
Employee Requests
Manage and track my requests
Request Title
Employee Name
Date of Request Starting
Date of Request Ending
Request Details
Request Category
Approver Assigned
Request Status
Submit
Office 365 Users
Office 365 Users Connection provider lets you access user profiles in your organization using your Office 365 account. You can perform various actions such as get your profile, a user's profile, a user's manager or direct reports and also update a user profile.
Connect
Cancel

3. Add in a Second Dropdown control. Change the Items property to the following values.

```
["All Requests","My Assigned Requests"]
```



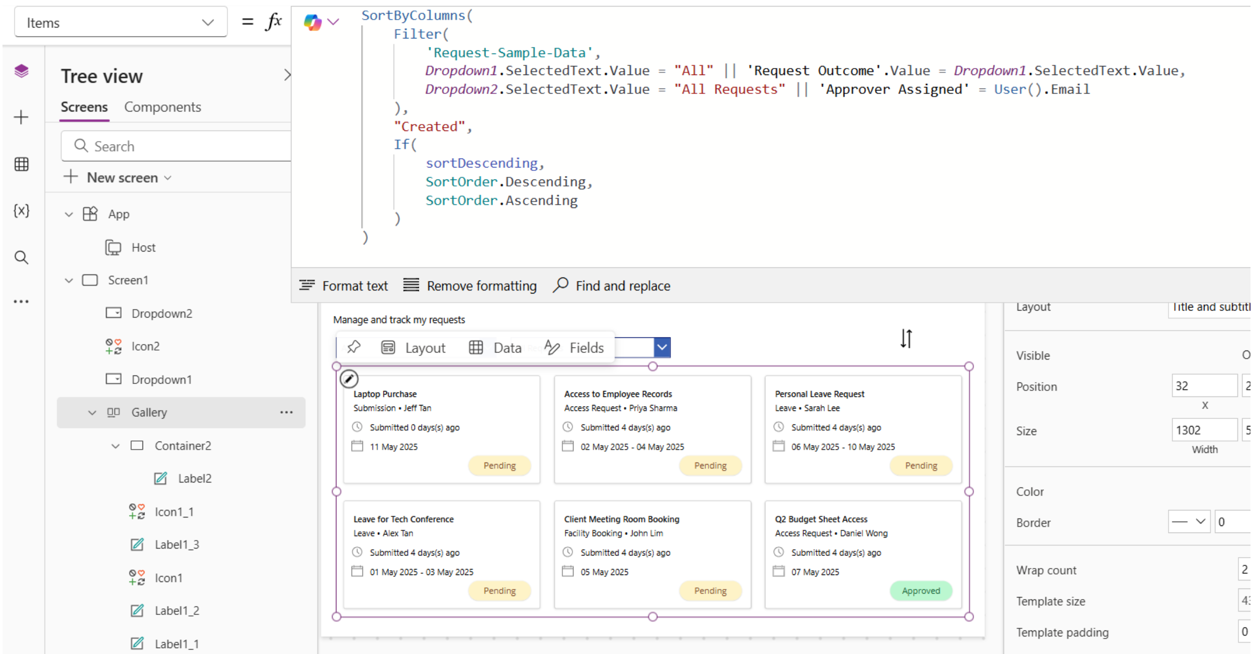
4. Change the Gallery's Items property.

Old:

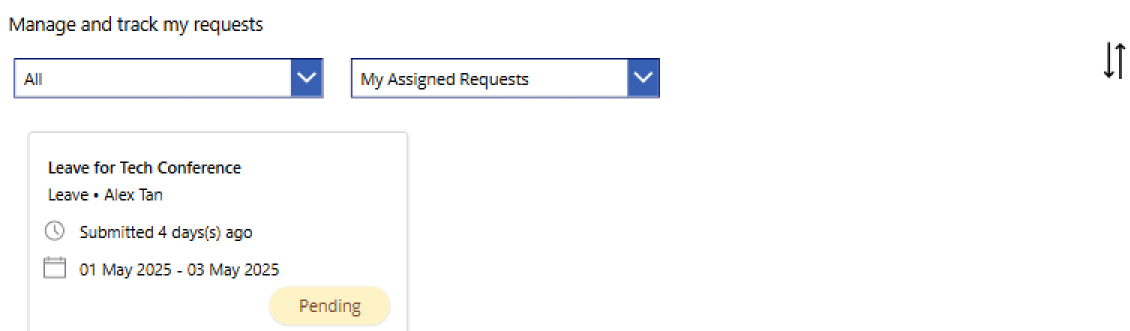
```
SortByColumns(
    Filter(
        'Request-Sample-Data',
        Dropdown1.SelectedText.Value = "All" || 'Request Outcome'.Value =
Dropdown1.SelectedText.Value
    ),
    "Created",
    If(
        sortDescending,
        SortOrder.Descending,
        SortOrder.Ascending
    )
)
```

New:

```
SortByColumns(
    Filter(
        'Request-Sample-Data',
        Dropdown1.SelectedText.Value = "All" || 'Request Outcome'.Value =
Dropdown1.SelectedText.Value,
        Dropdown2.SelectedText.Value = "All Requests" || 'Approver
Assigned' = Office365Users.MyProfileV2().mail || 'Approver Assigned' =
Office365Users.MyProfileV2().userPrincipalName

    ),
    "Created",
    If(
        sortDescending,
        SortOrder.Descending,
        SortOrder.Ascending
    )
)
```



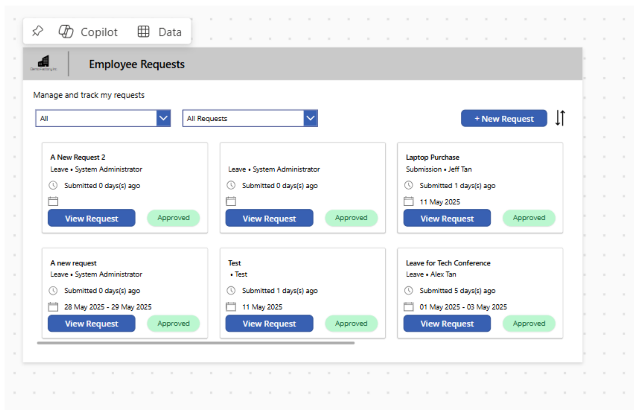
You should be able to test the filter.

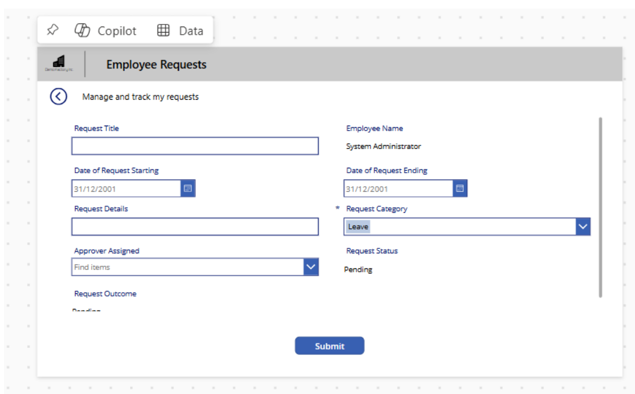


-- **End of Task** --

Exercise 5: Allow users to create new Requests

Users need to be able to create new requests. For they will need to fill in a form and enter data into the List they have created. This exercise will get users to create a new screen, allow users to navigate there and allow them to edit it.





Objectives

- Create a button to navigate to the next page
- Add in form that allows users to create a new request
- Allow submissions to the List

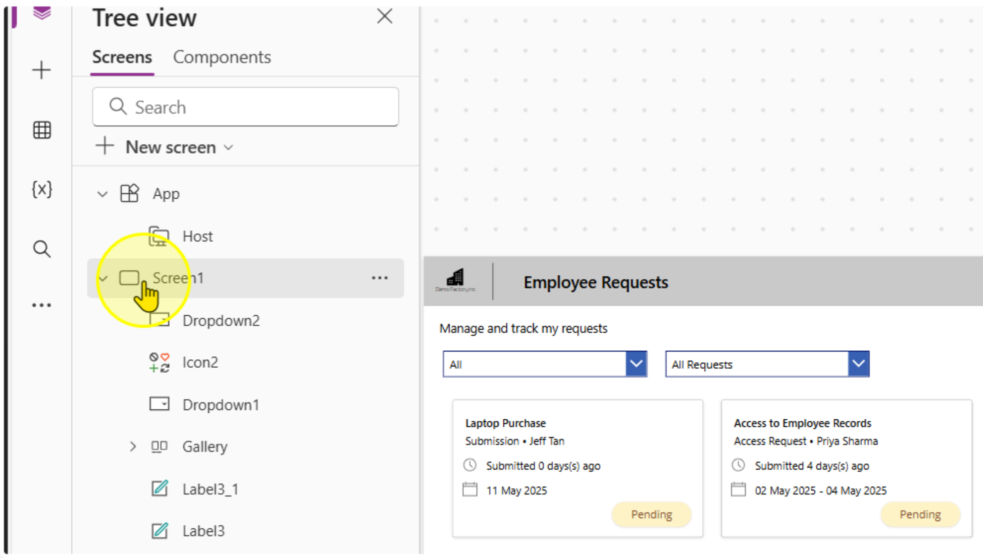
Estimated time to complete this lab

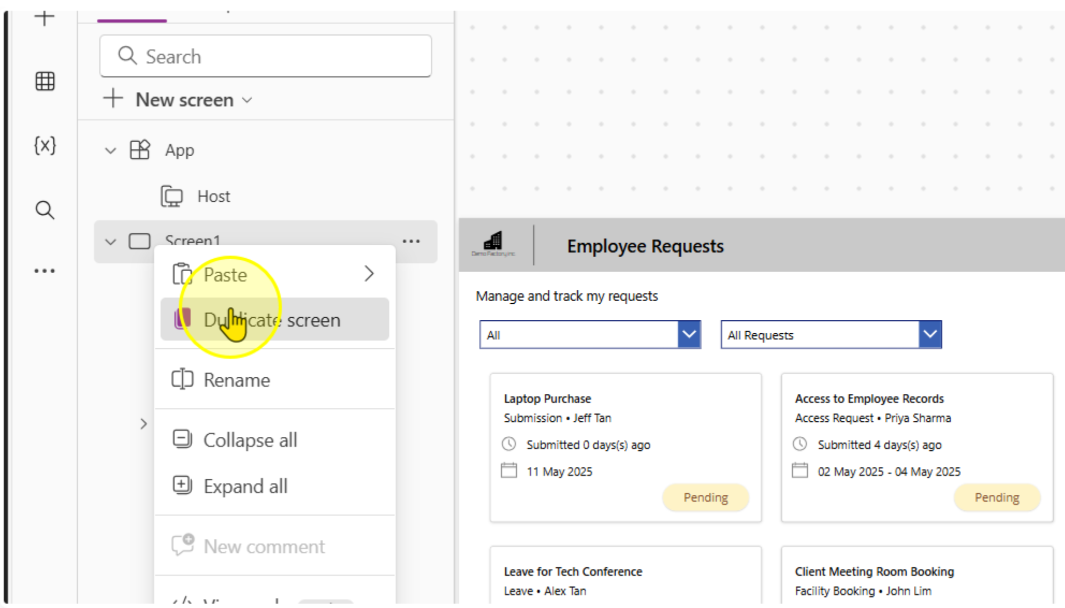
40 mins

Microsoft
Power Platform

Task 1: Duplicate the existing Screen

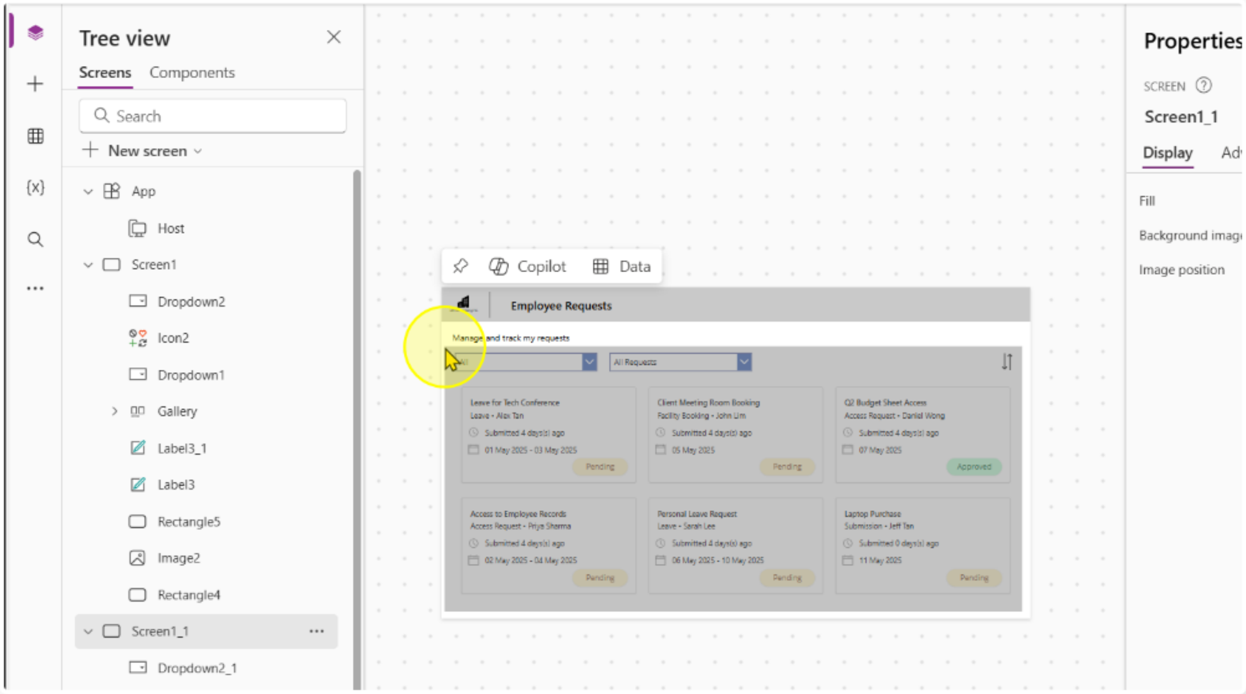
1. Duplicate the existing screen by following selecting the screen, right clicking it, and then selecting the 'Duplicate Screen' option.





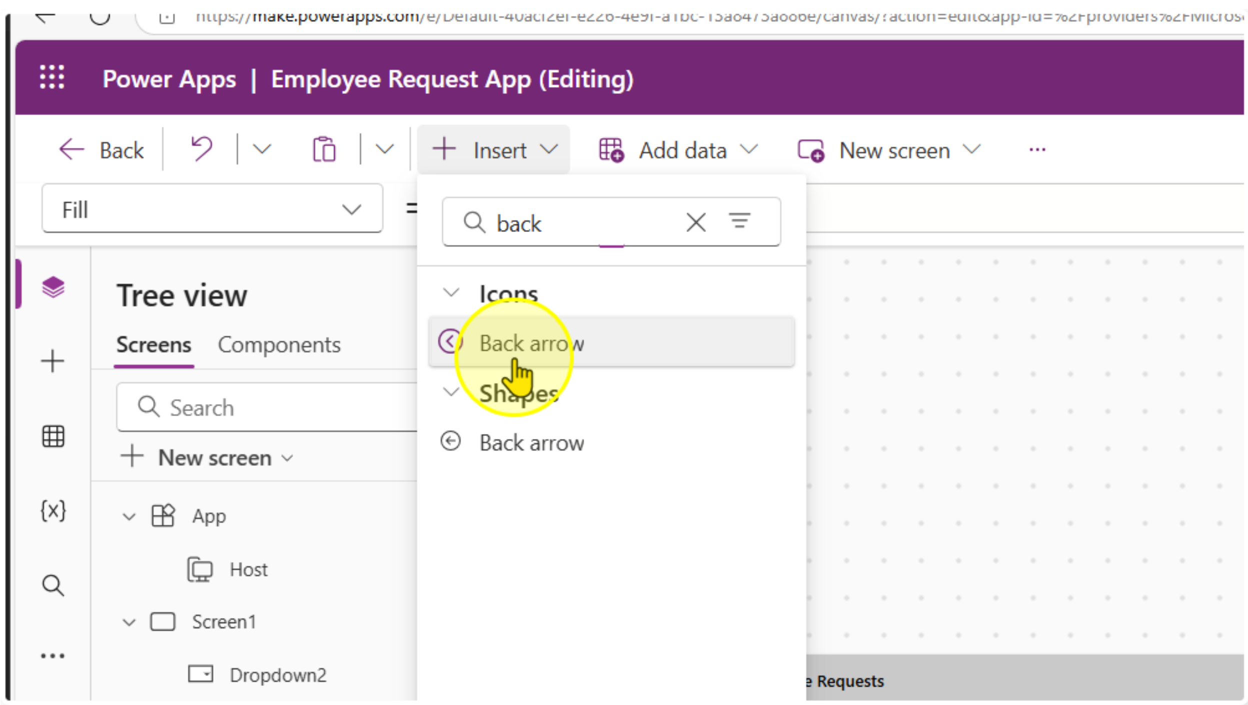
Be patient as this can sometimes take awhile.

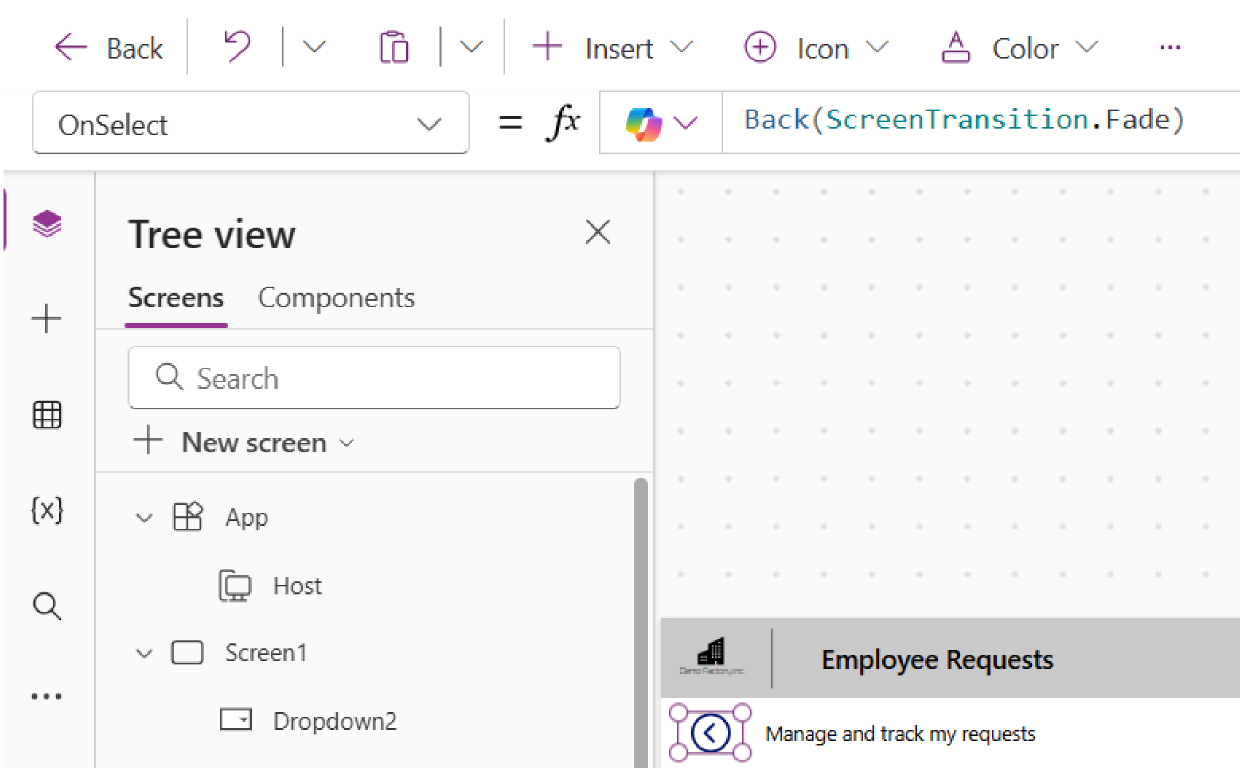
2. On the New Screen, Delete all of the content that is not present in the header.



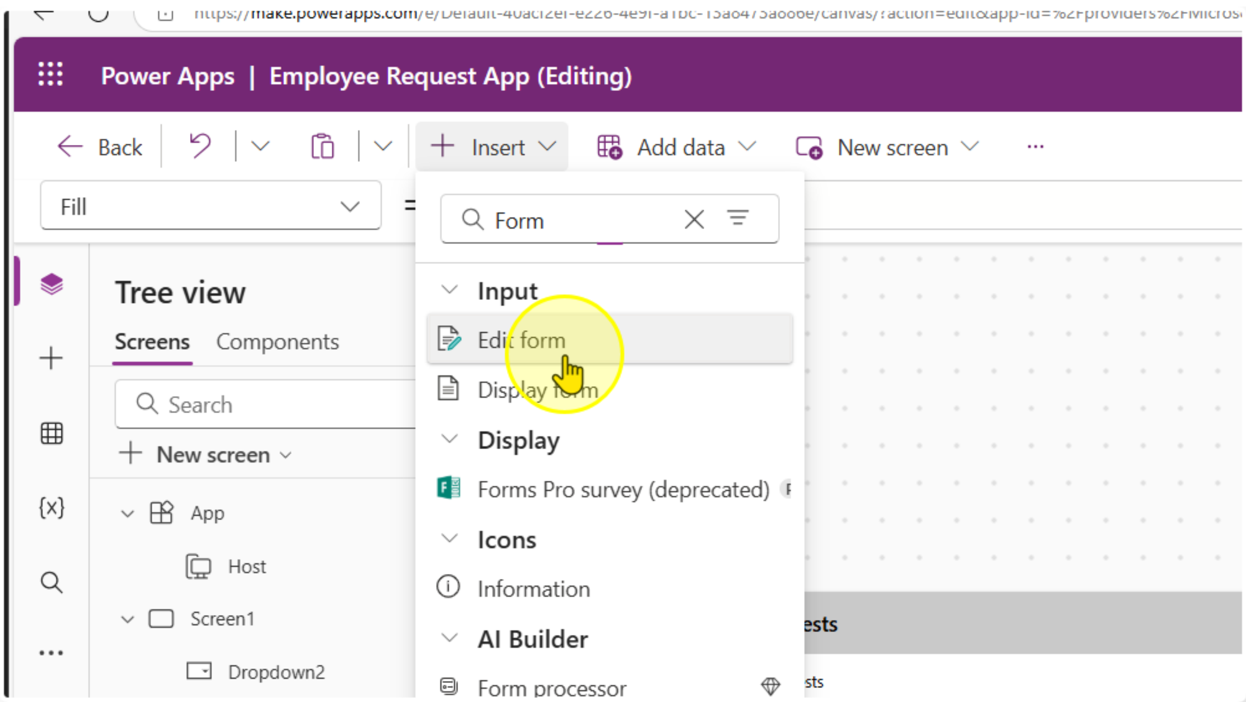
3. Add in a Back Arrow Icon. Size it and position it accordingly. On its OnSelect Property, enter the formula

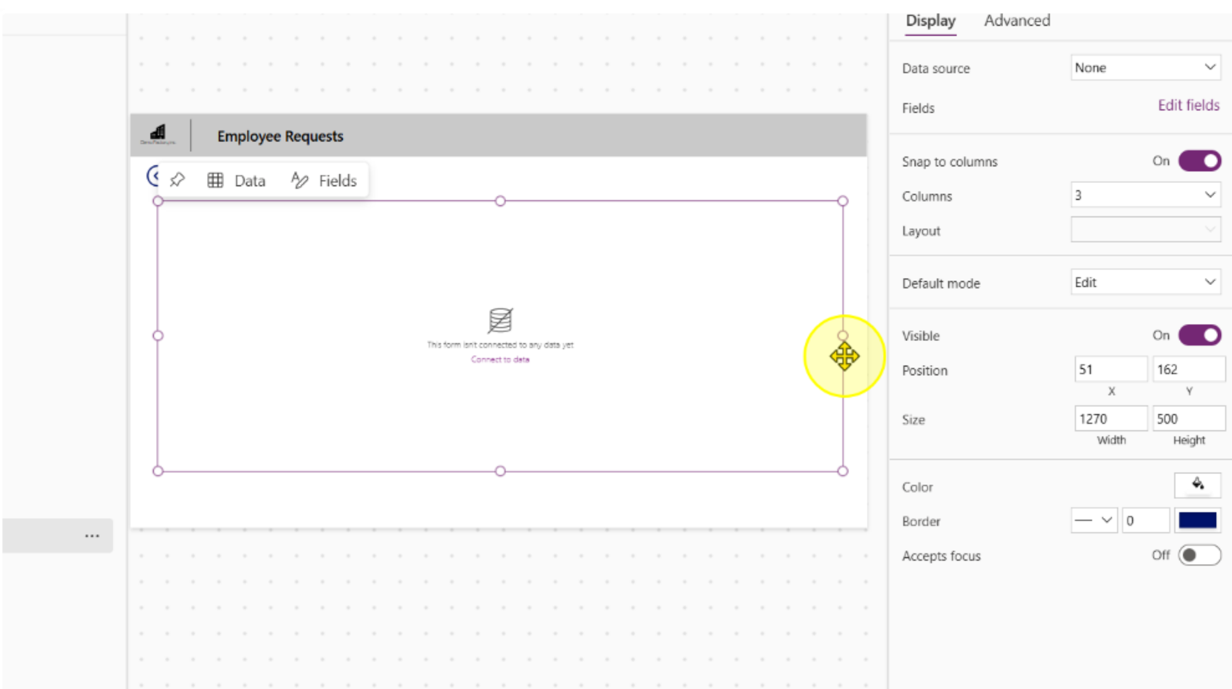
```
Back(ScreenTransition.Fade)
```





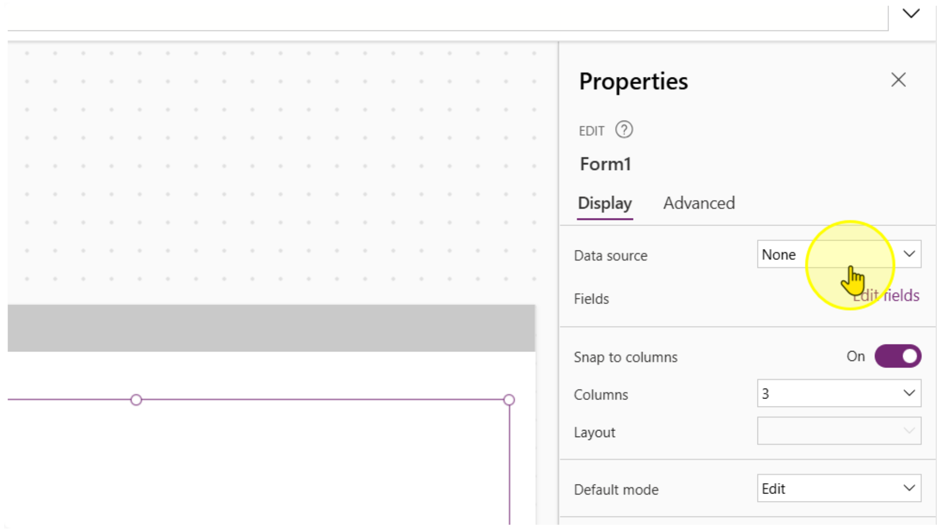
4. Add in a new Edit Form Control. Size it to occupy the screen as needed. Please leave some space at the bottom

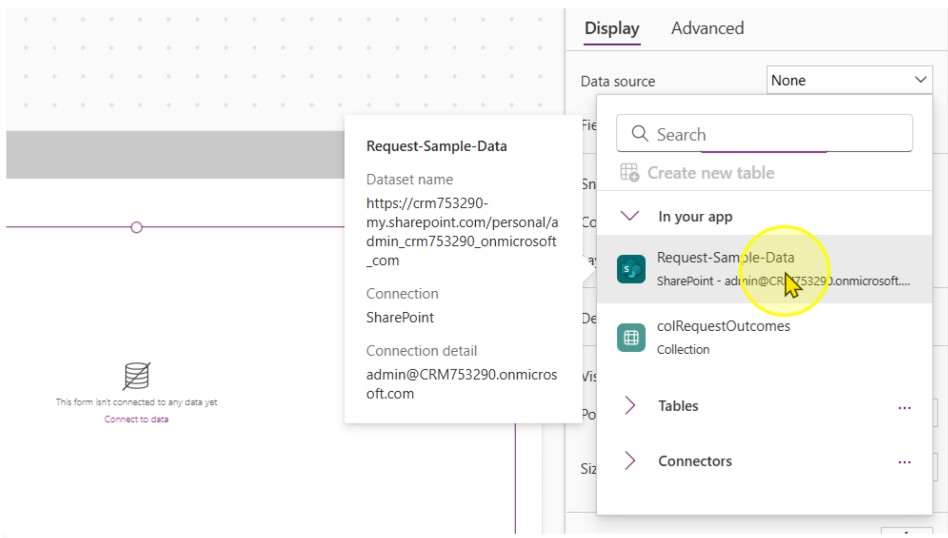




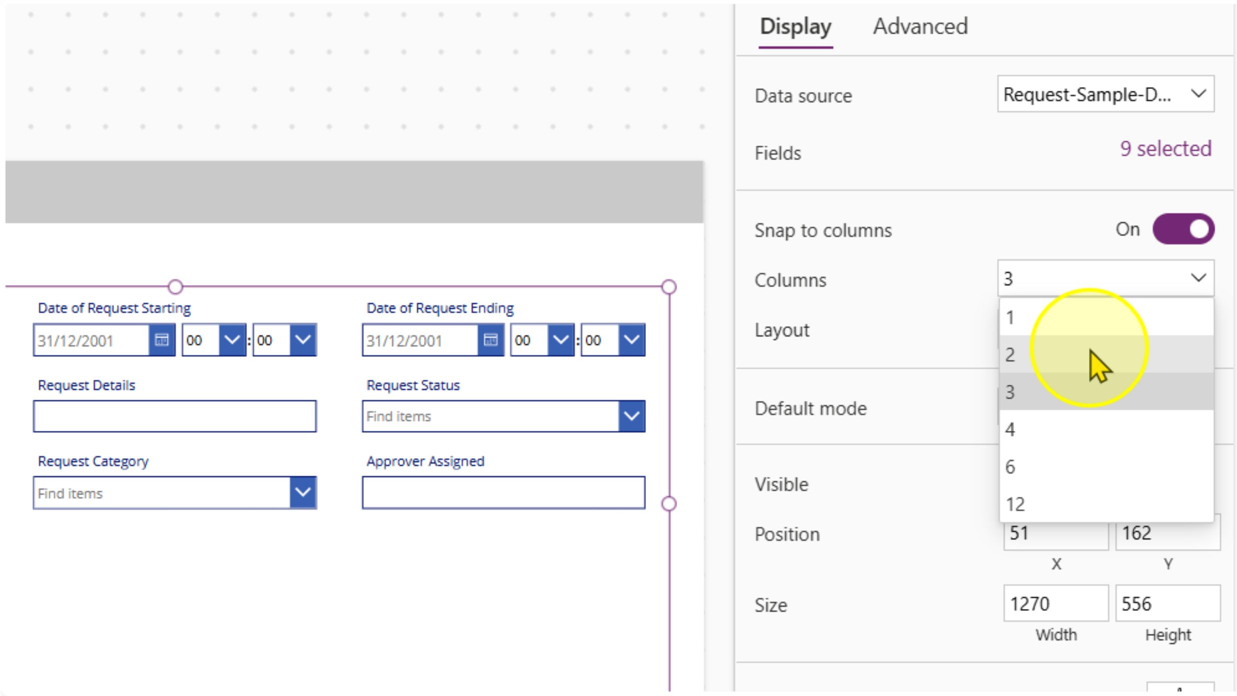
Microsoft
Power Platform

5. Point the Data Source for the Form to Microsoft List.





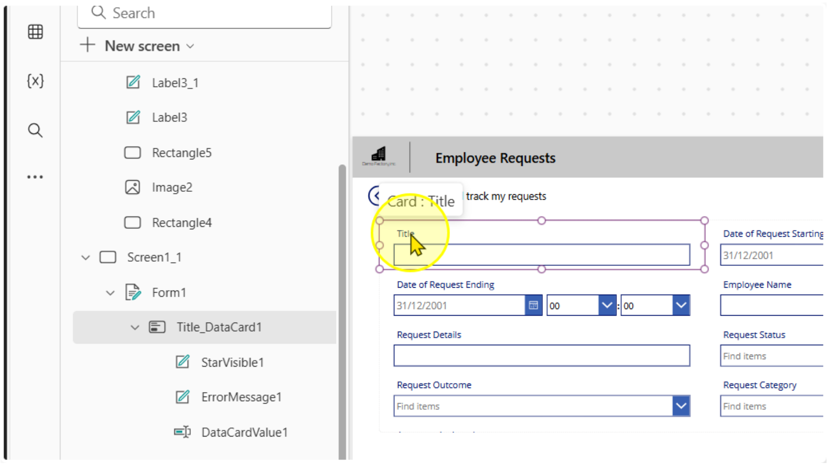
6. Change the number of Columns to 2

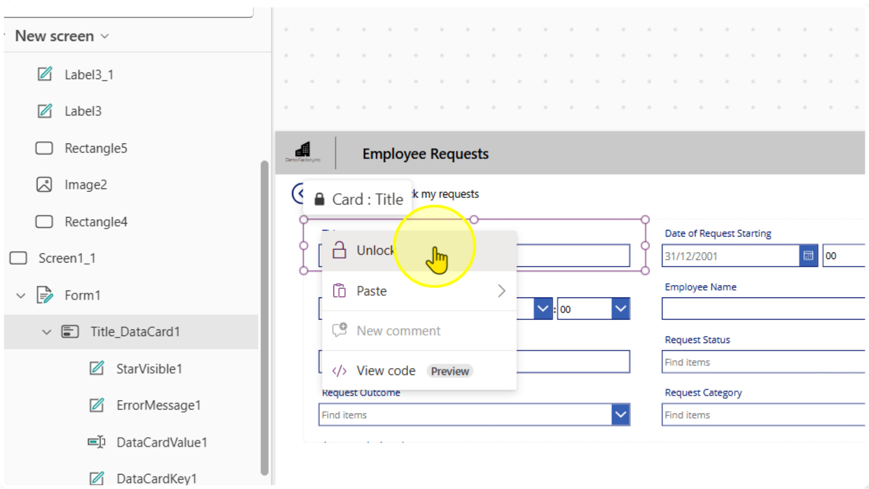


-- **End of Task** --

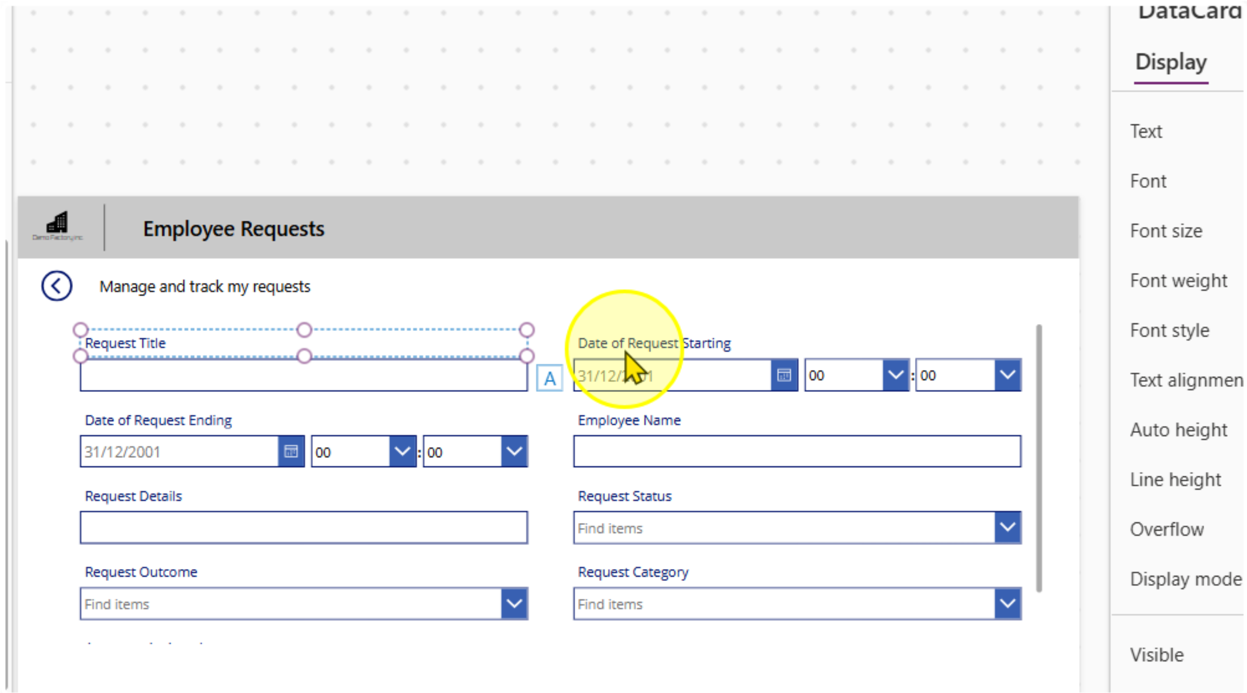
Task 2: Configuring the Form & Creating a submit button

1. Each Input box in a Form is encapsulated by a Card Control. They are all locked by default. You can unlock them by right clicking on a card and selecting the 'Unlock' option

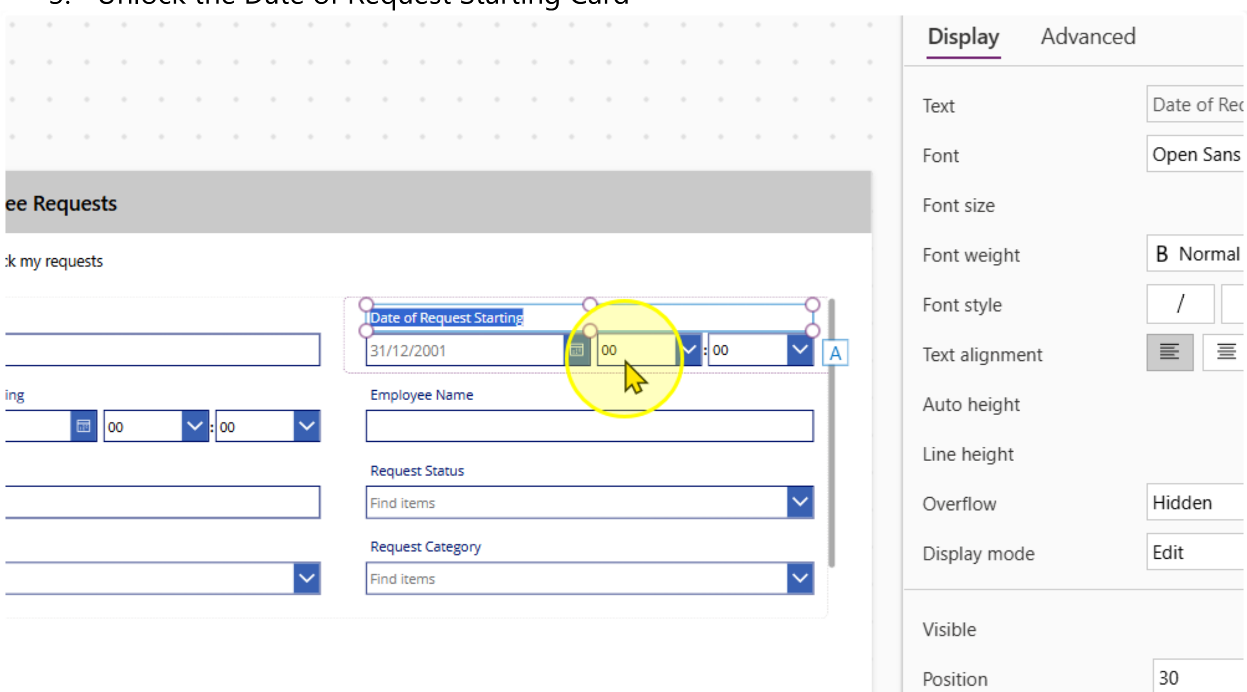




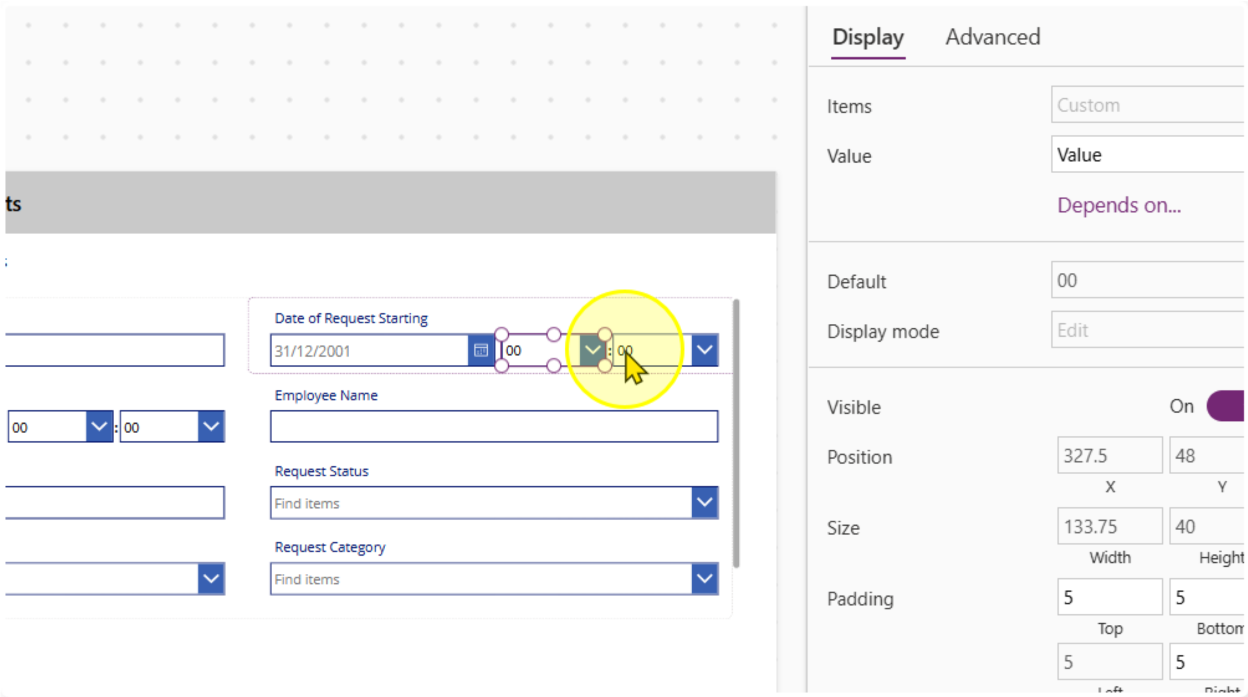
2. Rename the Title Column to “Request Title”



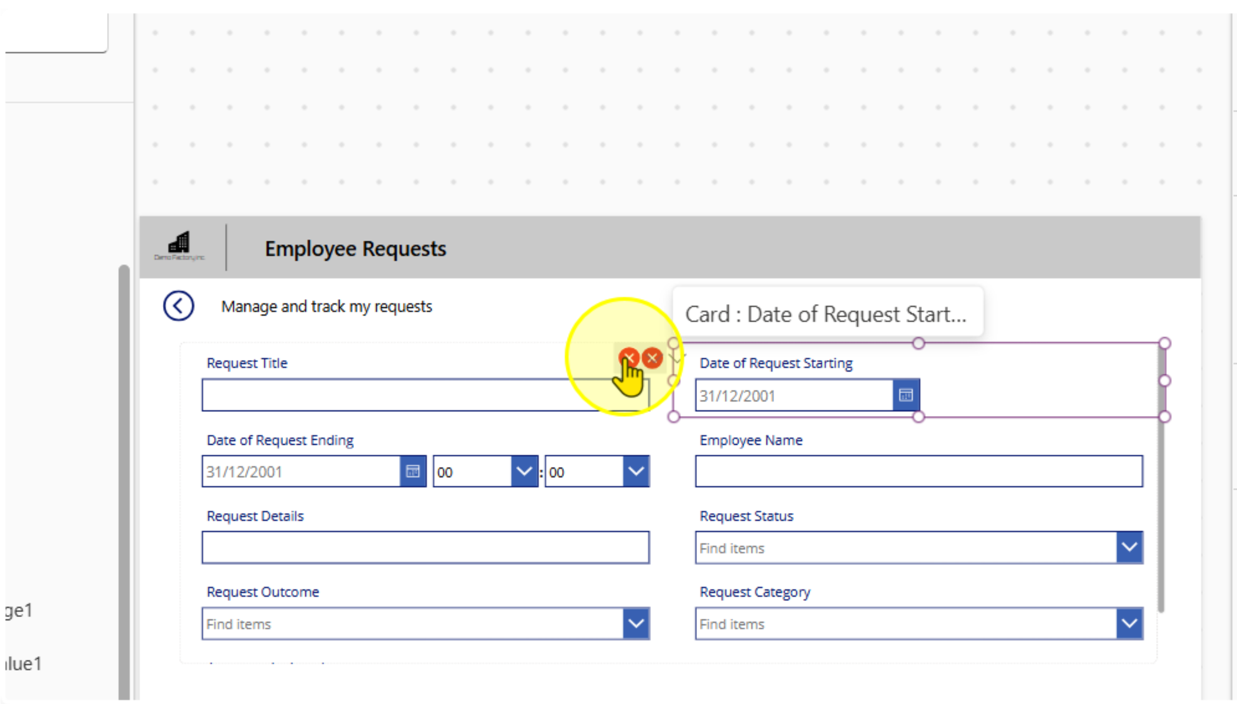
3. Unlock the Date of Request Starting Card



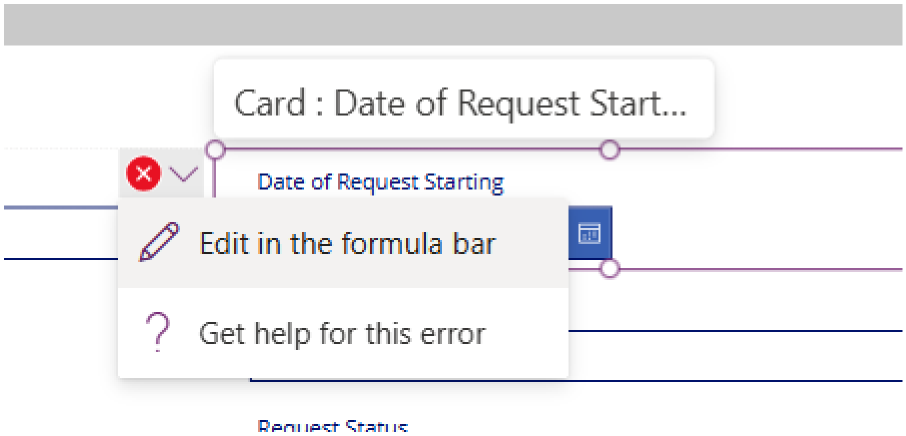
4. We want to delete the hour and minute input controls. Select them and delete them.



You should see several error messages pop up. It is expected to see them.



5. There are several errors we will need to correct. You can navigate to the respective error by clicking on the error message & selecting the 'Edit in the formula bar' option.



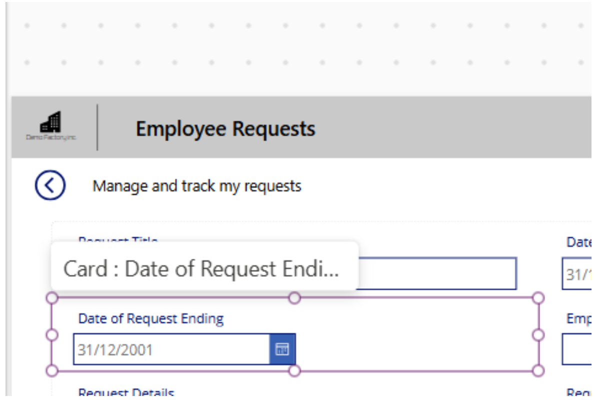
This is the list of errors you should be facing and the old and new values

Control & Property	Old value	New Value

Card | Update property	`If(Not IsBlank(DateValue1.SelectedDate),`
	`DateTime(Year(DateValue1.SelectedDate),`
	`Month(DateValue1.SelectedDate),`
	`Day(DateValue1.SelectedDate),`
	`Value(HourValue1.Selected.Value),`
	`Value(MinuteValue1.Selected.Value, 0))`
		`If(Not IsBlank(DateValue1.SelectedDate),`
		`DateTime(Year(DateValue1.SelectedDate),`
		`Month(DateValue1.SelectedDate),`
		`Day(DateValue1.SelectedDate), 0, 0, 0))`

ErrorMessage | Y	`HourValue1.Y + HourValue1.Height`	0
Separator | Y	`HourValue1.Y`	0
Separator | Height	`HourValue3.Height`	0
Separator | X	`HourValue3.X + HourValue3.Width`	0

6. Repeat the same for card for 'Date of Request Ending'



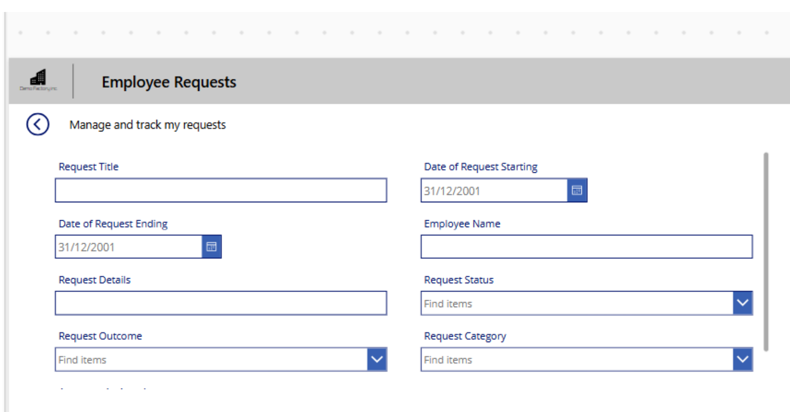
This is the list of errors you should be facing and the old and new values

Control & Property	Old value	New Value

Card | Update property	`If(Not IsBlank(DateValue2.SelectedDate),`
	`DateTime(Year(DateValue2.SelectedDate),`
	`Month(DateValue2.SelectedDate),`
	`Day(DateValue2.SelectedDate),`
	`Value(HourValue2.Selected.Value),`
	`Value(MinuteValue2.Selected.Value, 0))`
		`If(Not IsBlank(DateValue2.SelectedDate),`
		`DateTime(Year(DateValue2.SelectedDate),`
		`Month(DateValue2.SelectedDate),`
		`Day(DateValue2.SelectedDate), 0, 0, 0))`

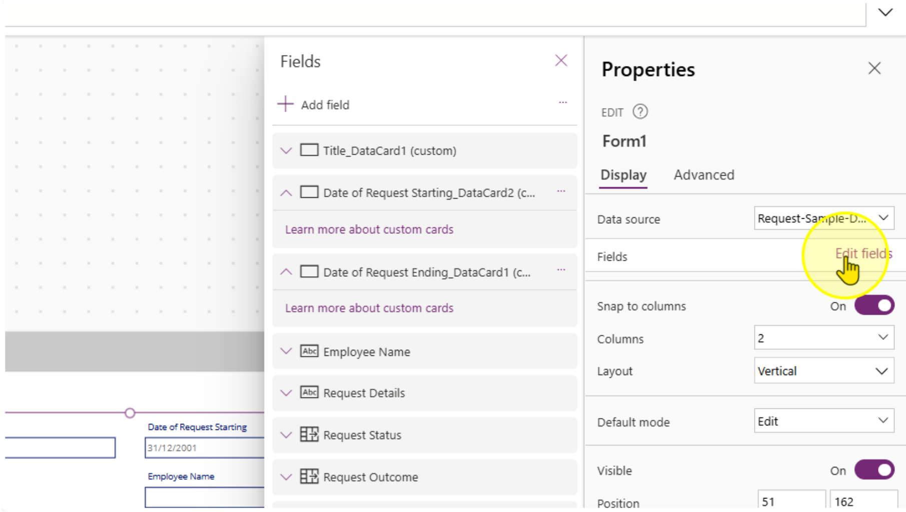
ErrorMessage | Y	`HourValue2.Y + HourValue2.Height`	0
Separator | Y	`HourValue2.Y`	0
Separator | Height	`HourValue2.Height`	0
Separator | X	`HourValue2.X + HourValue2.Width`	0

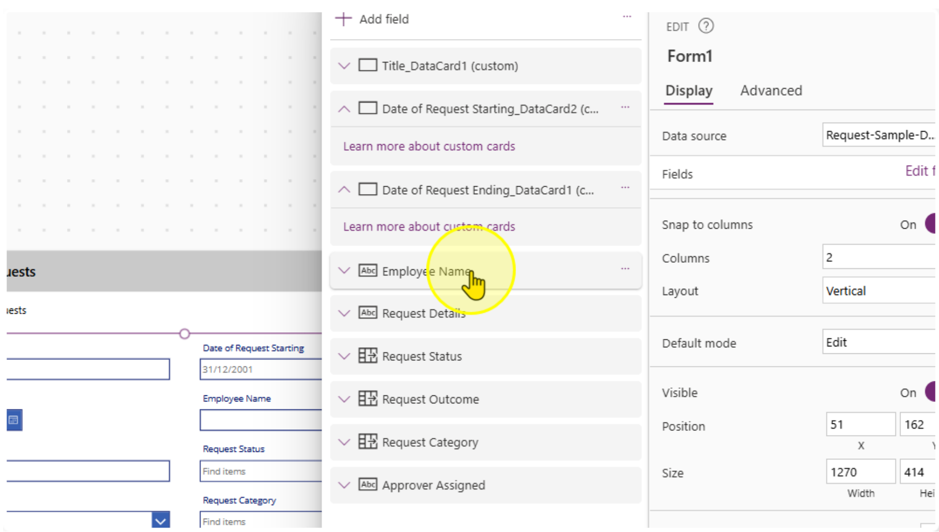
You should end up with a form that looks like this:



7. Reorder the forms to display the fields in the following order

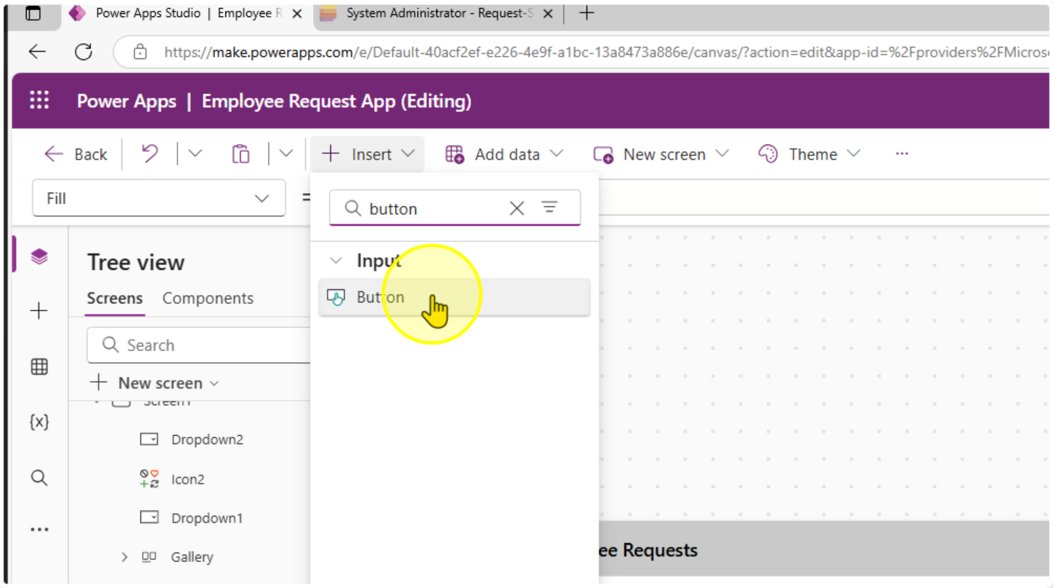
Title – Employee Name – Date of Request Starting – Date of Request Ending – Request Details – Request Category – Approver Assigned – Request Status – Request Outcome

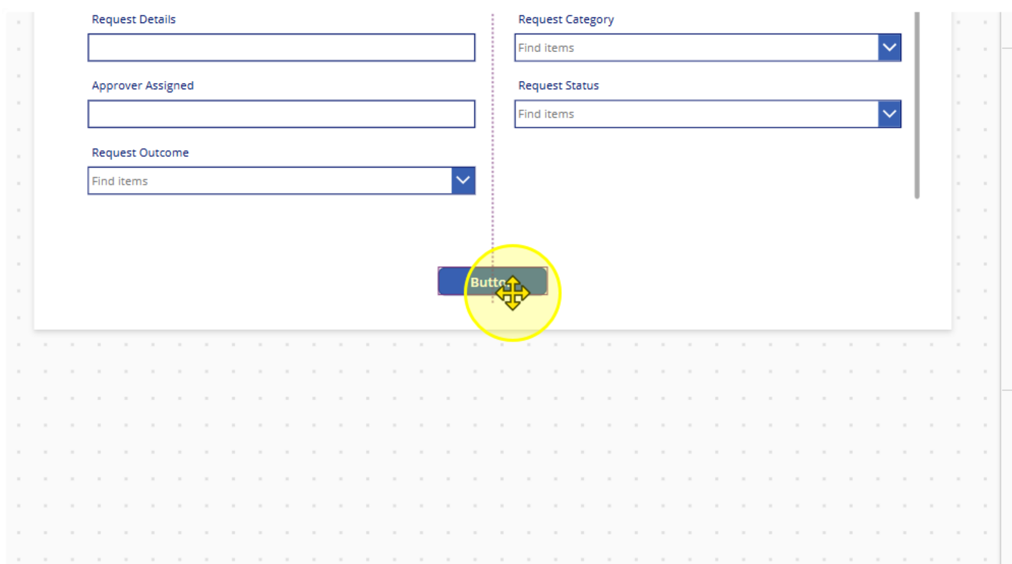




Microsoft Power Platform

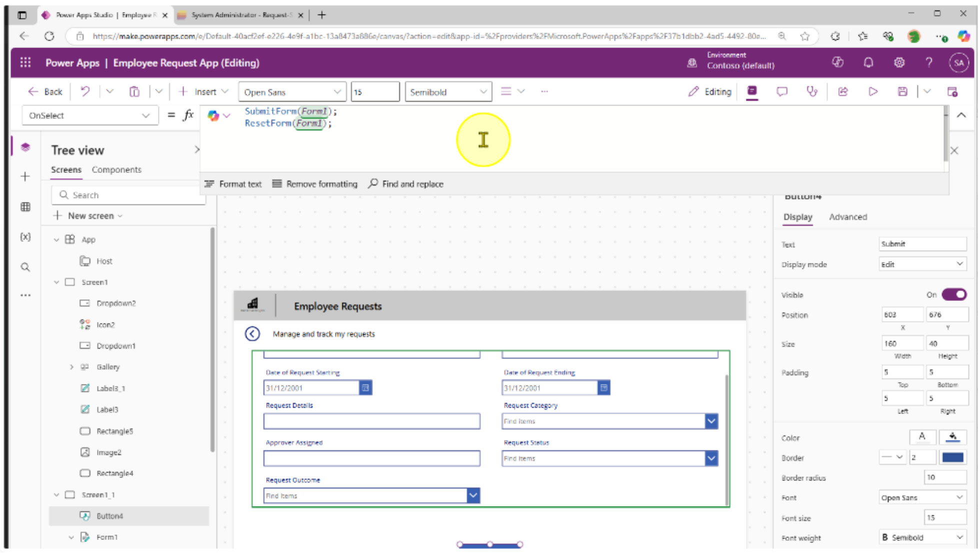
8. We will create a submit button for us to send the data to the Microsoft List. Add in a button control and position it at the center of the page at the bottom



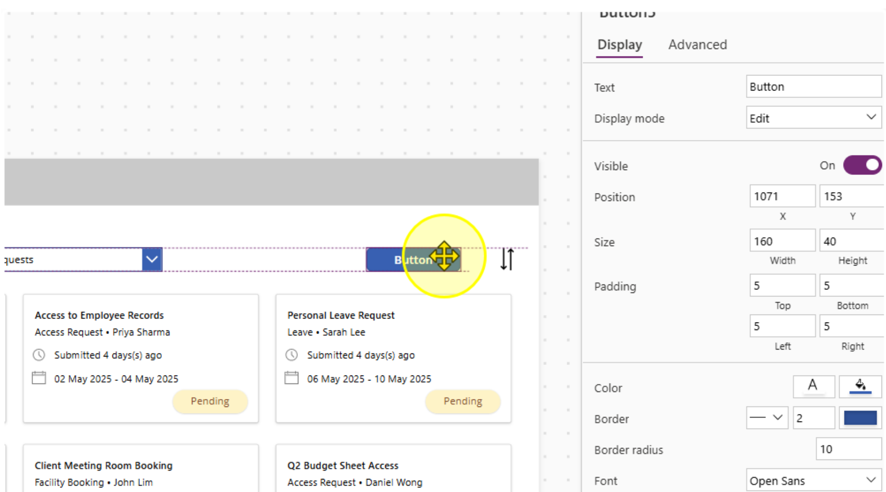


9. Change the Text property of the button to “Submit”. Then Change the OnSelect property to the following value

```
SubmitForm(Form1);
ResetForm(Form1);
```

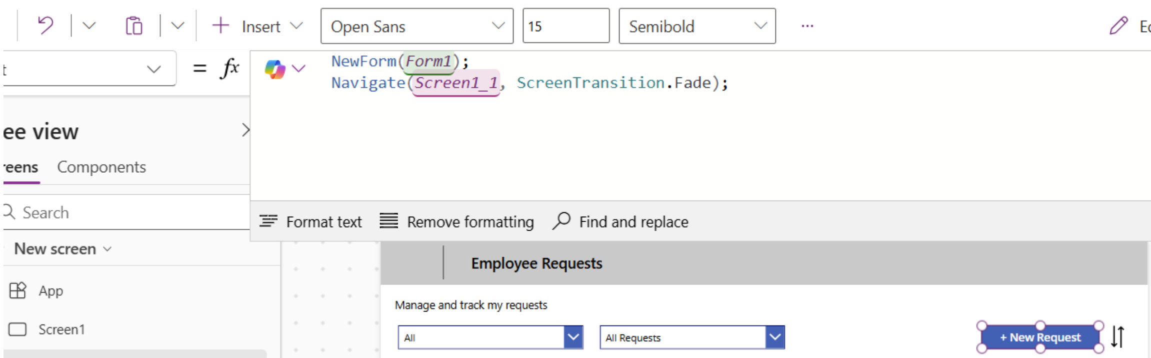


10. Navigate back to the first screen. Here we will add in a button for a New Request.



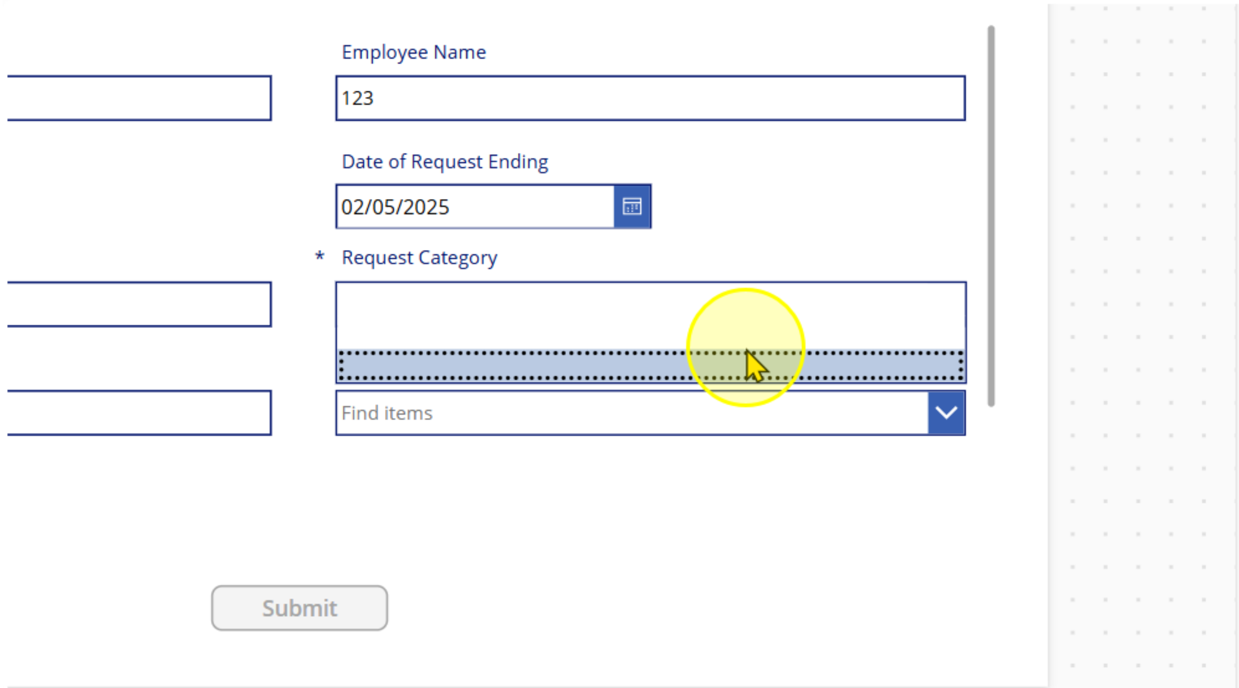
11. Change the Text property of the button to "+ New Request". Also, edit the OnSelect Property to the following values.

```
NewForm(Form1);
Navigate(Screen1_1, ScreenTransition.Fade);
```



Resize the button as needed.

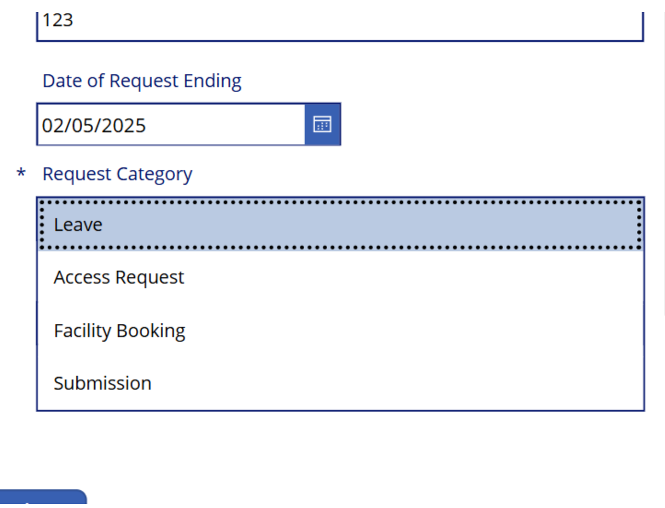
12. Note that some of the Fields with Dropdowns may not display values



To Fix this problem, add change the Items property to

```
Choices([@'Request-Sample-Data'].'field_7').Value
```

And refresh the datasource.

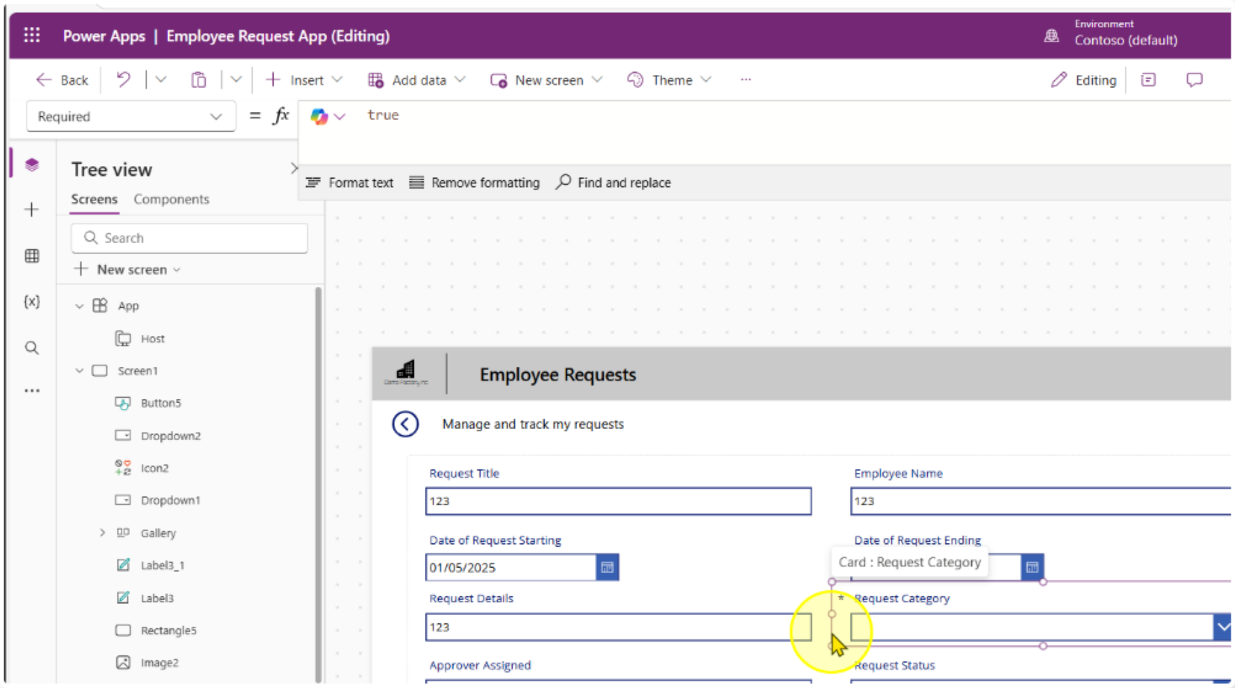


-- **End of Task** --

Task 3: Additional Controls to set on Form Submission

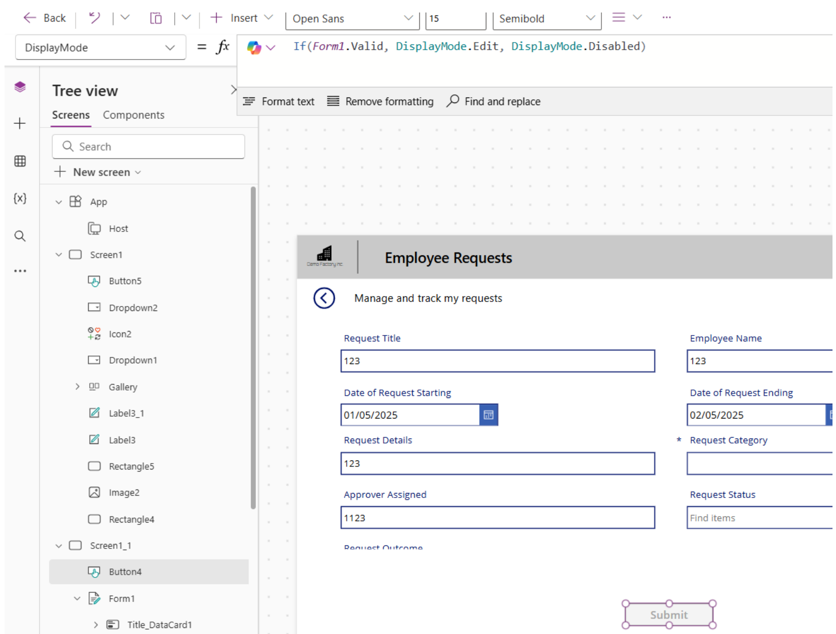
We can set controls on the form to only allow submissions when a form is valid.

1. Set the Request Category Card to be a Required field by setting the Required property to the value of true.



2. On the submit button, change the display mode of the button to the following expression

```
If(Form1.Valid, DisplayMode.Edit, DisplayMode.Disabled)
```



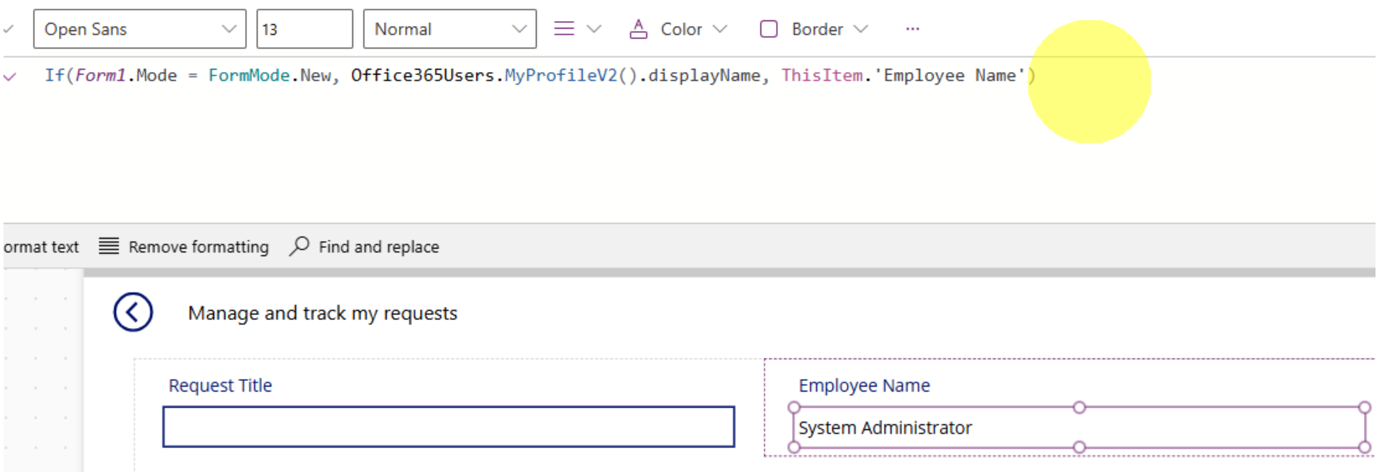
--- End of Task ---

Task 4: Default & Pre-populated values for Forms

1. Unlock the **Employee Name Card**. Edit the Input Control's Default and DisplayMode properties to contain the following expression.

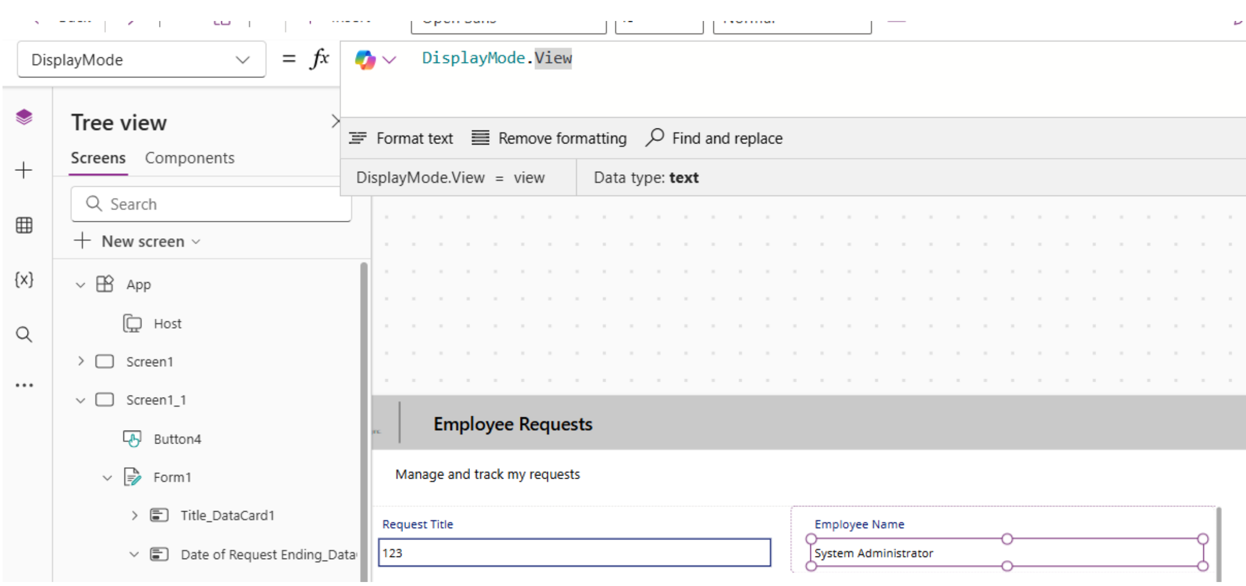
Default

```
If(Form1.Mode = FormMode.New, Office365Users.MyProfileV2().displayName,
ThisItem.'Employee Name')
```



DisplayMode

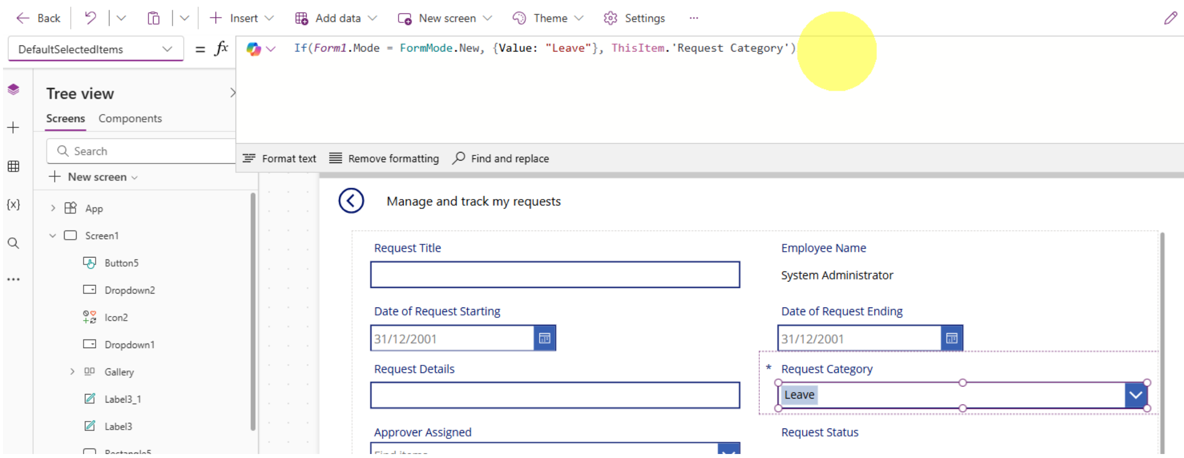
```
DisplayMode.View
```



2. Unlock the **Request Category** Card. Edit the Input Control's DefaultSelectedItems property to contain the following expression.

DefaultSelectedItems

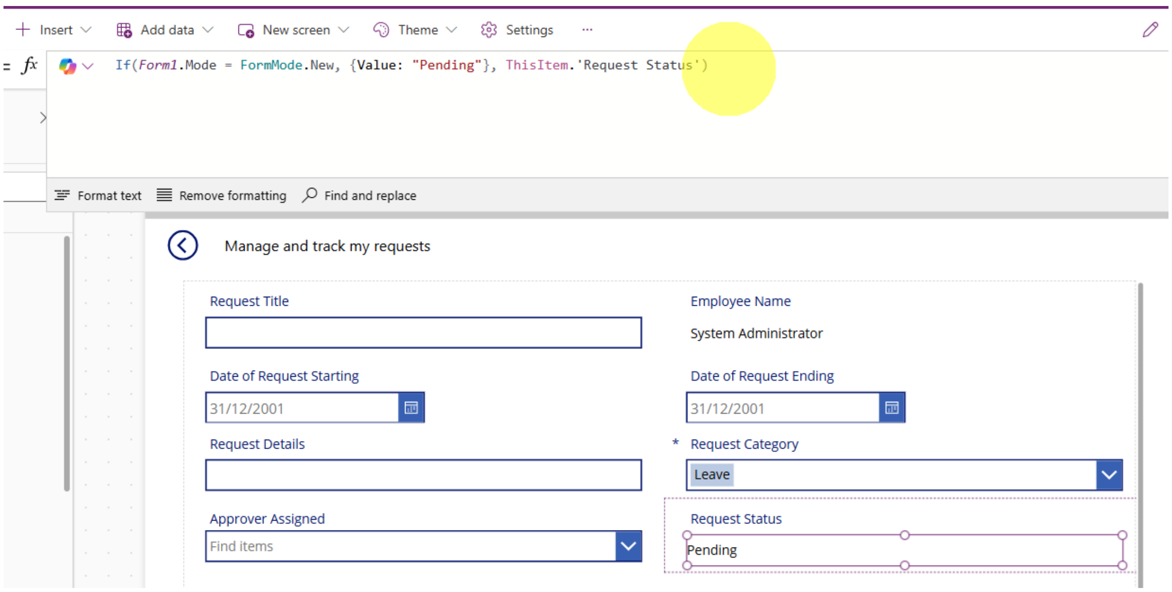
```
If(Form1.Mode = FormMode.New, {Value: "Leave"}, ThisItem.'Request Category')
```



3. Unlock the **Request Status** Card. Edit the Input Control's DefaultSelectedItems and DisplayMode properties to contain the following expression.

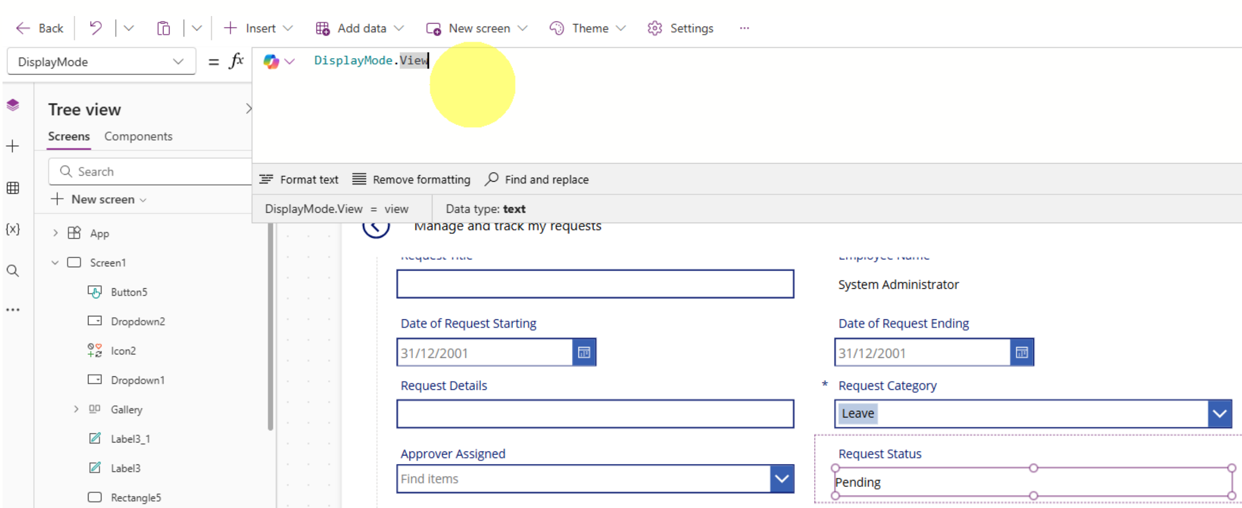
DefaultSelectedItems

```
If(Form1.Mode = FormMode.New, {Value: "Pending"}, ThisItem.'Request Status')
```



DisplayMode

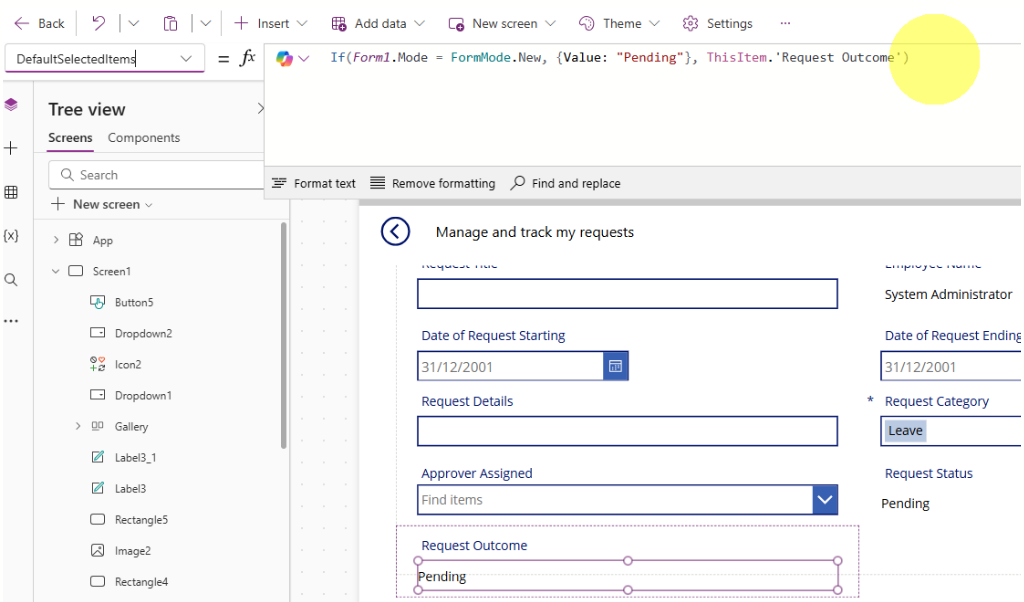
```
DisplayMode.View
```



4. Unlock the **Request Outcome** Card. Edit the Input Control's DefaultSelectedItems and DisplayMode properties to contain the following expression.

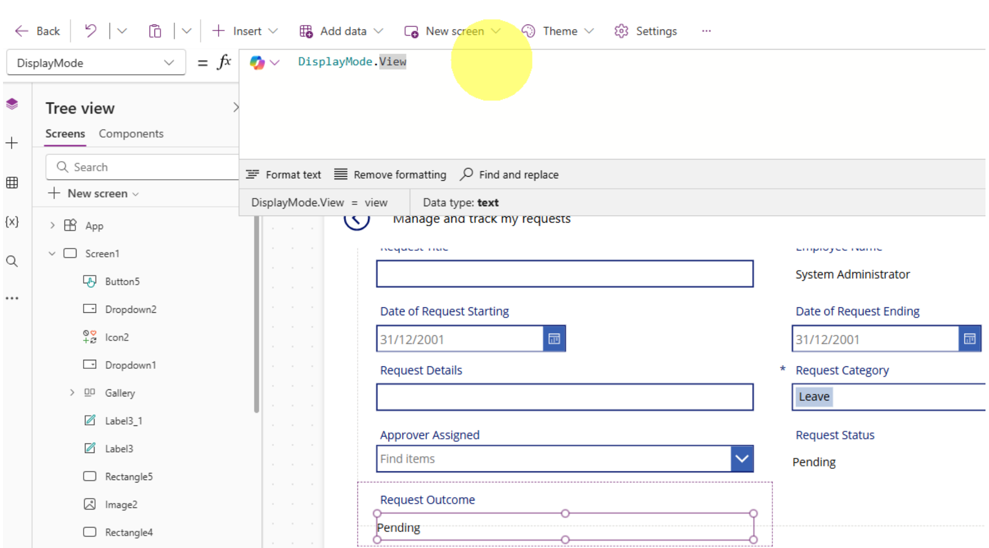
DefaultSelectedItems

```
If(Form1.Mode = FormMode.New, {Value: "Pending"}, ThisItem.'Request
Outcome')
```



DisplayMode

```
DisplayMode.View
```

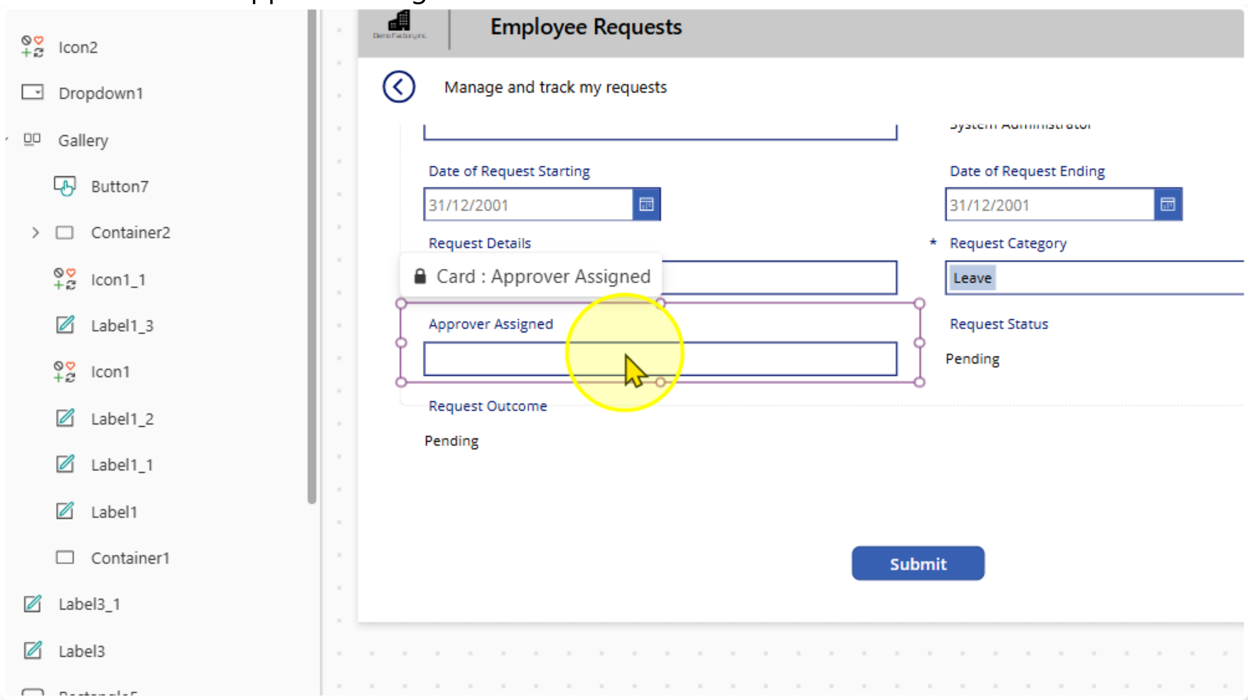


-- **End of Task** --

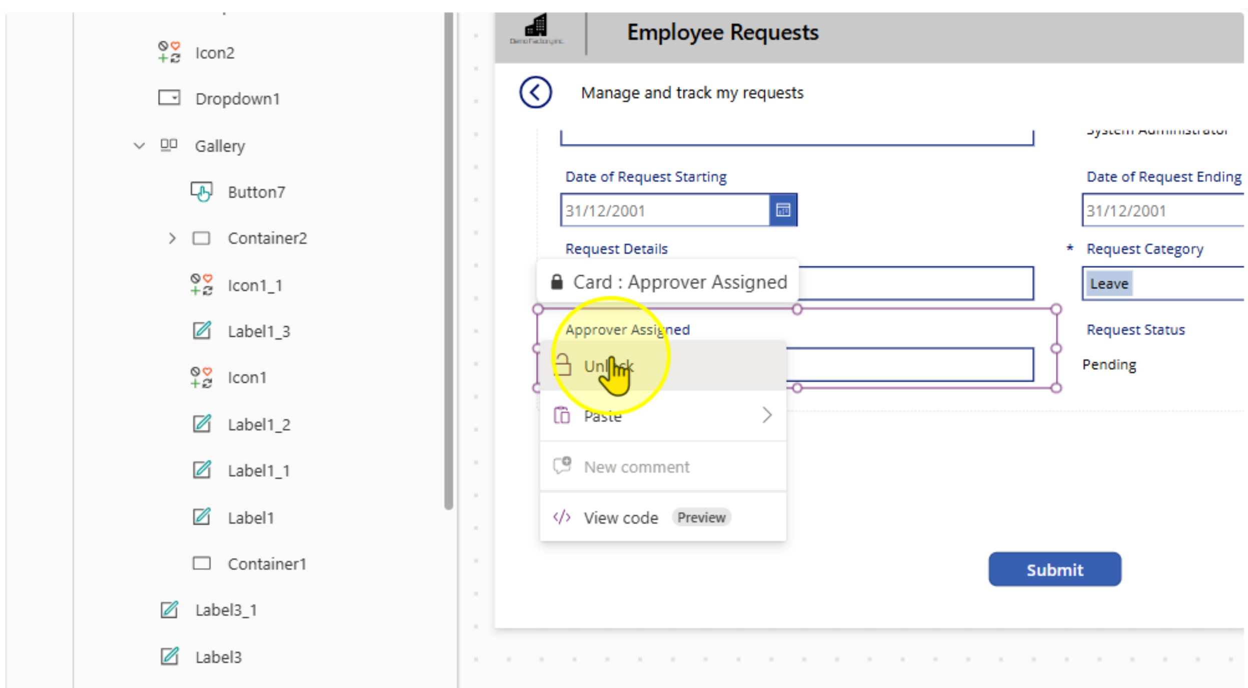
Microsoft Power Platform

Task 5: Allow users to search for An Approver

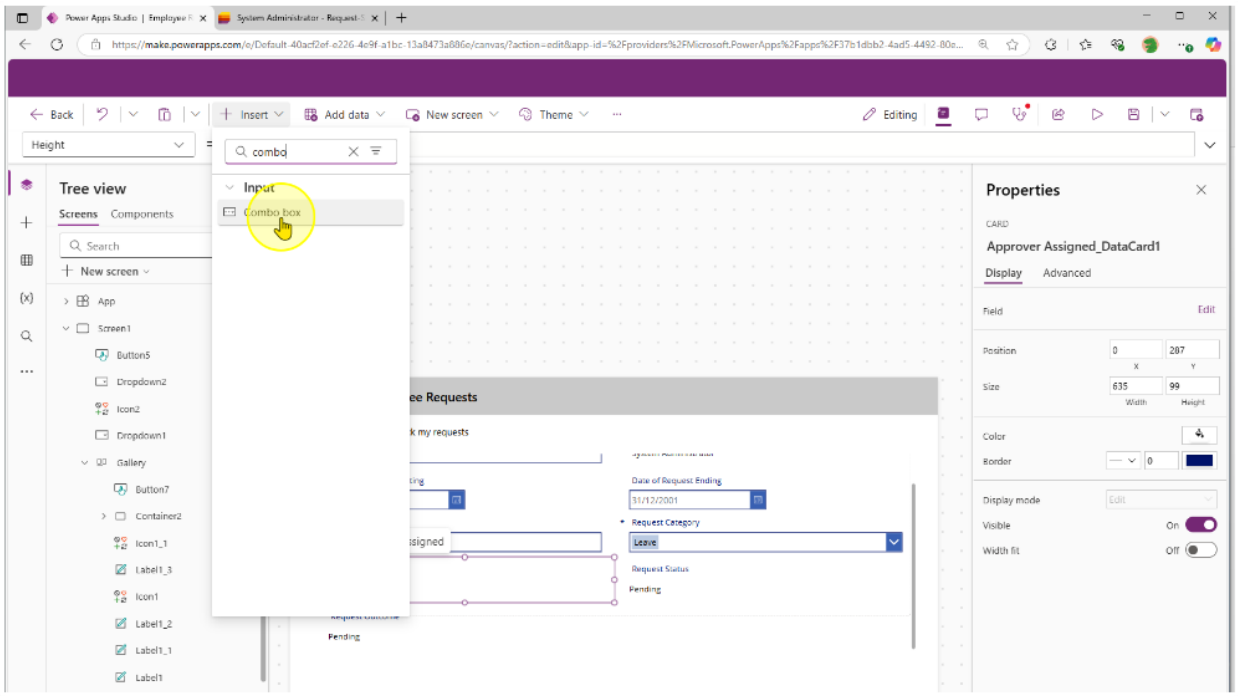
1. Select the Approver Assigned Card



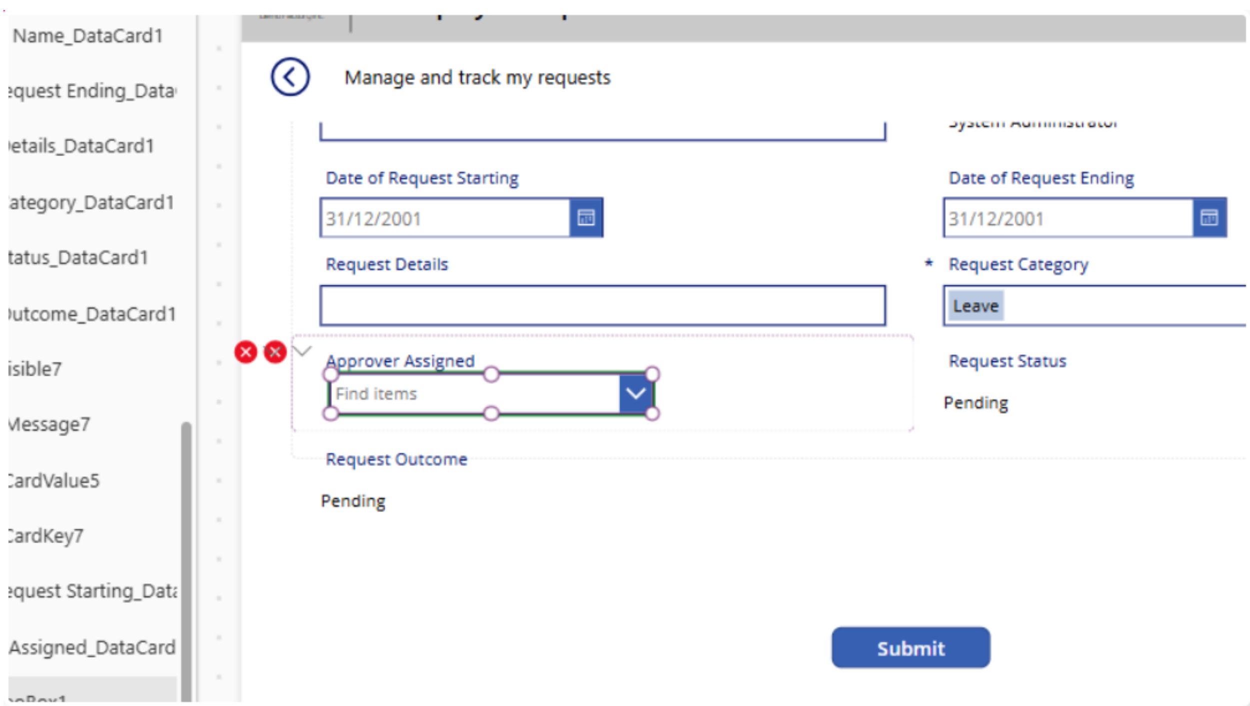
2. Unlock the card and select the Text input control. Delete it.



3. Without clicking anywhere on the page, navigate to the insert button and insert in a 'Combobox' Control

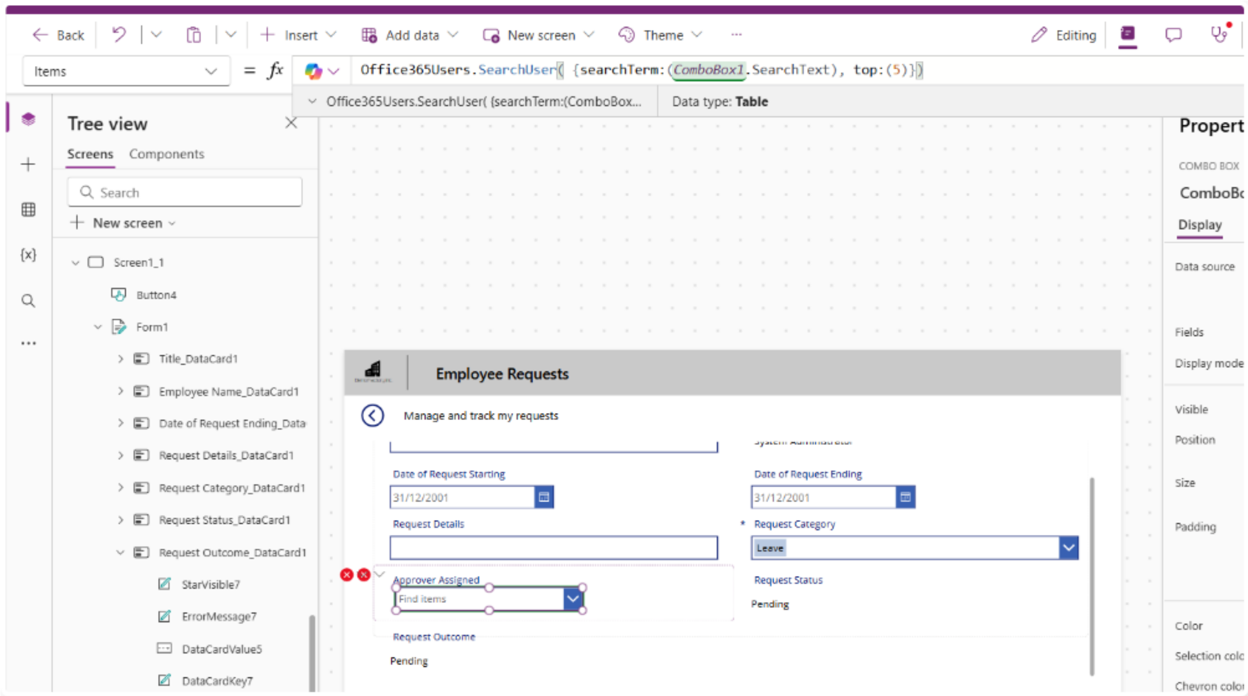


It is expected to see the 2 errors.



4. Change the 'Items' property on the combobox to the following

```
Office365Users.SearchUser( {searchTerm:(ComboBox1.SearchText), top:(5)})
```

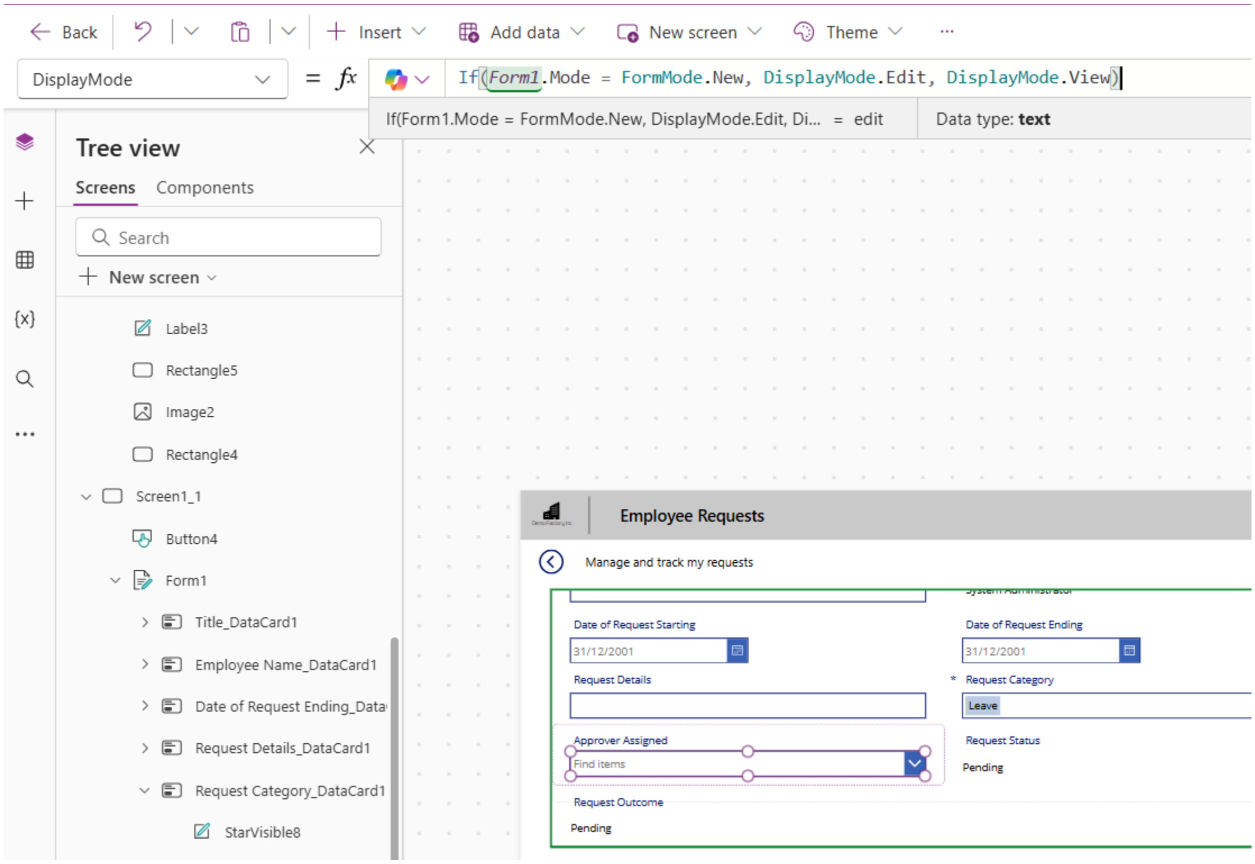


5. Size & position the combobox as needed. To get rid of the errors, select the error icons and edit each to the following properties

Control & Property	Old value	New Value
Card | Update	`DataCardValue7.Text`	`ComboBox1.Selected.Mail`
ErrorMessage | Y	`HourValue1.Y + HourValue1.Height`	`0`

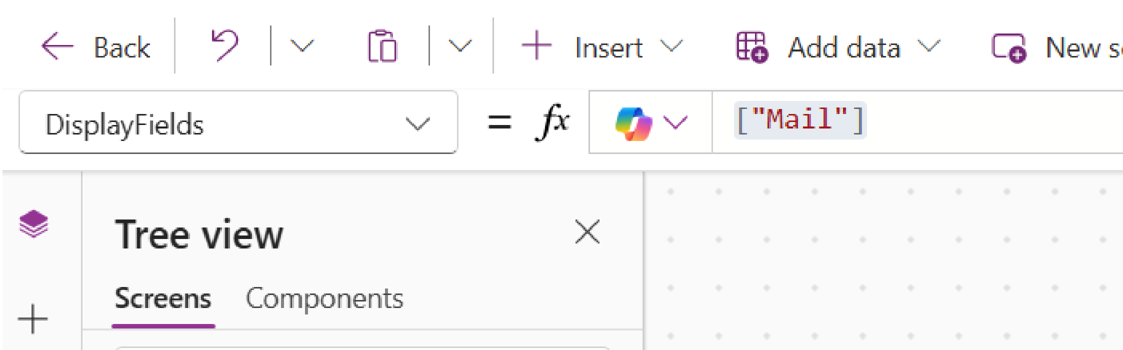
6. Edit the ‘DisplayMode’ Property of the combobox control to the following value:

```
If(Form1.Mode = FormMode.New, DisplayMode.Edit, DisplayMode.View)
```



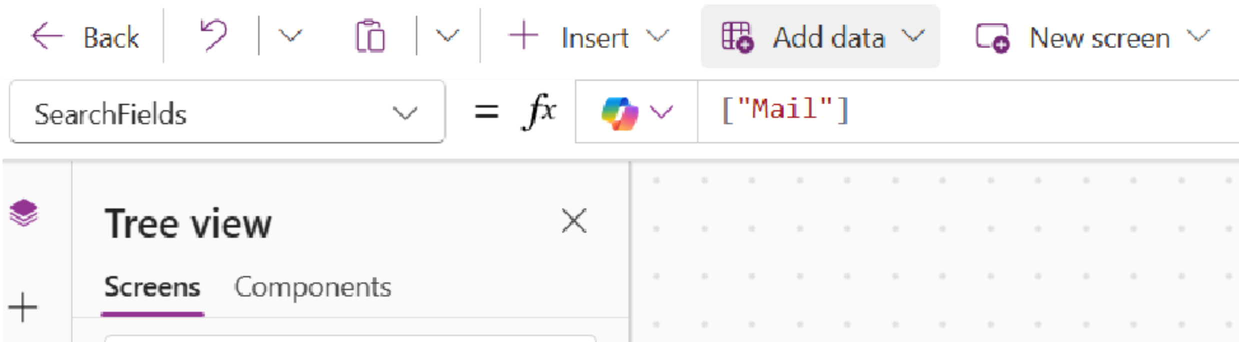
7. Edit the ‘DisplayFields’ Property of the combobox control to the following value:

```
["Mail"]
```



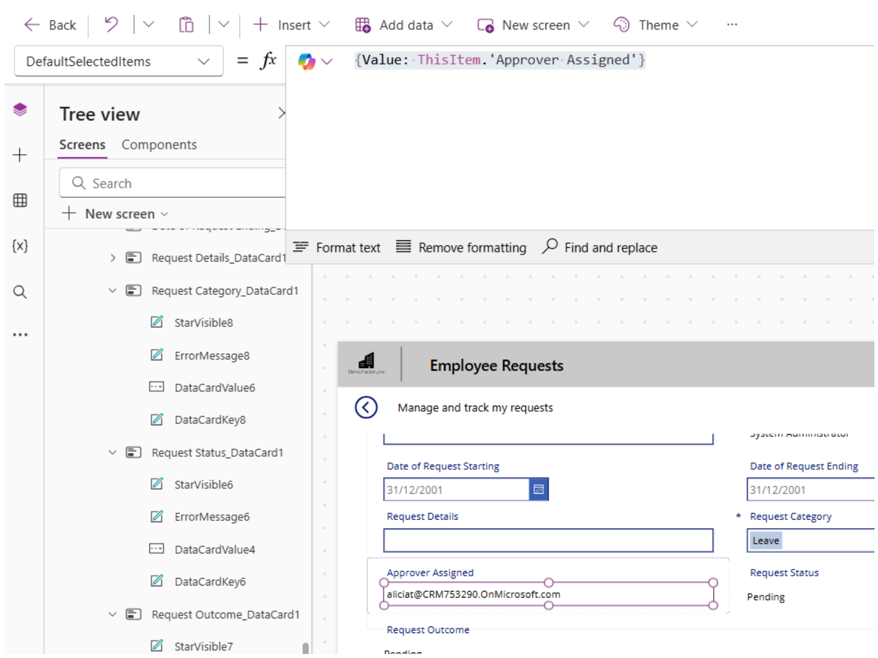
8. Edit the ‘SearchFields’ Property of the combobox control to the following value:

```
["Mail"]
```



9. Edit the DefaultSelectedItems property to the following value

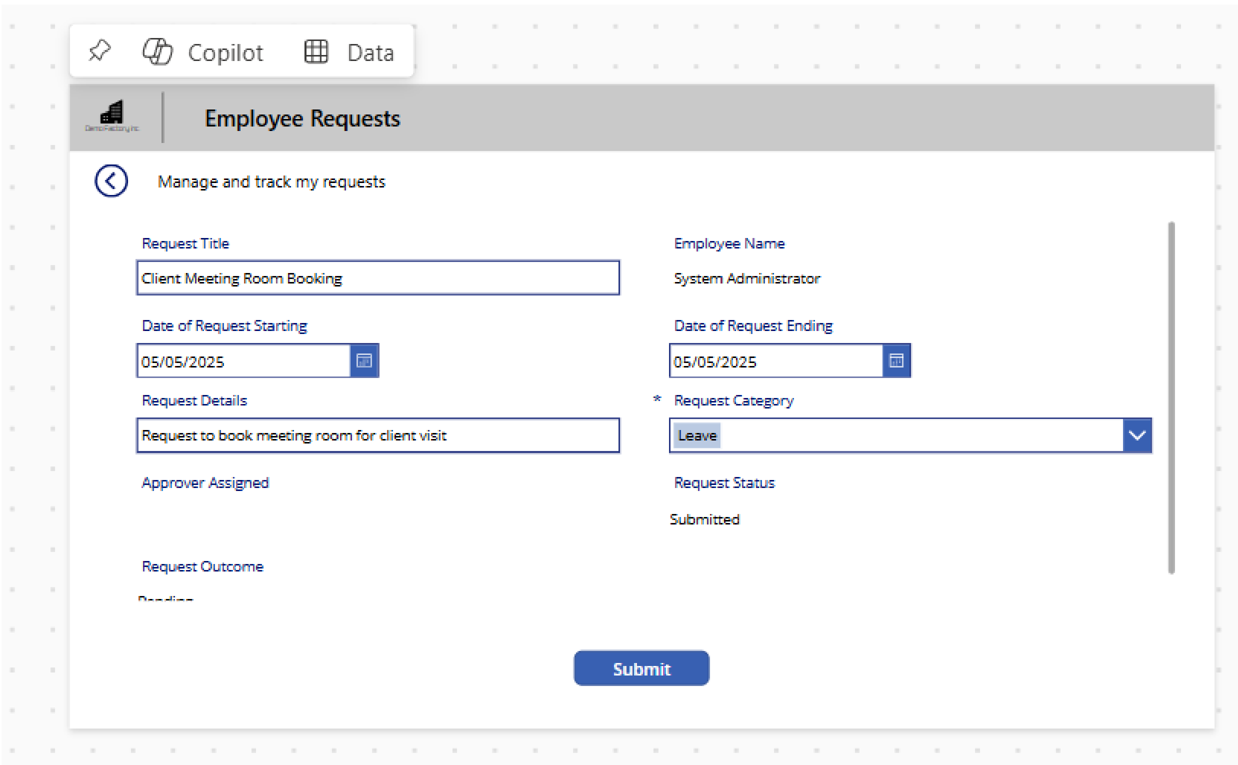
```
{Value: ThisItem.'Approver Assigned'}
```



-- **End of Task** --

Exercise 6: Allow Users to Edit Existing Requests

Users will also need to edit existing requests in case the enter the wrong information.



Objectives

- Allow users to select an existing request for editing
- Creating conditions so users can only edit certain requests

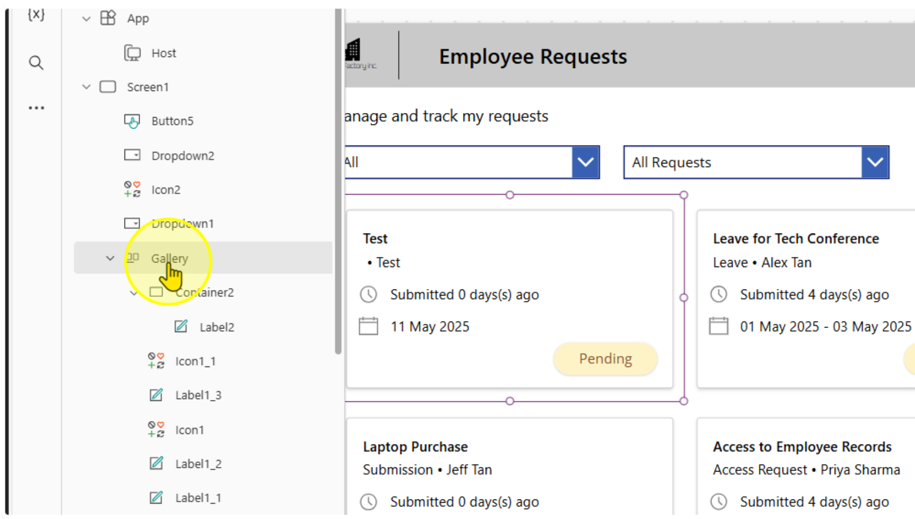
Estimated time to complete this lab

20 mins

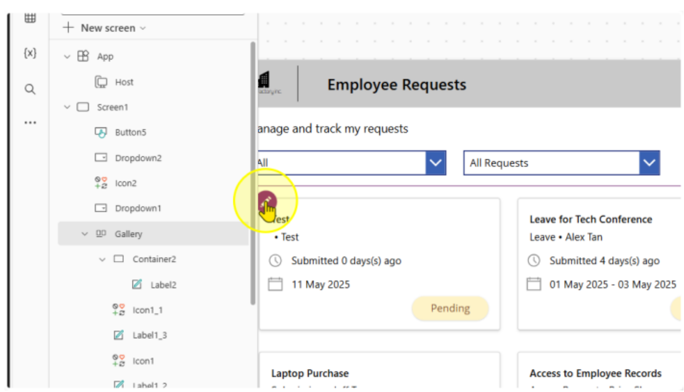
Task 1: Adding a Navigate Button to the Main Screen

1. Add in a button on the Gallery on the Main Screen.

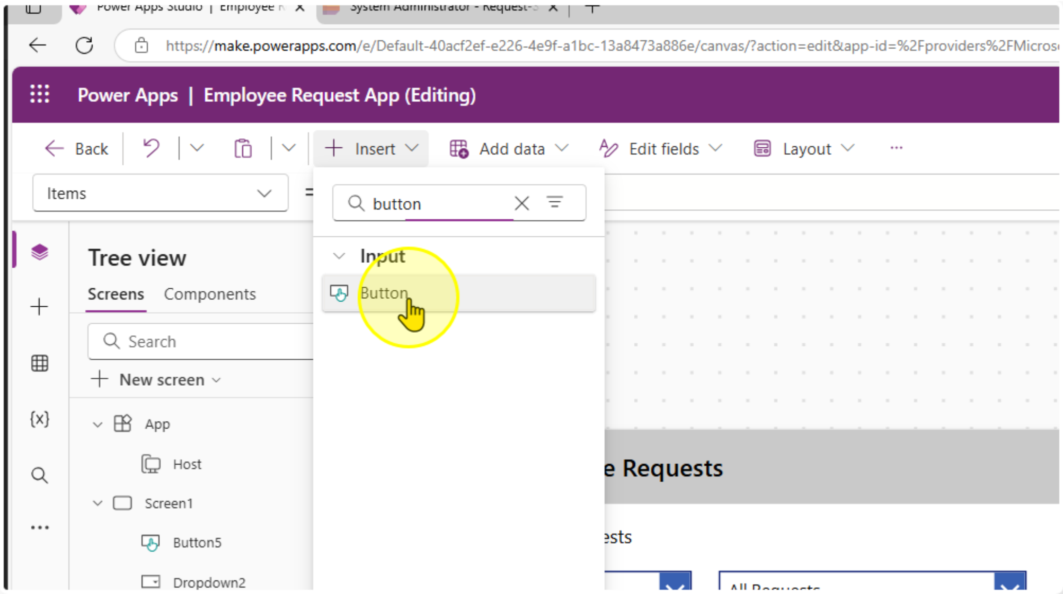
First, make sure you select the gallery



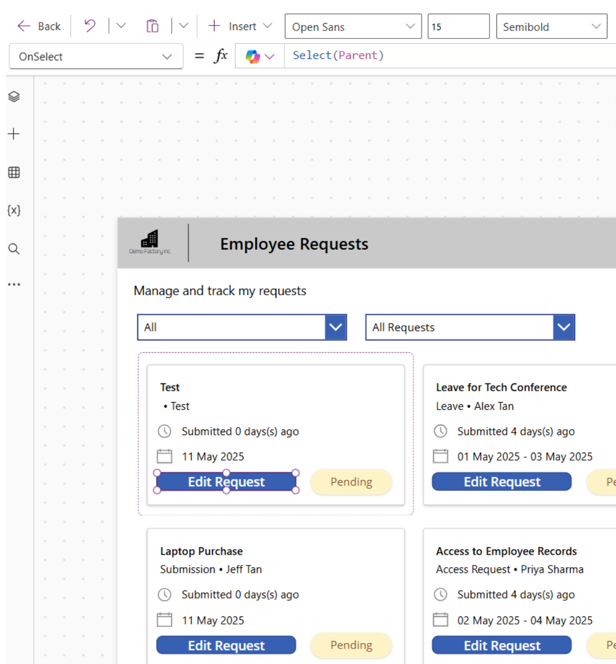
Click the Edit Button in the gallery template (First Record in the gallery, usually top left record at the top left hand corner)



Without clicking anywhere else on the page, click on the insert button and insert a button control

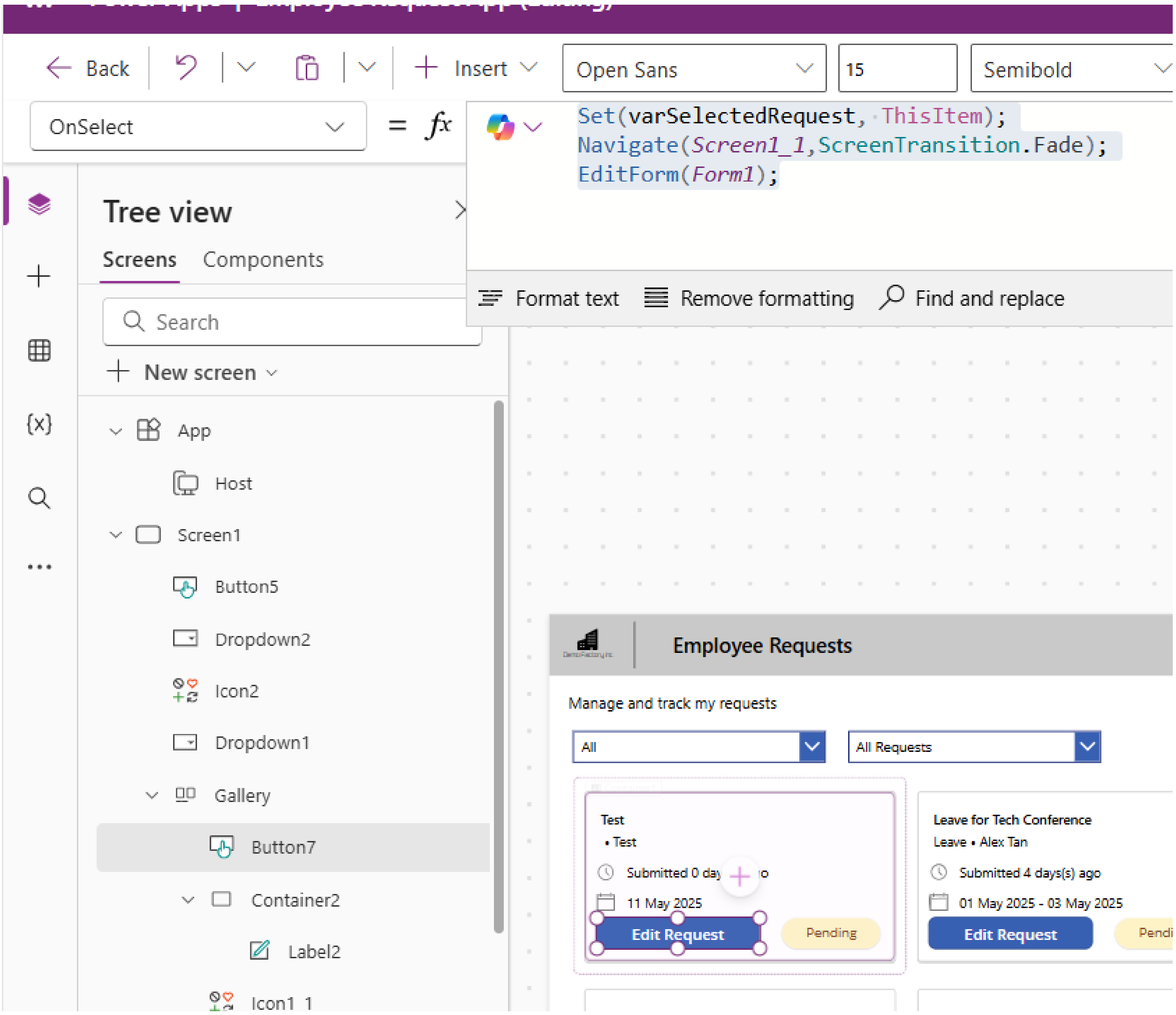


You should be able to create a button in the gallery. Change the text of the button to “Edit Request”



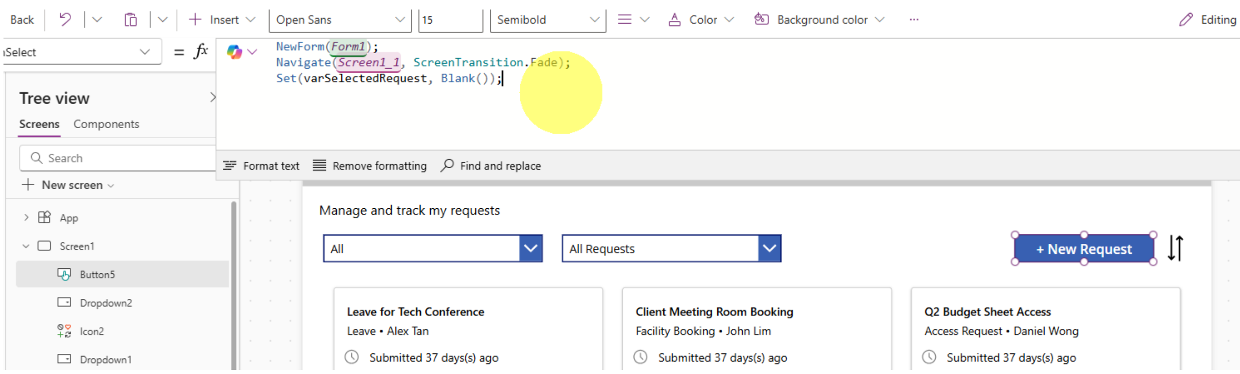
2. Change the OnSelect property of the button to the following expression

```
Set(varSelectedRequest, ThisItem);
Navigate(Screen1_1,ScreenTransition.Fade);
EditForm(Form1);
```

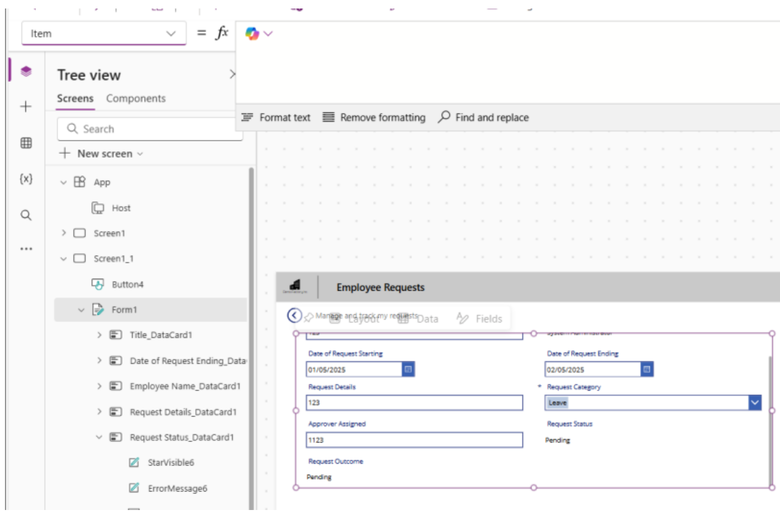


3. Edit the OnSelect Property of the "+ New Request" Button to reset the varSelectedRequest value.

```
NewForm(Form1);
Navigate(Screen1_1, ScreenTransition.Fade);
Set(varSelectedRequest, Blank());
```

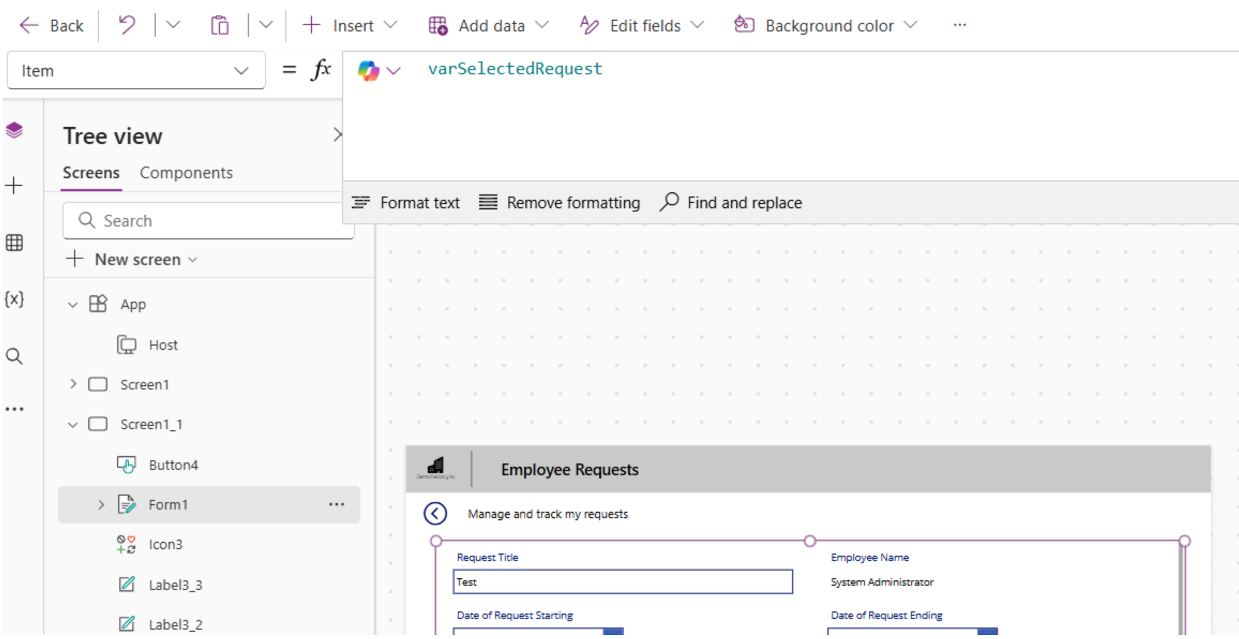


4. Go back to your second screen with your form. On the form there is a property called 'Item'.



Edit the Item property to the following

```
varSelectedRequest
```



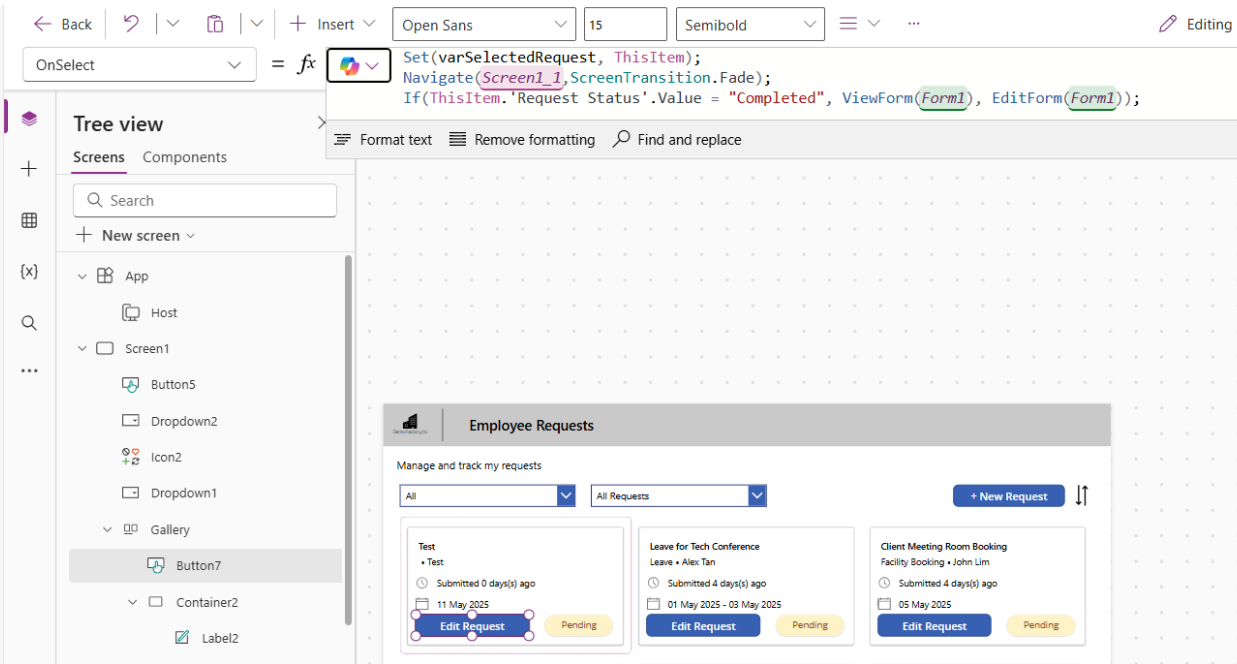
Control & Property	Old value	New Value
Card | Update	`DataCardValue7.Text`	*`If(Form1.Mode = FormMode.New, ComboBox1.Selected.Mail, varSelectedRequest.'Approver Assigned')`*
Submit | OnSelect	`Same`	*`Add Back(ScreenTransition.Fade) to end of the other commands`*

--- **End of Task** ---

Task 2: Preventing Users from editing completed Requests

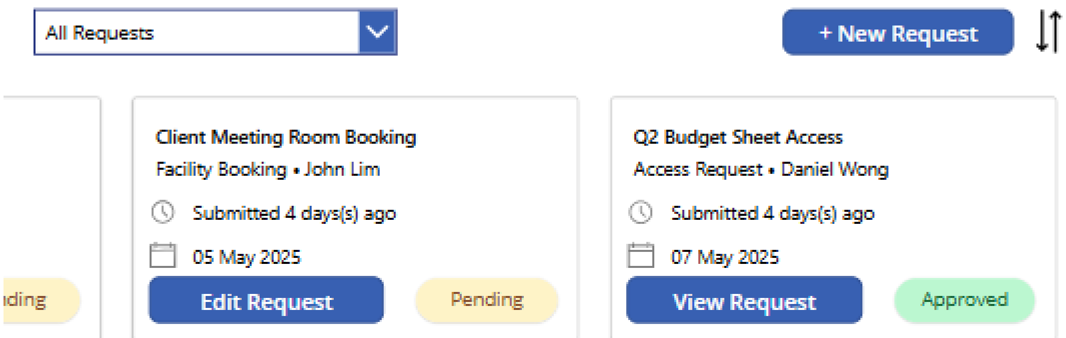
1. Change the OnSelect Property for the 'Edit Request' Button to this expression.

```
Set(varSelectedRequest, ThisItem);
Navigate(Screen1_1,ScreenTransition.Fade);
If(ThisItem.'Request Status'.Value = "Completed", ViewForm(Form1),
EditForm(Form1));
```

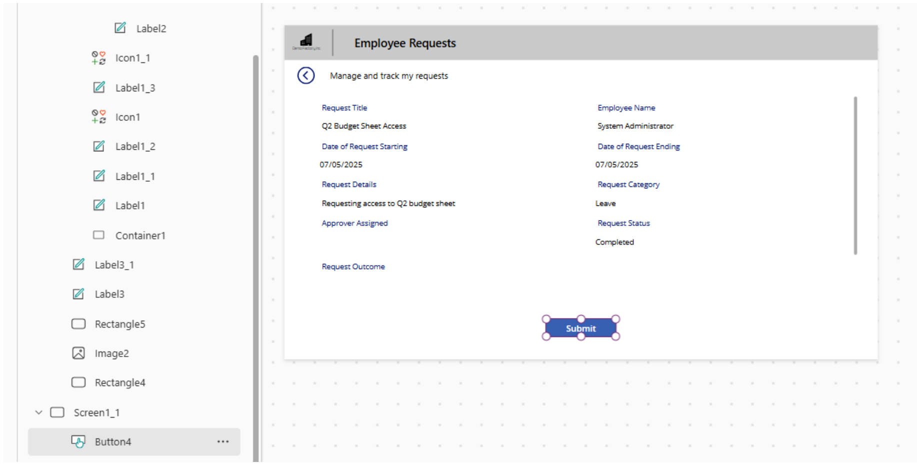


2. Change the text property of the button to the following value

```
If(ThisItem.'Request Status'.Value = "Completed", "View Request", "Edit
Request")
```

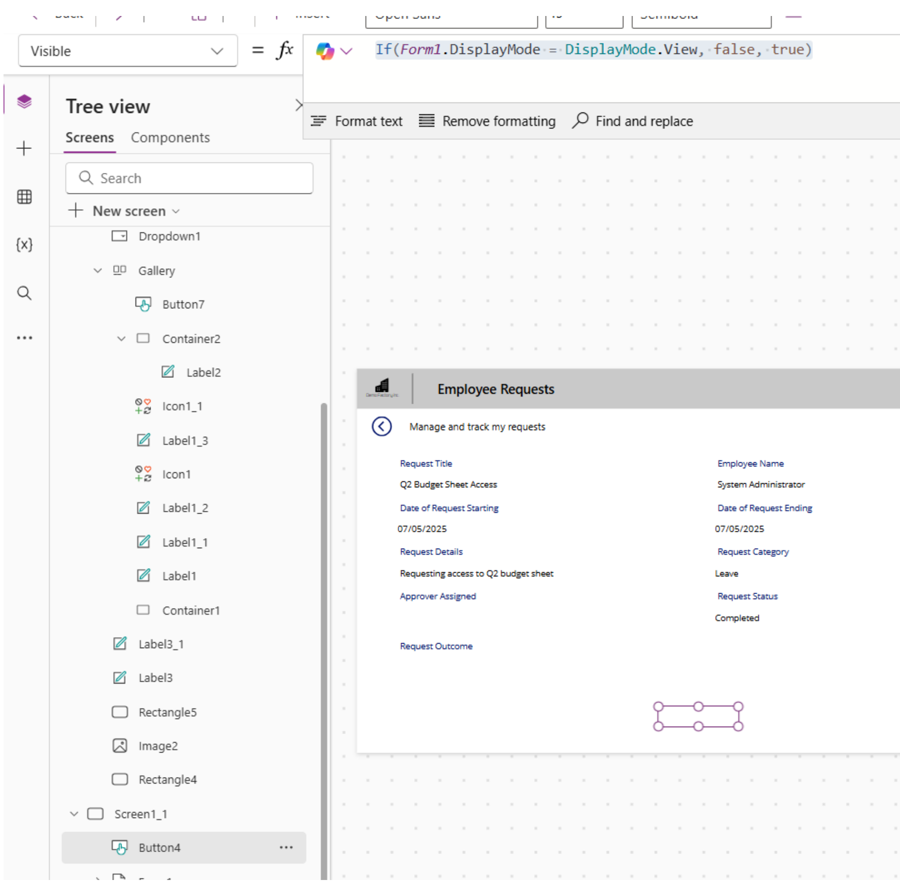


3. Select the “submit” button on the Form page.



Change the visible property of the button to the following value

```
If(Form1.DisplayMode = DisplayMode.View, false, true)
```



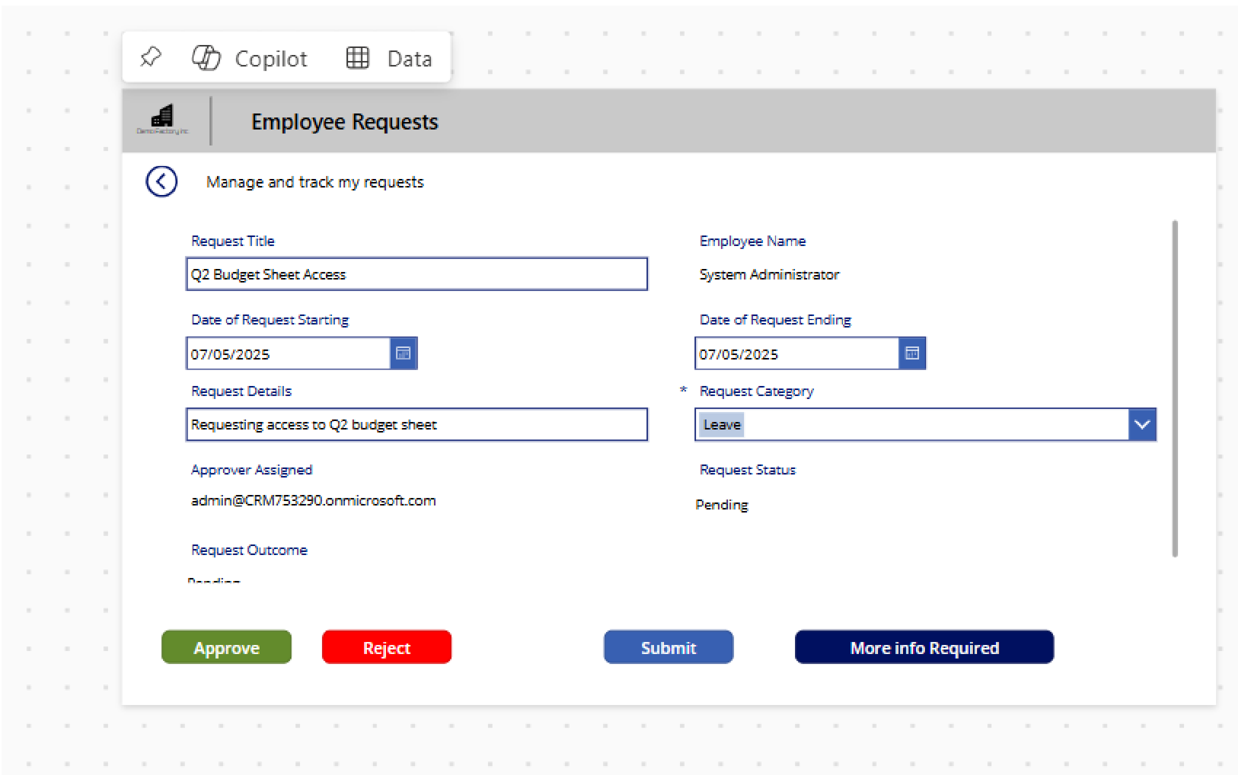
--- **End of Task** ---

Exercise 7: Enabling Approvers to take actions on a request

Approvers need to be able to review a request & take actions such as approve or deny.

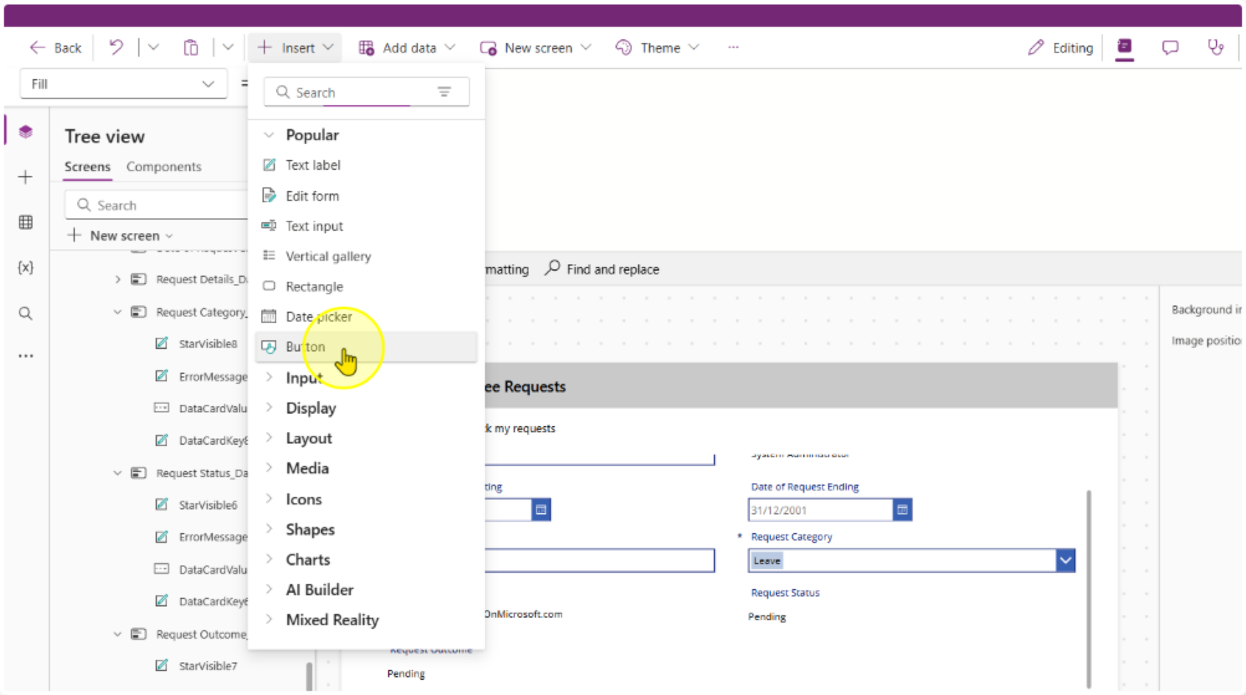
Objectives

- Approvers should be able to 'Approve', 'Reject', or 'Ask for more information'
- Only the Approver of the Request should be able to see this

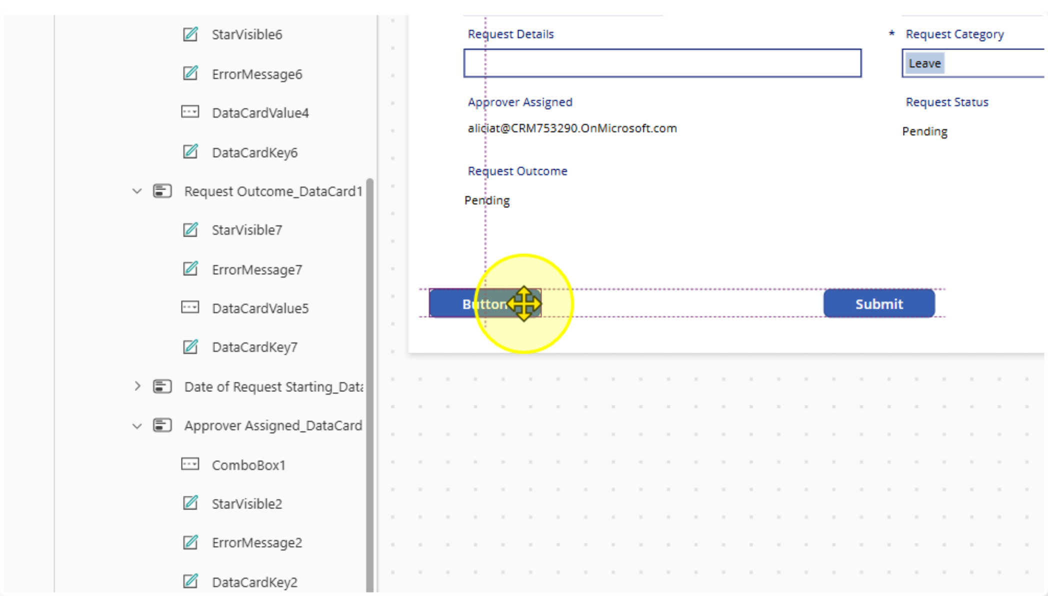


Task 1: Creating Buttons for Approvers and their other functions

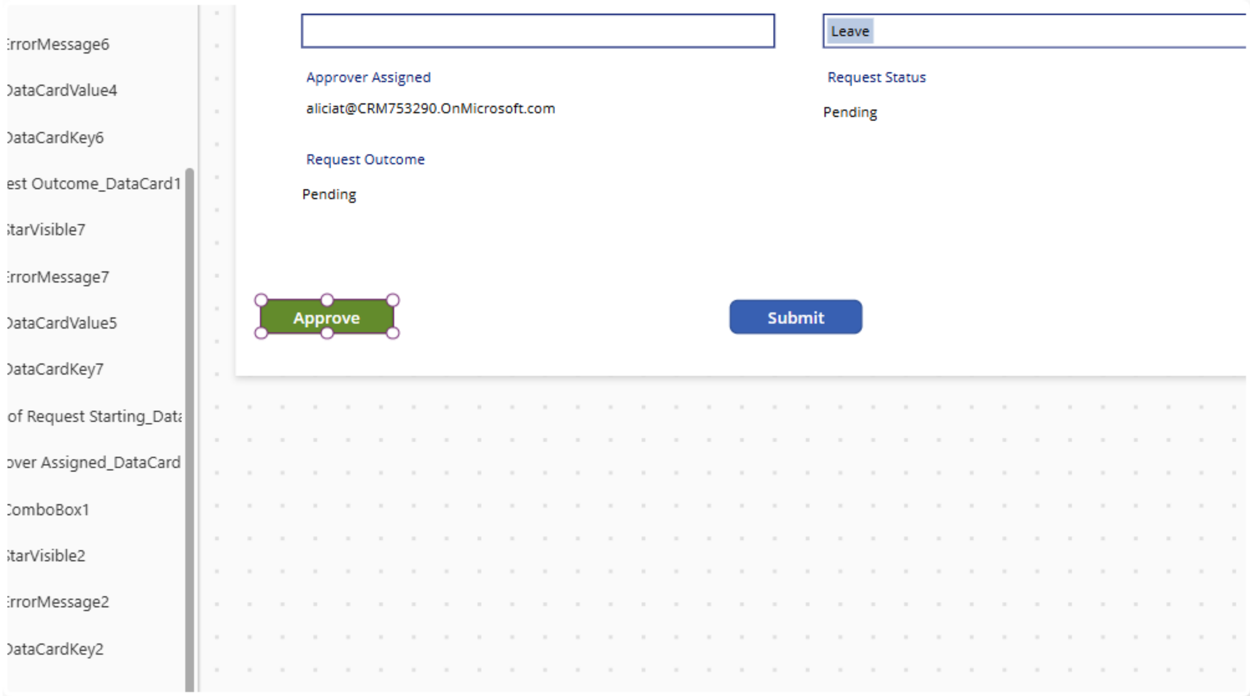
1. Insert in a new button control on the second screen



2. Position it as needed

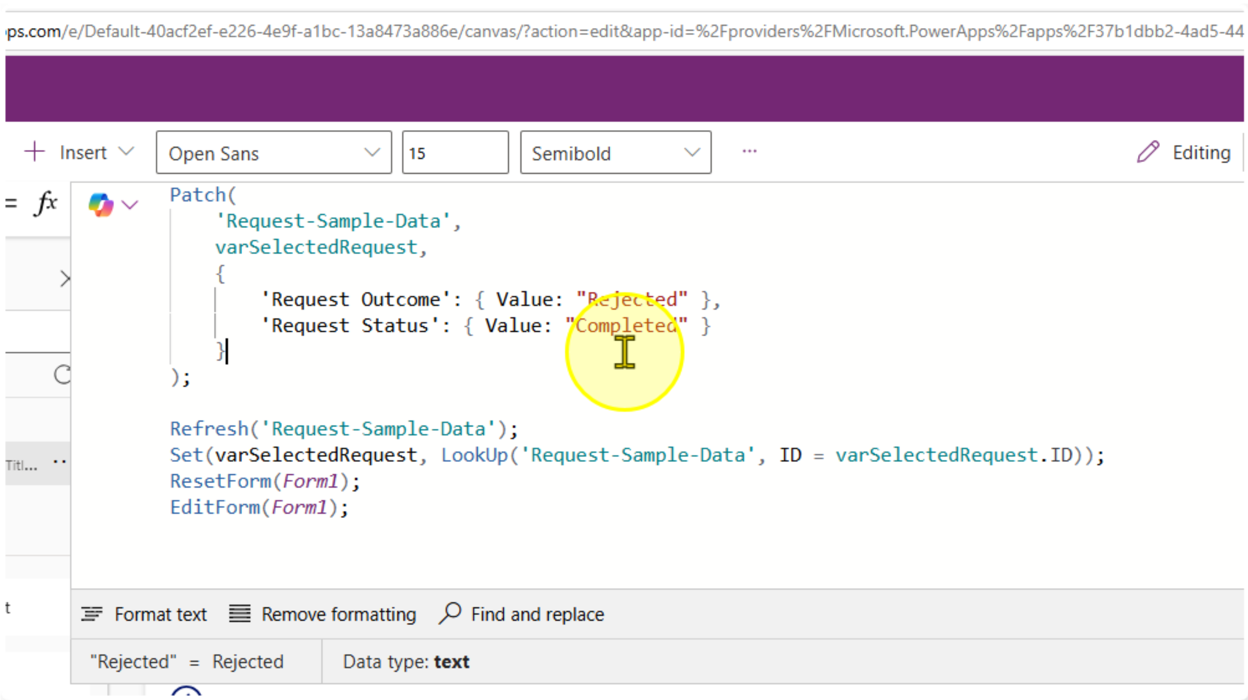


3. Style and position button appropriately. Change the Text property to "Approve" and the fill color to green.

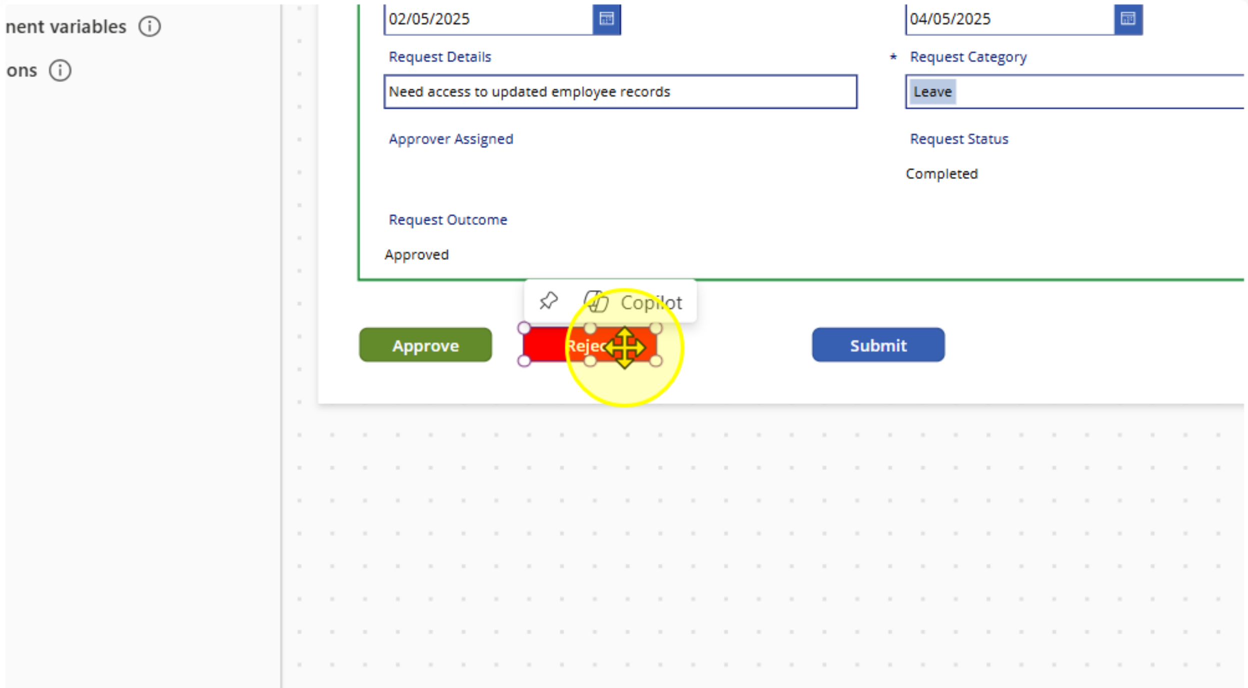


4. Change the 'OnSelect' property of the button control to the following value

```
Patch(
    'Request-Sample-Data',
    varSelectedRequest,
    {
        'Request Outcome': { Value: "Approved" },
        'Request Status': { Value: "Completed" }
    }
);

Refresh('Request-Sample-Data');
Set(varSelectedRequest, LookUp('Request-Sample-Data', ID =
varSelectedRequest.ID));
ResetForm(Form1);
EditForm(Form1);
```

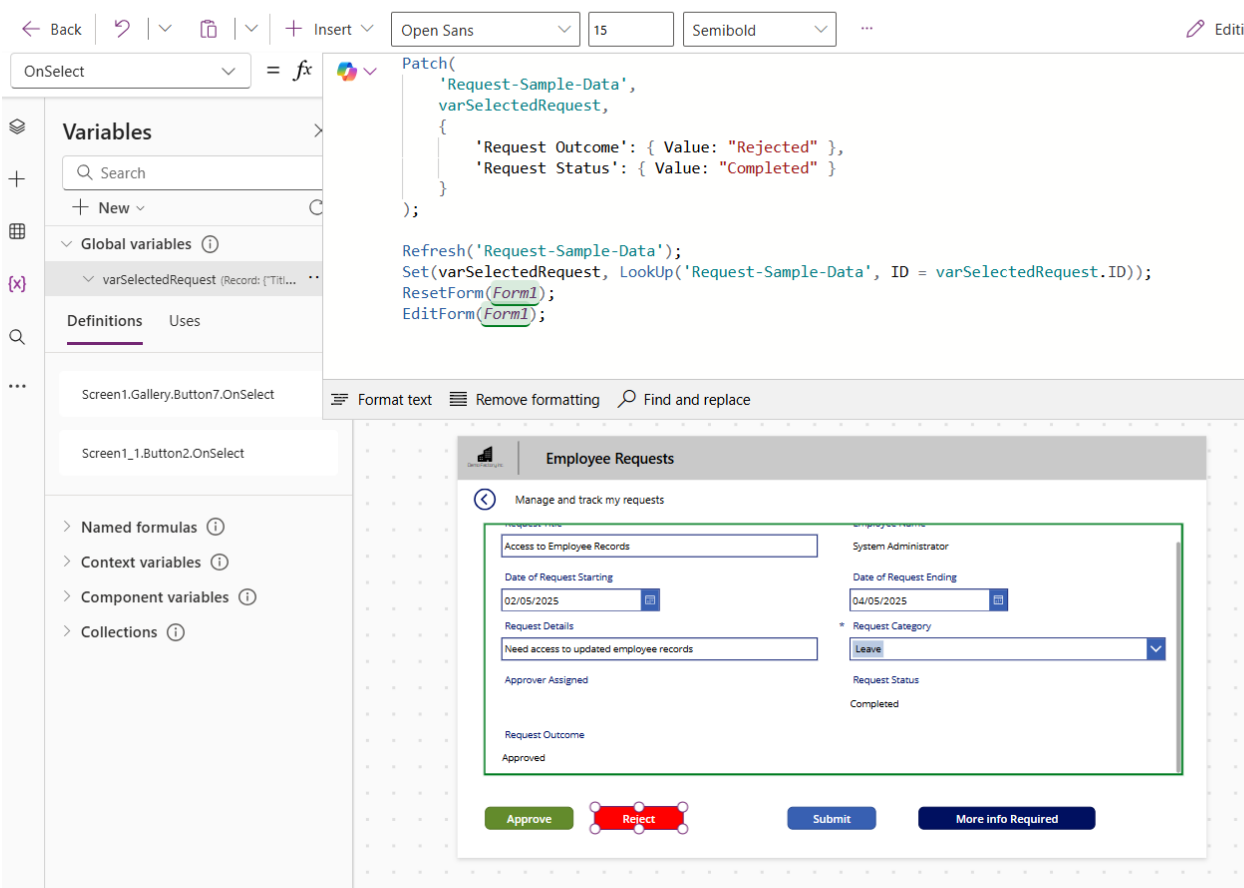


5. Copy the Submit button control and create a new one. Change the text property to ‘Reject’ and the fill color to Red.

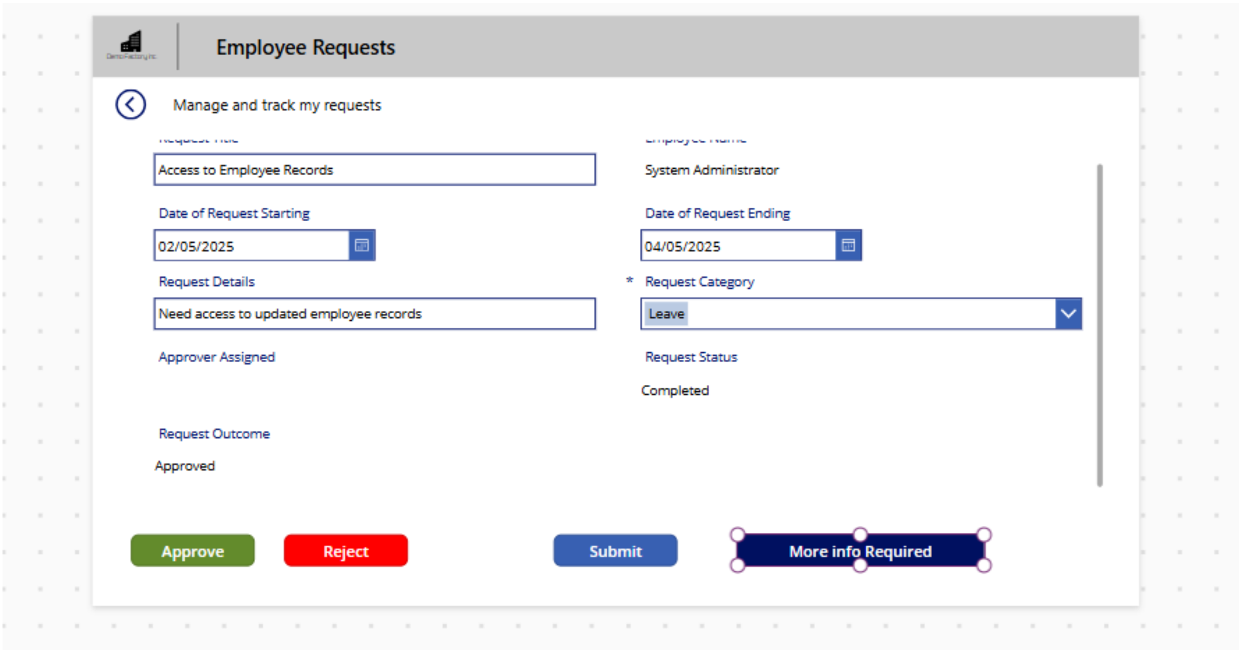


6. Change the OnSelect Property to the following

```
Patch(
    'Request-Sample-Data',
    varSelectedRequest,
    {
        'Request Outcome': { Value: "Rejected" },
        'Request Status': { Value: "Completed" }
    }
);

Refresh('Request-Sample-Data');
Set(varSelectedRequest, LookUp('Request-Sample-Data', ID =
varSelectedRequest.ID));
ResetForm(Form1);
EditForm(Form1);
```

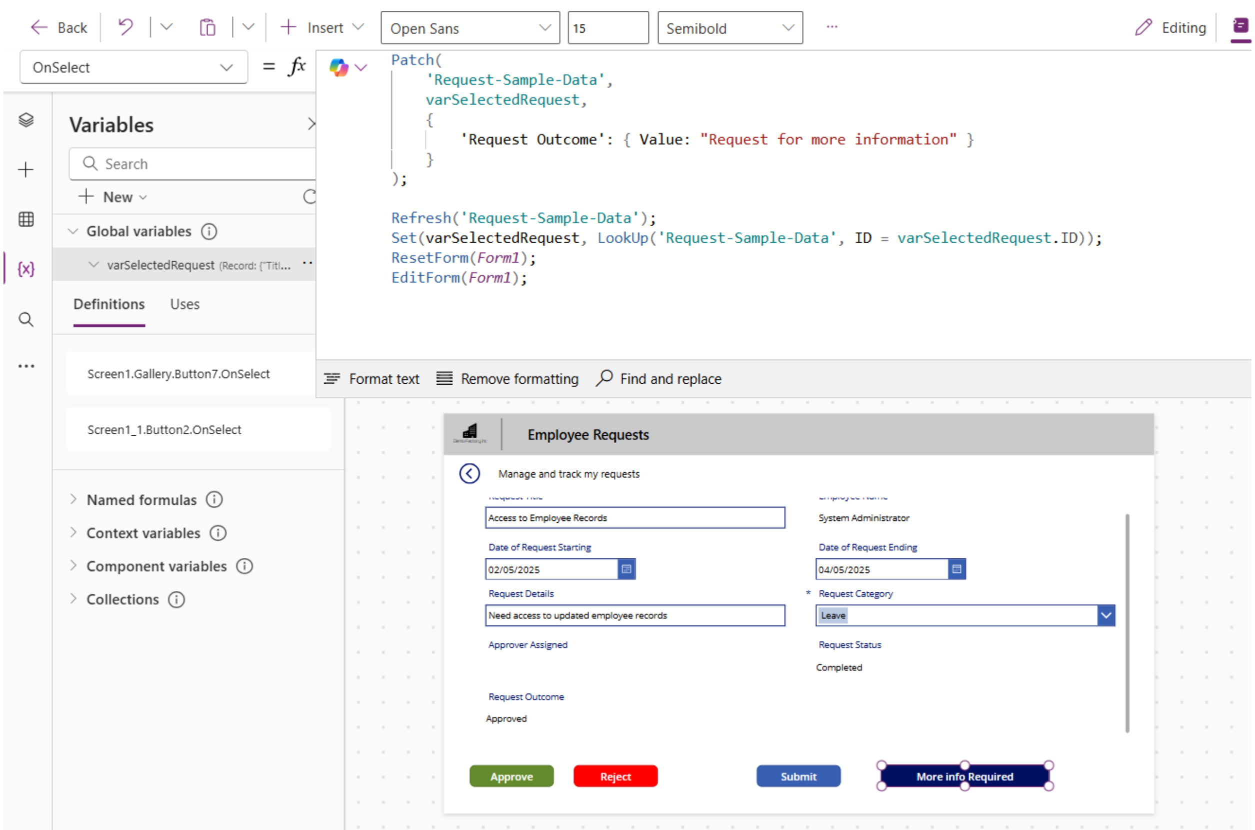


7. Copy and paste the reject button. Change the text property to "more info required" and the fill color to any fill color of your choice



8. Change the Onselect property to the following value

```
Patch(
    'Request-Sample-Data',
    varSelectedRequest,
    {
        'Request Outcome': { Value: "Request for more information" }
    }
);

Refresh('Request-Sample-Data');
Set(varSelectedRequest, LookUp('Request-Sample-Data', ID =
varSelectedRequest.ID));
ResetForm(Form1);
EditForm(Form1);
```

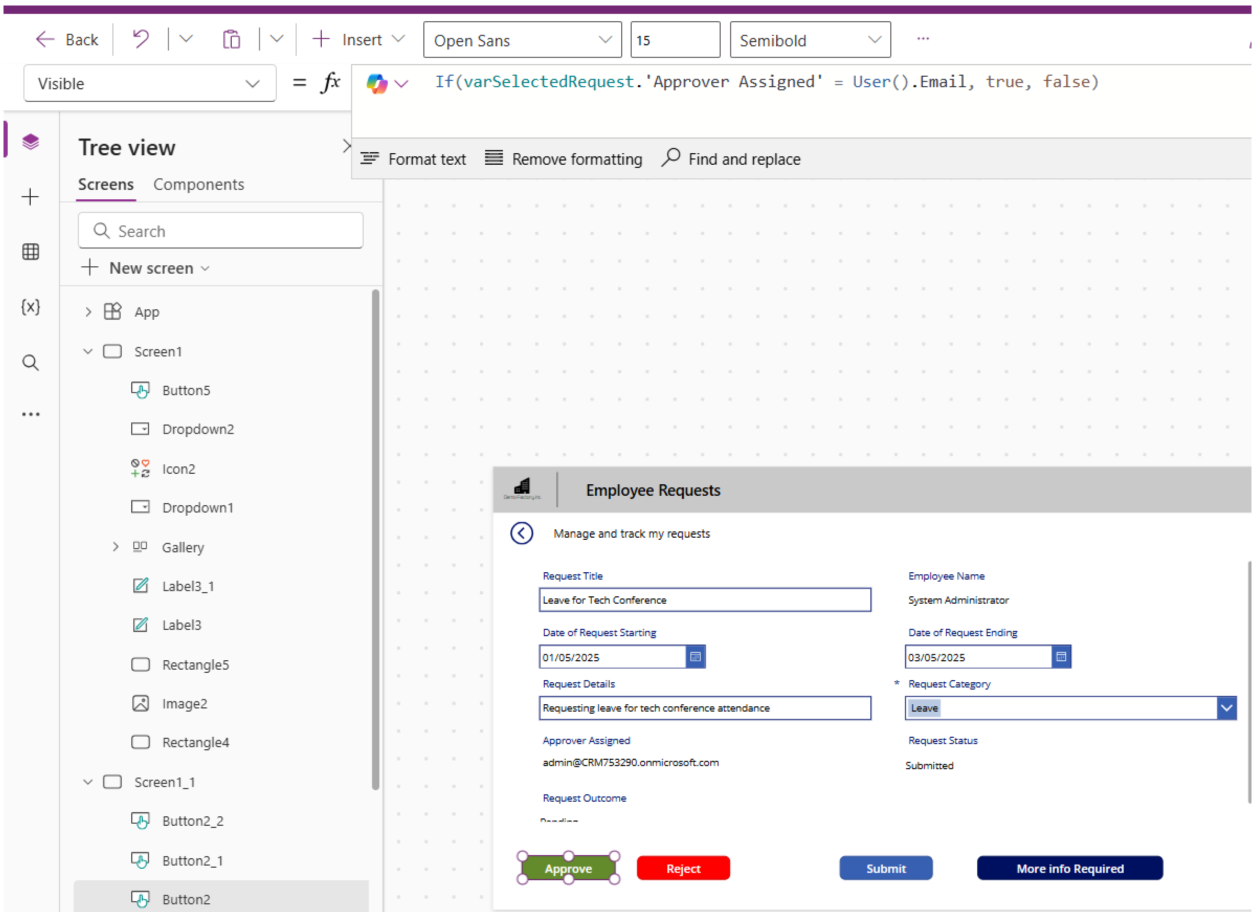


-- **End of Task** --

Task 2: Hiding Buttons for Requestors and showing it for approvers only

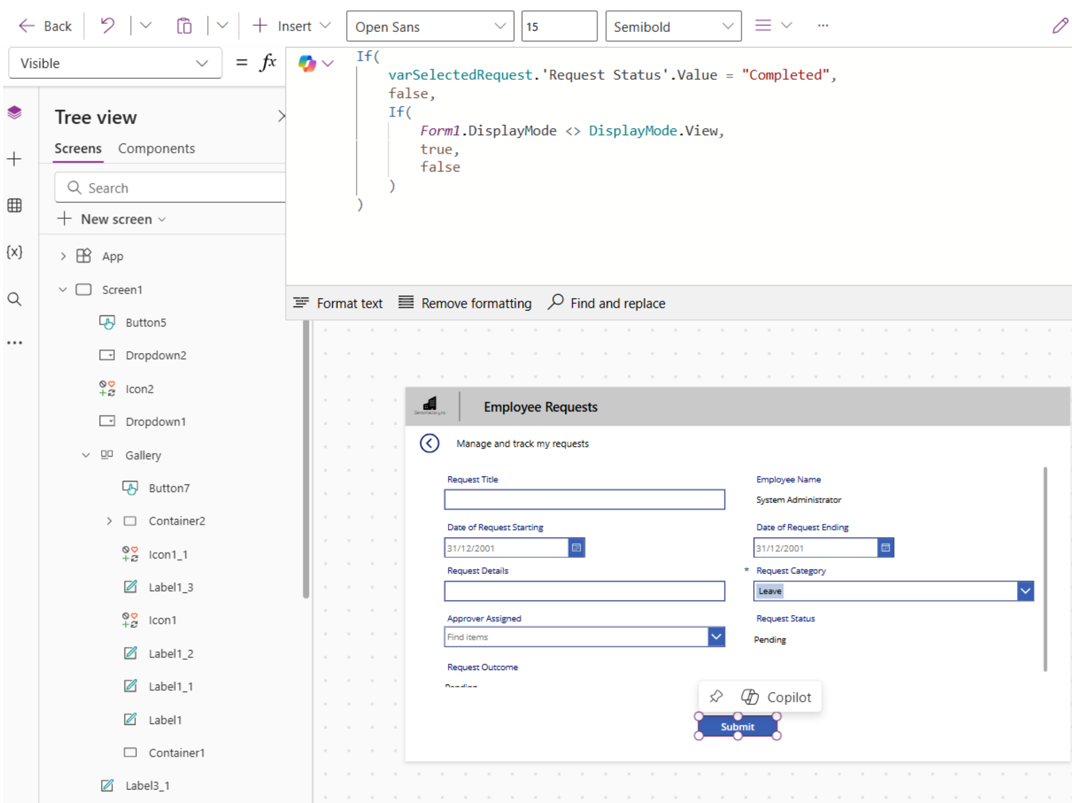
1. Change the onvisible property of all buttons to the following value.

```
If(
    varSelectedRequest.'Request Status'.Value = "Completed",
    false,
    (varSelectedRequest.'Approver Assigned' = User().Email ||
varSelectedRequest.'Approver Assigned' =
Office365Users.MyProfileV2().mail) && Form1.Mode <> FormMode.New)
```



2. Change the Visible Property of the Submit Button to the following

```
If(
    varSelectedRequest.'Request Status'.Value = "Completed",
    false,
    If(
        Form1.DisplayMode <> DisplayMode.View,
        true,
        false
    )
)
```



-- **End of Task** --

Task 3: Format, Default and Validation enhancements for Form (Optional)

Control | Control Type - Sub Control	Property	New Value
Date of Request Starting_DataCard1 | DatePicker - DataCardValue2 Date of Request Ending_DataCard1 | DatePicker – DataCardValue3	`DefaultDate` `DefaultDate`	`Today()` `Today() + 1`
Date of Request Starting_DataCard1 | DatePicker - DataCardValue2 Date of Request Ending_DataCard1 | DatePicker – DataCardValue3	`Format` `Format`	`"dd-mmm-yyyy"` `"dd-mmm-yyyy"`
Date of Request Starting_DataCard1 | DatePicker - DataCardValue2 Date of Request Ending_DataCard1 | DatePicker – DataCardValue3	`Width` `Width`	`Parent.Width – 60` `Parent.Width - 60`
Request Details_DataCard1 | Text Input - DataCardValue5	`Mode` `Height`	`Multiline` `90`

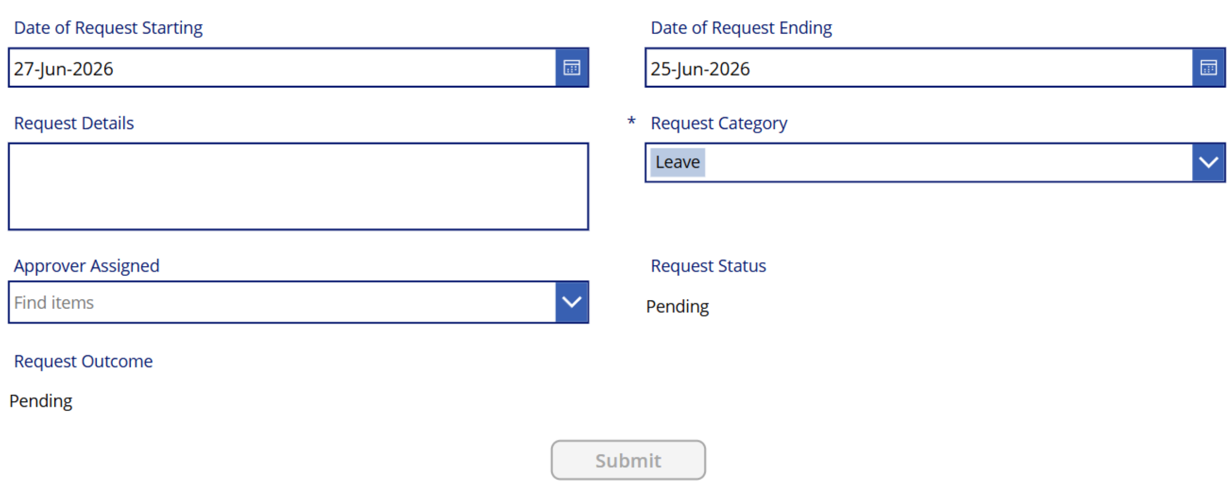
For more validation control before submission, change the DisplayMode property of the Submit button. From the below

```
If(Form1.Valid, DisplayMode.Edit, DisplayMode.Disabled)
```

Change to

```
If(Form1.Valid && DataCardValue2.SelectedDate <=
DataCardValue3.SelectedDate, DisplayMode.Edit, DisplayMode.Disabled)
```

See Submit button in disabled mode when Date of Request Starting is more than Date of Request Ending



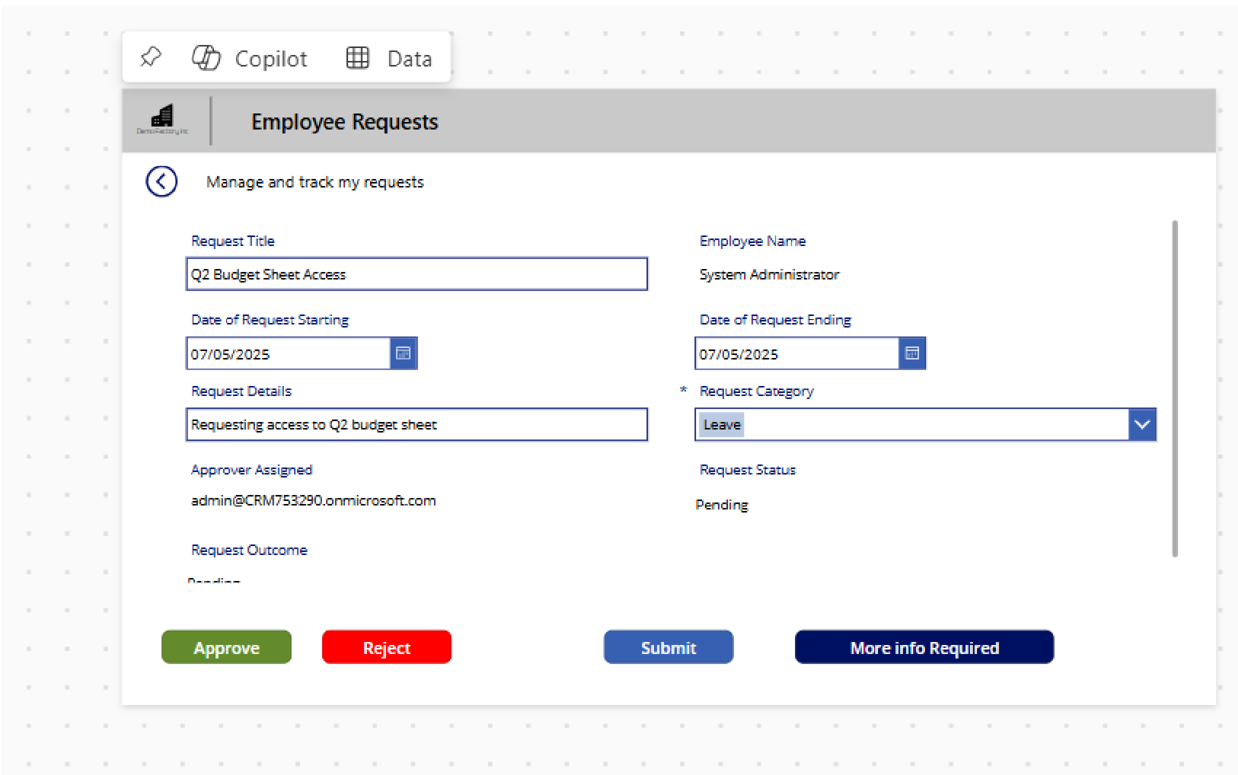
--- **End of Task** ---

Exercise 8: (Optional) Send Teams Notifications

For new requests submitted, send a notification via Teams using Power Automate workflow.

Objectives

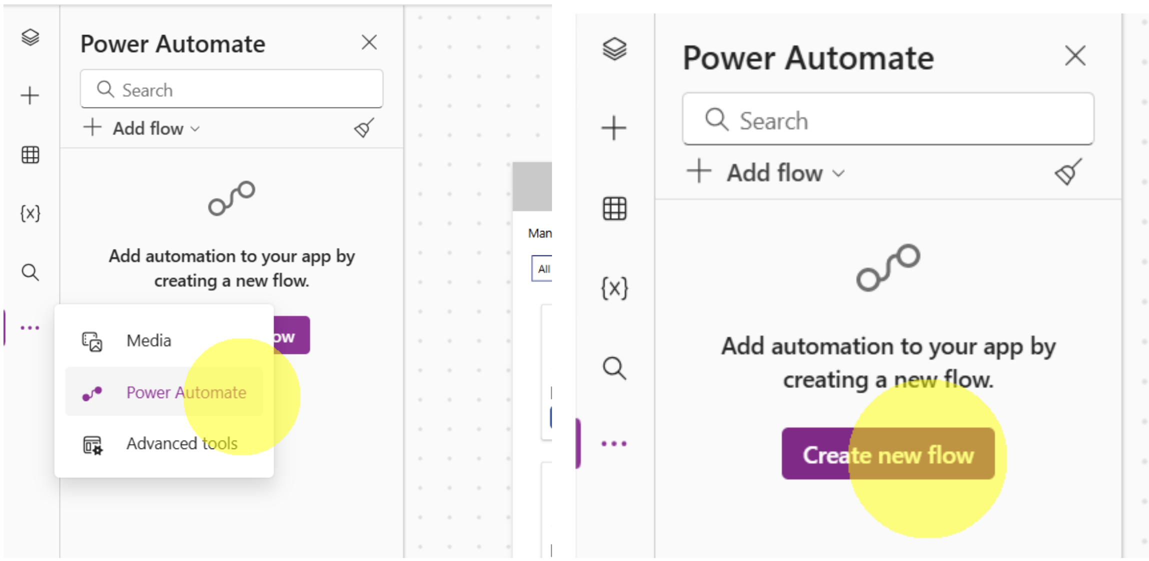
- Teams notifications to be sent out when there is a new request submitted
- Learn how to call Power Automate workflow from Power App when a button is clicked



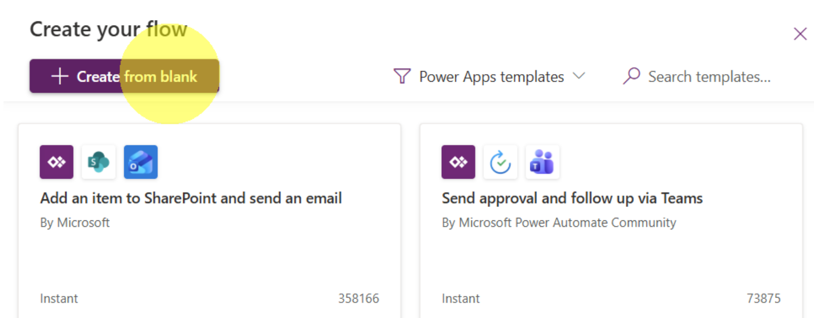
Microsoft Power Platform

Task 1: Creating a Power Automate Flow to Send Teams Notification

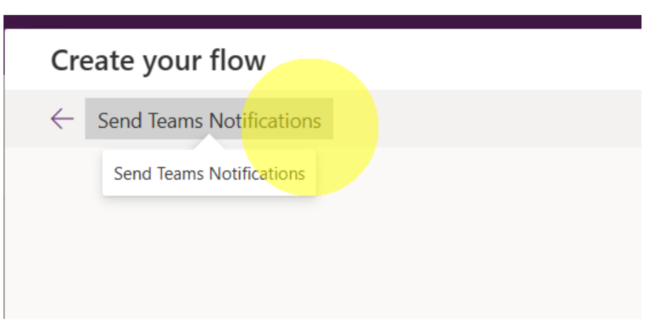
1. Click on Power Automate from left-hand menu and click “Create new flow”.



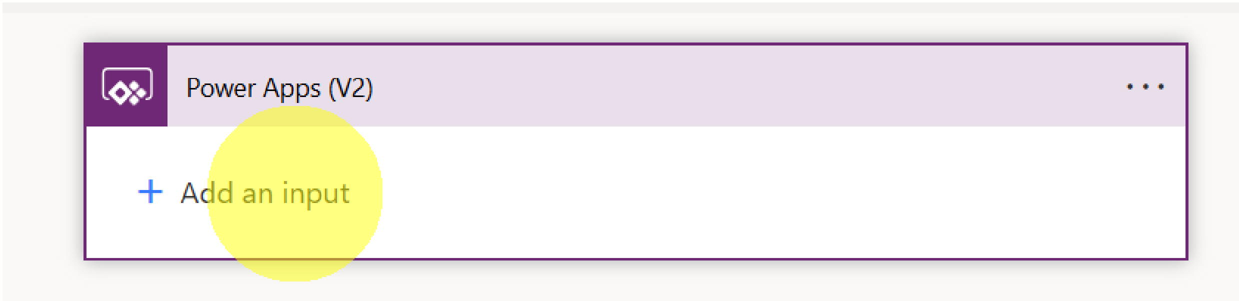
2. Select “Create from blank”.

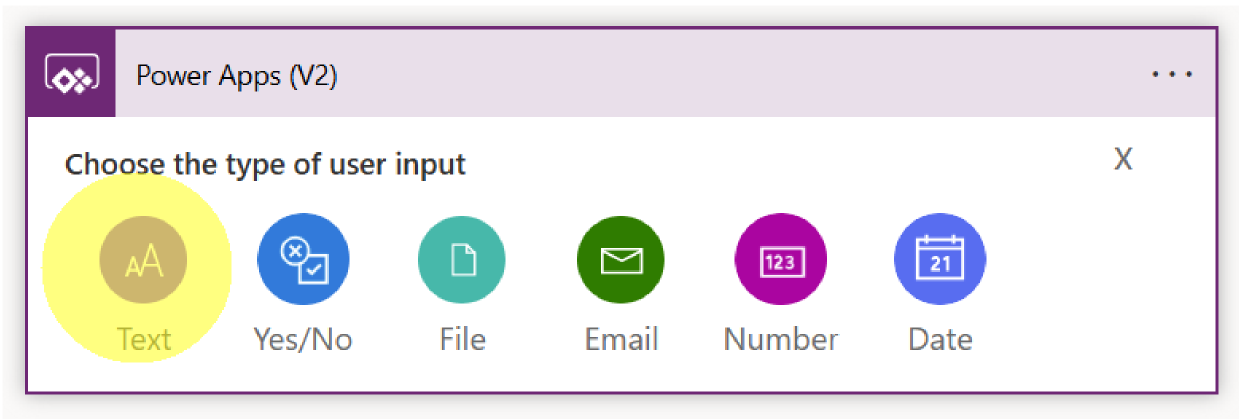


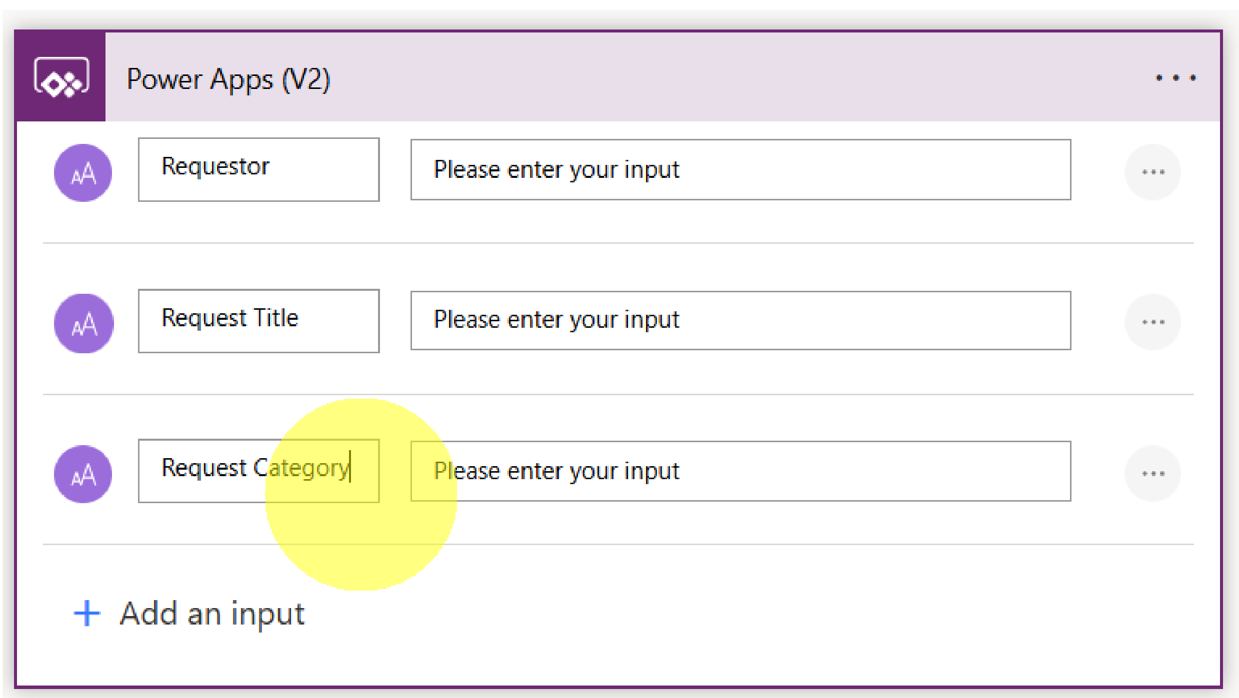
3. Rename the flow to “Send Teams Notifications”.



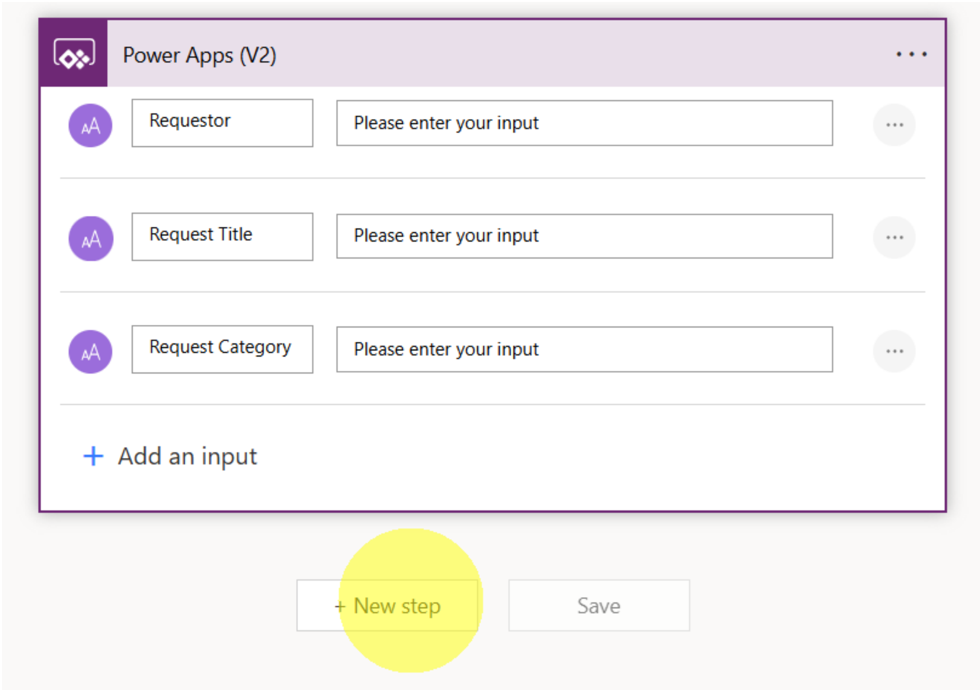
4. Expand Power Apps (V2) step and create 3 inputs as below.

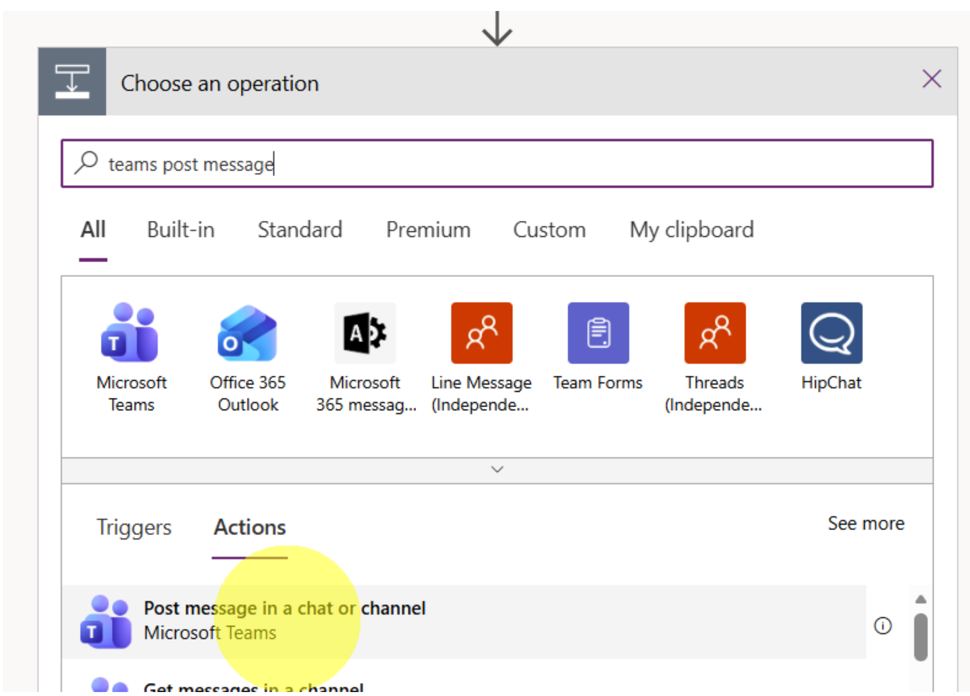




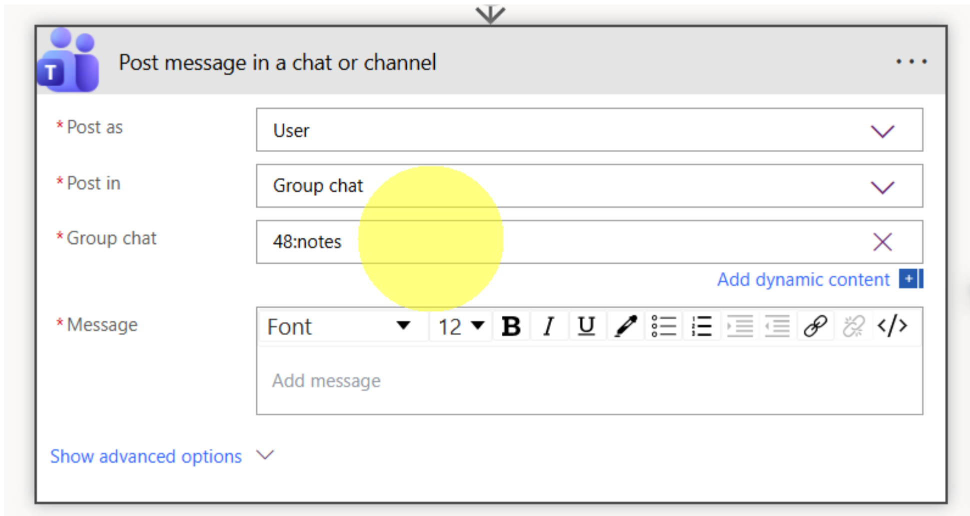


5. Click "New Step" button to add a new step and add Teams → Post message in a chat or channel action.





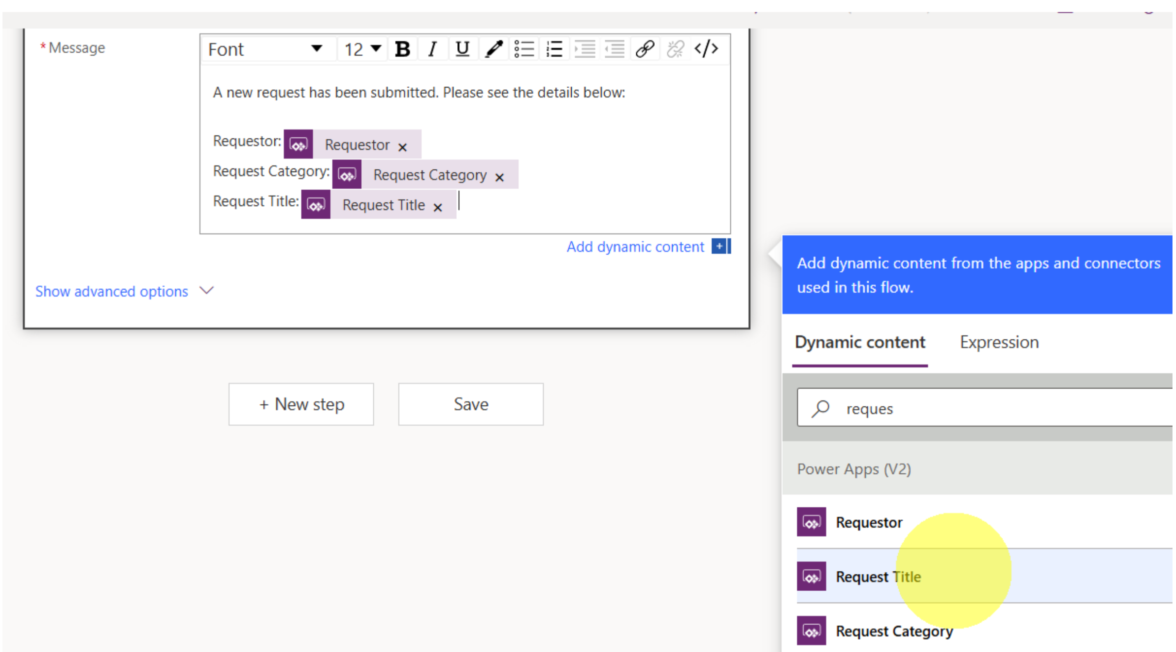
6. Select the Inputs as below to send message to yourself.



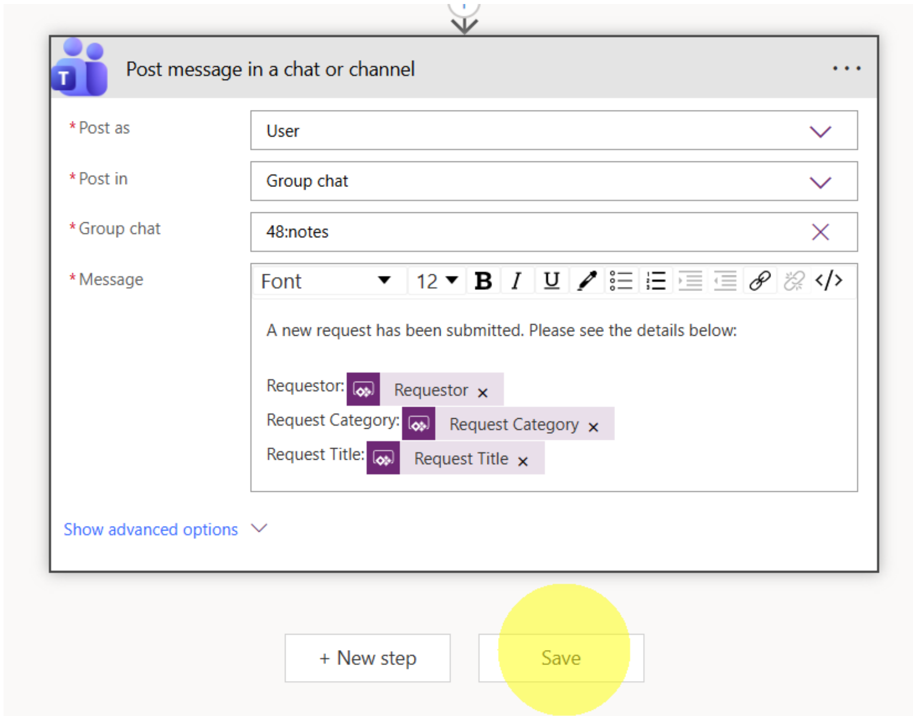
7. Type below in Message input. For the variables, select from the pop-up on the right

A new request has been submitted. Please see the details below:

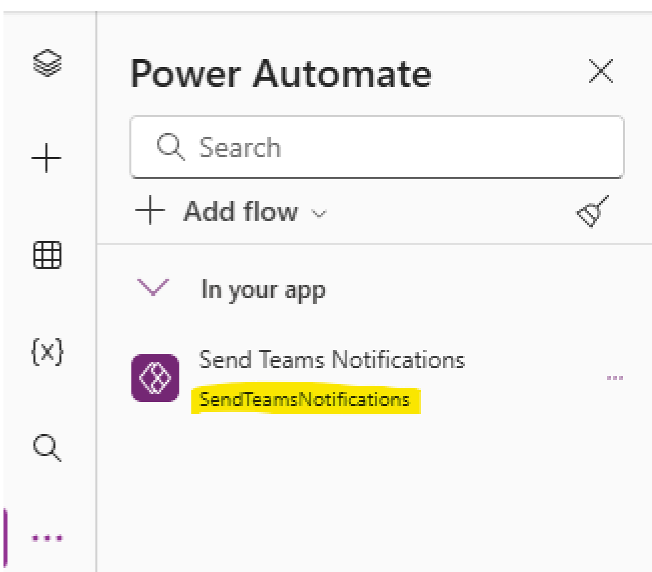
Requestor: <Select>
Request Category: <Select>
Request Title: <Select>



8. Save the flow by clicking on "Save" button.



You will need to add the newly created flow to your app by clicking on … for more options, select Power Automate > Add flow and select the flow shown. Remember highlighted name

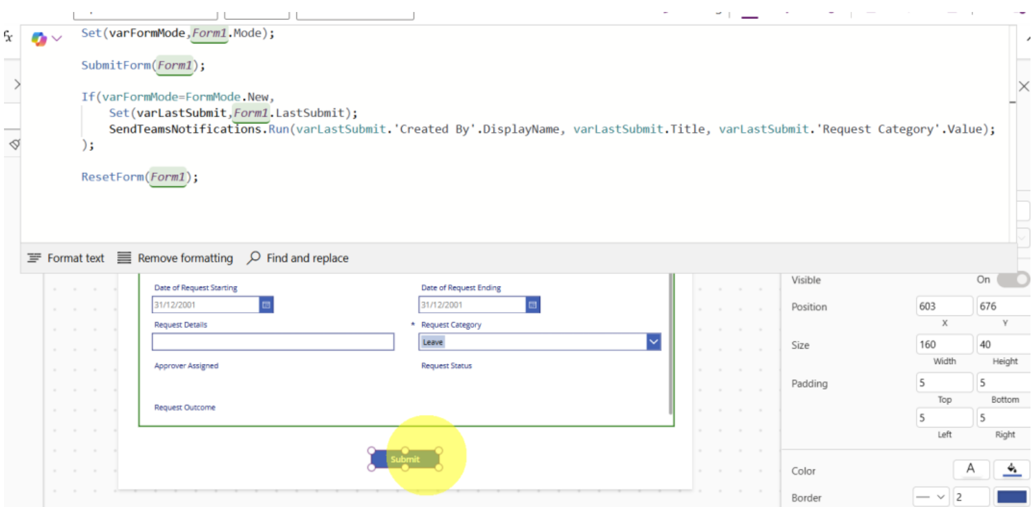


--- **End of Task** ---

Task 2: Call the Flow from Submit Button

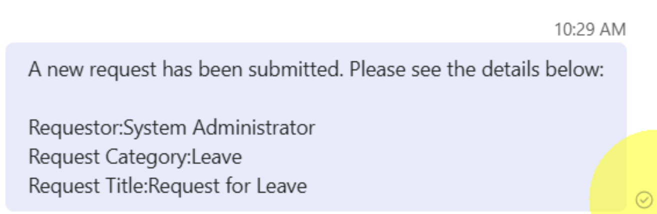
1. Change the OnSelect property of Submit Button to below to incorporate calling of Power Automate flow.

```
Set(varFormMode,Form1.Mode);

SubmitForm(Form1);

If(varFormMode=FormMode.New,
    Set(varLastSubmit,Form1.LastSubmit);
    SendTeamsNotifications.Run(varLastSubmit.'Created By'.DisplayName,
varLastSubmit.Title, varLastSubmit.'Request Category'.Value);
);

ResetForm(Form1);
```



When you submit a new request, you will receive a message that will be sent via Power Automate flow.

2. Change the Visible Property of the Submit Button to the following



--- **End of Task** ---